



MOST25 Intelligent Network Interface Controller

Features

- Complete MOST[®] Network Interface on a single chip
- Embedded MOST Network management
 - Network protection mode
 - Hardware and application watchdog timer
 - Intelligent muting
- Media Local Bus (MediaLB[®]) Port
 - Supports all MOST data types
 - Eases inter-chip communication and streaming
 - MediaLB 3-pin interface at speeds up to 1024×Fs
 - Legacy MediaLB 5-pin interface at speeds up to 512×Fs
- Control Port supports I²C format for control messages and asynchronous packets
- Streaming Port supports up to four serial interfaces for synchronous data
- Dedicated application reset output
- External switching power supply synchronization output
- 44-pin QFP or ETQFP package
- Operating voltages 3.3/2.5 V

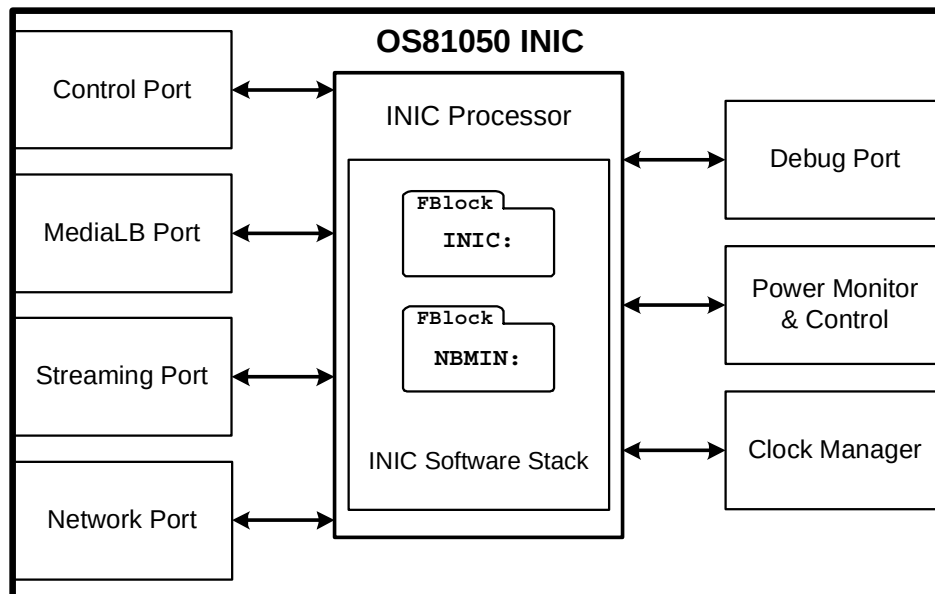
General Description

The OS81050 is a highly integrated *Intelligent Network Interface Controller* (INIC) for the 24.5 Mbps MOST Network (MOST25) with an optical physical layer (oPHY).

INIC provides encapsulation of all low-level functions necessary to develop a MOST-compliant device, significantly simplifying MOST implementation in a node. Supervision of the application is also provided, including a protection mode that is entered when an application is not present (i.e. start-up) or the External Host Controller (EHC) malfunctions. This protection mode prevents application malfunctions from influencing the integrity of the network and the system.

INIC includes the *INIC Software Stack*, which provides MOST-compliant real-time behavior. Integration of the *INIC Software Stack* significantly relieves the EHC from real-time processing tasks.

INIC makes a MOST device accessible according to the rules of the MOST Specification, even without participation from the EHC. When the EHC is engaged, a message-based interface, as opposed to a register-based interface, is available for communication with INIC. Configuration is encapsulated within a Function Block (FBlock INIC) which allows the EHC to manage INIC in the same way as other functions in the system.



Ordering Information

Order Number	Package
OS81050AQ	44-pin QFP
OS81050AQR	44-pin QFP, Tape and Reel
OS81050AH	44-pin ETQFP
OS81050AHR	44-pin ETQFP, Tape and Reel

This table represents valid part numbers at the time of printing and may not represent parts that are currently available. For the latest list of valid ordering numbers for this product, please contact the nearest sales office.

Further Information

For more information on SMSC automotive products, including integrated circuits, software, and MOST development tools and modules, visit our web site: <http://www.smisc-ais.com>. Direct contact information is available at: <http://www.smisc-ais.com/offices>.

SMSC Europe GmbH

Bannwaldallee 48
D-76185 Karlsruhe
Germany

SMSC

80 Arkay Drive
Hauppauge, New York 11788
USA

Technical Support

Contact information for technical support is available at: <http://www.smisc-ais.com/contact>.

Legend

Copyright © 2003-2010 SMSC. All rights reserved.

Ensure that all information within a document marked as 'Confidential' or 'Confidential Controlled Access' is handled solely in accordance with the agreement pursuant to which it is provided, and is not reproduced or disclosed to others without the prior written consent of SMSC. The confidential ranking of a document can be found in the footer of every page. This document supersedes and replaces all information previously supplied. The technical information in this document loses its validity with the next edition. Although the information is believed to be accurate, no responsibility is assumed for inaccuracies. Specifications and other documents mentioned in this document are subject to change without notice. SMSC reserves the right to make changes to this document and to the products at any time without notice. Neither the provision of this information nor the sale of the described products conveys any licenses under any patent rights or other intellectual property rights of SMSC or others. There are a number of patents and patents pending on the MOST technology and other technologies. No rights under these patents are conveyed without any specific agreement between the users and the patent owners. The products may contain design defects or errors known as anomalies, including but not necessarily limited to any which may be identified in this document, which may cause the product to deviate from published descriptions. Anomalies are described in errata sheets available upon request. SMSC products are not designed, intended, authorized or warranted for use in any life support or other application where product failure could cause or contribute to personal injury or severe property damage. Any and all such uses without prior written approval of an officer of SMSC will be fully at your own risk. TrueAuto is a trademark and MediaLB, SMSC and MOST are registered trademarks of Standard Microsystems Corporation ("SMSC"). Other names mentioned may be trademarks of their respective holders.

SMSC DISCLAIMS AND EXCLUDES ANY AND ALL WARRANTIES, INCLUDING WITHOUT LIMITATION ANY AND ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, AND AGAINST INFRINGEMENT AND THE LIKE, AND ANY AND ALL WARRANTIES ARISING FROM ANY COURSE OF DEALING OR USAGE OF TRADE. IN NO EVENT SHALL SMSC BE LIABLE FOR ANY DIRECT, INCIDENTAL, INDIRECT, SPECIAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES; OR FOR LOST DATA, PROFITS, SAVINGS OR REVENUES OF ANY KIND; REGARDLESS OF THE FORM OF ACTION, WHETHER BASED ON CONTRACT; TORT; NEGLIGENCE OF SMSC OR OTHERS; STRICT LIABILITY; BREACH OF WARRANTY; OR OTHERWISE; WHETHER OR NOT ANY REMEDY OF BUYER IS HELD TO HAVE FAILED OF ITS ESSENTIAL PURPOSE, AND WHETHER OR NOT SMSC HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Conventions

Within this manual, the following abbreviations and symbols are used to improve readability.

Example	Description
BIT	Name of a single bit within a field
FIELD.BIT	Name of a single bit (BIT) in FIELD
x...y	Range from x to y, inclusive
BITS[m:n]	Groups of bits from m to n, inclusive
PIN	Pin Name
msb, lsb	Most significant bit, least significant bit
MSB, LSB	Most significant byte, least significant byte
zzzzb	Binary number (value zzzz)
0xzzz	Hexadecimal number (value zzz)
zzh	Hexadecimal number (value zz)
rsvd	Reserved memory location. Must write 0, read value indeterminate
code	Instruction code, or API function or parameter
<i>Multi Word Name</i>	Used for multiple words that are considered a single unit, such as: <i>Resource Allocate</i> message, or <i>Connection Label</i> , or <i>Decrement Stack Pointer</i> instruction.
<i>Section Name</i>	Section or Document name.
$\overline{\text{VAL}}$	Over-bar indicates active low pin or register bit
x	Don't care
<Parameter>	<> indicate a Parameter is optional or is only used under some conditions
{,Parameter}	Braces indicate Parameter(s) that repeat one or more times.
[Parameter]	Brackets indicate a nested Parameter. This Parameter is not real and actually decodes into one or more real parameters.

TrueAuto™

TrueAuto is SMSC's automotive quality process. It has proven its ability to deliver leading-edge quality and services for IC device products to fulfill the needs of the most demanding automotive customers. TrueAuto is a proven total automotive-grade quality approach. TrueAuto IC device robustness begins with SMSC's design for reliability techniques within the silicon IC itself: automotive-grade robustness and testability are designed into the IC. Once available in silicon, the IC is fully-characterized and qualified over a multitude of operating parameters to prove quality under the harshest conditions. In this, SMSC's TrueAuto approach significantly exceeds the usual automotive reliability standards and customer- specific requirements and goes far beyond the stress tests prescribed by the AEC-Q100 specifications. During the fabrication of TrueAuto products, extensive technologies and processes, such as enhanced monitors are used in order to continuously drive improvements in accordance with SMSC's zero Defects per Million (DPM) goals.

TABLE OF CONTENTS

1 OVERVIEW	7
1.1 MOST25 Network	8
1.1.1 Control Network	8
1.1.2 Asynchronous (Packet) Network	8
1.1.3 Synchronous Streaming Network	8
1.2 INIC Processor	9
1.3 Hardware Ports	10
1.3.1 Network Port	10
1.3.2 MediaLB Port	10
1.3.3 Streaming Port	11
1.3.4 Control Port	11
1.3.5 Port Expansion	12
1.4 MOST NetServices	13
2 PINOUT	15
2.1 Pinout List	15
2.2 Pinout	18
2.3 Equivalent Schematics for Pins	19
3 INIC PROCESSOR	21
3.1 INIC Message Interface	22
4 PORT MESSAGE PROTOCOL	23
4.1 Generic Port Messages	23
4.2 FIFO Status and Command Messages	24
4.3 FIFO Data Messages	25
4.4 FIFO Message Exchange	26
4.4.1 MOST Control Messages (MCM)	26
4.4.2 INIC Control Messages (ICM)	26
4.4.3 MOST Data Packets (MDP)	27
5 CONTROL PORT	28
5.1 Writing to the Control Port	29
5.2 Reading from the Control Port	30
6 MEDIALB PORT	31
6.1 MediaLB 3-pin Mode	32
6.2 MediaLB 5-pin Mode	37
6.3 MediaLB Commands and Responses Not Supported	39
6.4 MediaLB Debug Header	39
7 STREAMING PORT	40
7.1 Left-Justified Alignment	43
7.2 Right-Justified Alignment	43
7.3 Delayed-Bit Alignment	44
7.4 Sequential Alignment	44

8 EXTERNAL POWER MANAGEMENT	45
9 CLOCK MANAGER	47
9.1 PLL Lock Status	47
9.2 FLT Pin	48
9.3 Crystal Pins (XTI/XTO)	48
10 JTAG PORT	49
10.1 Instruction Register	50
10.2 Identification Register	50
11 ELECTRICAL CHARACTERISTICS	51
11.1 Absolute Maximum Ratings	51
11.2 Guaranteed Operating Conditions	51
11.3 Thermal Characteristics	51
11.4 DC Characteristics	52
11.5 Switching Characteristics	53
11.5.1 Clocks	53
11.5.2 Reset and Power Management	54
11.6 Network Characteristics	55
11.7 MediaLB Port (3-pin)	56
11.8 MediaLB Port (5-pin)	58
11.9 Streaming Port	59
11.10 Control Port	60
11.11 JTAG Timing	61
12 PACKAGE OUTLINES	62
12.1 QFP44	62
12.2 ETQFP44	63
13 APPLICATION INFORMATION	64
13.1 Power	64
13.1.1 External Regulator	64
13.1.2 Internal Regulator	65
13.2 Physical Layer	66
13.3 Reset	66
13.4 Other Application Pins	67
13.5 Configuration and Debug	68
13.5.1 EHC via Control Port	68
13.5.2 INIC Explorer via JTAG Port	69
13.6 Crystal Oscillator Selection	70
13.7 Layout Guidelines	71
13.7.1 Power and Decoupling	73
13.7.2 Physical Layer	73
13.7.3 MediaLB Interfaces	73
13.7.4 Thermal Considerations	74
13.7.5 Miscellaneous	74
APPENDIX A: REFERENCES	75
APPENDIX B: REVISION HISTORY	76

APPENDIX C: LIST OF FIGURES 77

APPENDIX D: LIST OF TABLES 79

1 Overview

The OS81050 is a network transceiver device belonging to the SMSC *Intelligent Network Interface Controller* (INIC) family. The OS81050 INIC (also known as INIC25) supports a network data rate of 64 bytes/frame (22.579 Mbps at 44.1 kHz; 24.576 Mbps at 48 kHz - also known as MOST25) when interfacing to a *Media Oriented Systems Transport* (MOST) Network. Any reference to MOST in this data sheet implies MOST compliance at the MOST25 data rate.

All relevant network management functions are handled on-chip, providing a complete system interface to the physical-layer components. Minimal additional components are required due to the high-level of integration. An on-chip PLL with ultra-low jitter guarantees accurate audio and video transmission and clock recovery over a wide frequency range.

A typical MOST Network node would consist of the physical-layer devices connected to the MOST Network, the OS81050 to handle the low-level protocols, and an *External Host Controller* (EHC) for the mid- to high-level functions. This architecture eliminates the need for the user to implement the lower levels of the *MOST Specification* [1] protocol, thereby drastically shrinking development time. Network management functions are off-loaded from the programmer, allowing full concentration on the application being developed. Figure 1-1 illustrates the OS81050, with the matching protocol stack implementation.

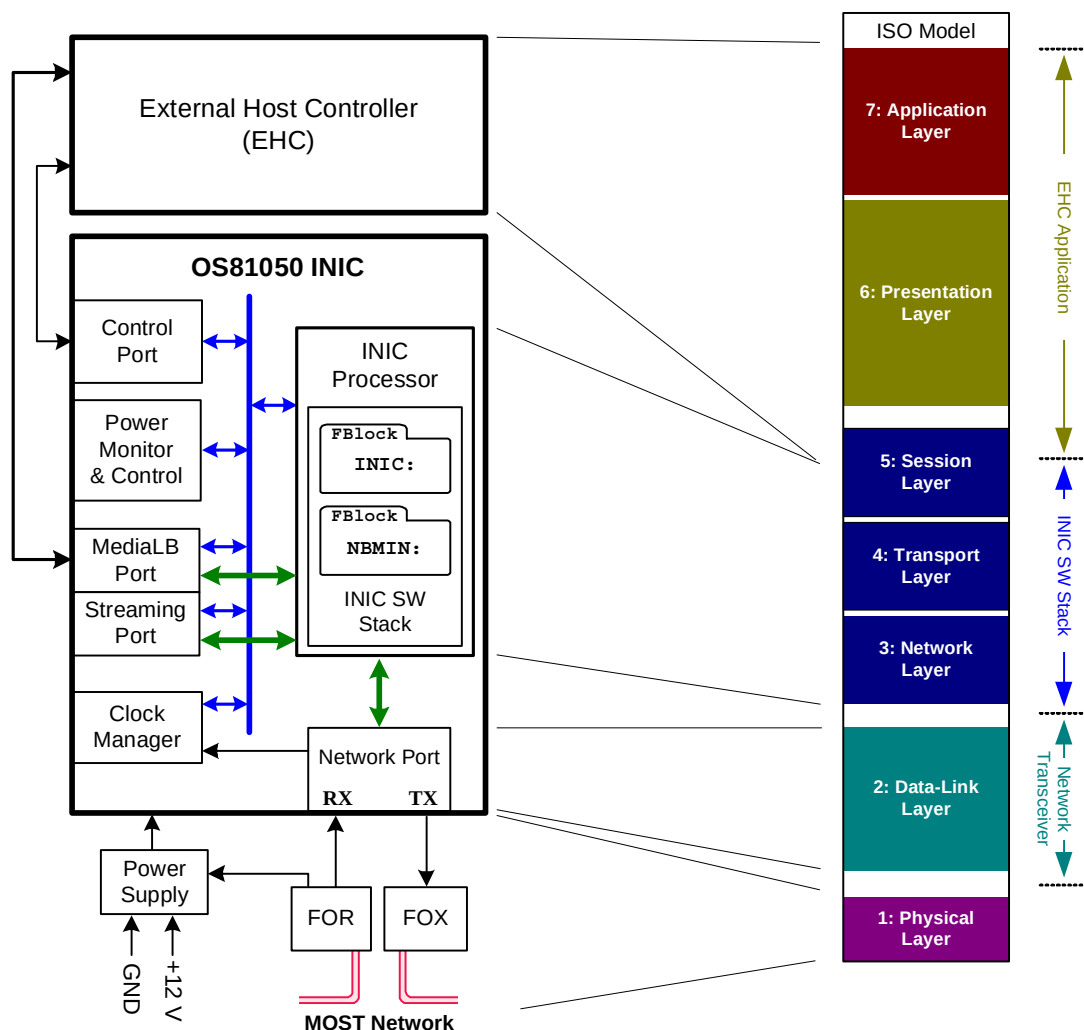


Figure 1-1: INIC Hardware/System Overview

1.1 MOST25 Network

To minimize costs, the MOST Network supports a peer-to-peer methodology, eliminating excessive hardware overhead such as a hub (although hub-based architectures can also be implemented). In addition to handling network interface and communication management functions, the OS81050 INIC also handles all of the important low-level network management functions; such as node position sensing (Plug-and-Play), network delay detection, start up and shut down. Other features include: error reporting, fail-safe operation, and channel allocation. Placing these features in hardware off-loads the EHC, allowing it to focus on higher-level network functions.

The MOST Network contains multiple simultaneous networks operating across a single medium. Once bandwidth is allocated, each network (or *data transfer method*) operates independently (not affecting the others), providing a robust, dependable, and deterministic system architecture. The OS81050 INIC supports the following data transfer methods, including:

- Control Network - consists of a single *control channel*
- Asynchronous (Packet) Network - consists of a single *packet channel*
- Synchronous Streaming Network - consists of one or more *synchronous streaming channels*

1.1.1 Control Network

The *MOST Control Network* consists of a single control channel, shared by all network nodes. Messages transported across this channel adhere to the *MOST Control Message* (MCM) format and are used to manage the network and exchange control data among nodes. Any network node that needs to transmit a control message must arbitrate for the channel.

INIC monitors the control channel on the Network Port by default and uses information in the message header to determine whether the node is the intended receiver of the control message. When a control message is to be received (e.g. node address match), it is buffered within INIC. For MCMs, the application is notified of the message reception and the packet is retrieved by the EHC via one of the OS81050 hardware ports that supports control message exchange (see Table 1-1).

1.1.2 Asynchronous (Packet) Network

The *MOST Asynchronous Network* consists of a single packet channel, shared by all network nodes. Applications use this channel to transmit large data packets (e.g. navigation maps) in bursts to other network nodes. Data packets on the packet channel are referred to as *asynchronous packets* and adhere to the *MOST Data Packet* (MDP) format. Any network node that needs to transmit a data packet can arbitrate for the packet channel.

INIC monitors the packet channel on the Network Port by default and uses information in the packet header to determine whether the node is the intended receiver. When an asynchronous packet is to be received (e.g. node address match), it is buffered within INIC and made available to the application at one of the OS81050 hardware ports that supports asynchronous packets (see Table 1-1).

1.1.3 Synchronous Streaming Network

The *MOST Synchronous Streaming Network* can consist of multiple synchronous streaming channels, each transporting raw, real-time synchronous data (e.g. audio, video). The data on a synchronous streaming channel is sourced by a single transmitting node that has been granted channel bandwidth. Synchronous streaming channels are broadcast, making the data available for reception by one or more sink nodes.

For high-speed synchronous data, INIC acts like a cross-point switch to connect synchronous streaming channels to sources/sinks attached to OS81050 hardware ports that support synchronous data (see Table 1-1).

1.2 INIC Processor

The INIC Processor manages the transfer of data to and from the network. The Network Port connects the INIC Processor to the network using the OS81050 network transmission and reception pins (**TX** and **RX**). These Network Port reception and transmission pins require connection to external physical layer circuitry: a Fiber Optic Transmitter (FOX) driving an LED on the transmit side and a Fiber Optic Receiver (FOR) with a photodiode on the receive side.

Working in conjunction with the Clock Manager, the Network Port recovers the network clock, decodes received data, and passes the data to the INIC Processor. The INIC Processor then routes data to the appropriate destinations on and off the chip.

The network frame to be transmitted by the OS81050 is constructed by the INIC Processor by combining incoming network data from the Network Port with outgoing data from other on-chip resources. The Network Port then encodes the data for network transmission.

The *INIC Software Stack* runs on the INIC Processor and configures the Network Port at power up, which allows the network to become fully operational without any external protocol stack. This has been accomplished by placing all the lower layers of the ISO protocol stack inside of the device. In addition to the *INIC Software Stack*, the INIC Processor supports two groups of functions: FBlock *NBMIN* and FBlock *INIC*. Figure 1-1 illustrates the position of the *INIC Software Stack* within the ISO software model.

Each network node contains a general set of functions, referred to as the FBlock *NetBlock* functions. INIC manages network-related *NetBlock* functions and the EHC manages the higher-level *NetBlock* functions. To discern between these two sections of the FBlock *NetBlock*, the INIC portion is referred to as *NBMIN* and the EHC portion is labeled *NBEHC*.

FBlock *INIC* contains functions used to configure the chip. This includes the setup of hardware ports and the utilization of network bandwidth for transporting streaming data.

The upper levels of the network protocol must be implemented by an EHC that is capable of processing the user application with the minimal overhead imposed by the INIC API. During real-time operation, the EHC and the *INIC Software Stack* within the INIC Processor communicate using control messages via the MediaLB Port or the Control Port.

1.3 Hardware Ports

Each OS81050 hardware port transports different network data types. Table 1-1 shows the data types that can be transported across each hardware interface.

Hardware Port	Control Messages*	Asynchronous Packets	Synchronous Data
Network	X	X	X
MediaLB*	X	X	X
Streaming			X
Control*	X	X	

* In addition to MCMs, Control and MediaLB Ports can exchange *INIC Control Messages* (ICMs). ICMs specifically target INIC and are used for internal configuration and control only (never routed to another hardware port or target a network node).

Table 1-1: Hardware Port Data Transport

Using the OS81050 API, a *socket* is created for each data stream transported across a hardware port. The specific data type to be exchanged is specified when calling the `CreateSocket()` function. Data routing between hardware ports is setup using the `ConnectSockets()` function, where sockets of like data types are linked together. For more information on sockets, refer to the *OS8105x/6x INIC API User's Manual* [2].

1.3.1 Network Port

The Network Port receiver (**RX**) is over-sampled at a high-frequency and data is recovered by a digital state machine. When the port is configured as a timing-slave, the OS81050 recovers the network clock, which the Clock Manager then uses to generate other internal clocks. When the port is configured as the timing-master, the internal clocks and the network clock are generated by the Clock Manager based on an external crystal (**XTI/XTO**). The Network Port transmitter (**TX**) works in conjunction with the INIC Processor to encode data for network transmission.

After reset, the OS81050 automatically interacts with the network, performing all necessary low-level network management functions. This enables the EHC to configure its interface to the OS81050 when the application is ready.

1.3.2 MediaLB Port

The MediaLB Port supports communication between the EHC and the OS81050 INIC by means of the Media Local Bus (MediaLB) protocol. MediaLB provides a low cost, easy to implement hardware interface that standardizes and simplifies network application development. MediaLB has one bus master, referred to as the *MediaLB Controller* (or simply *Controller*), which is always the OS81050. All other connected components (including the EHC) are known as *MediaLB Devices* (or simply *Devices*). MediaLB Device functionality is a subset of Controller functionality; the Controller (OS81050) functions as a Device when it is receiving data from other connected components.

The OS81050 supports the MediaLB protocol in either MediaLB 3-pin mode or MediaLB 5-pin mode. Both modes support the exchange of all network data types, including: control messages, asynchronous packets, and synchronous data. The transmitting Device sends data on a dedicated data line, while information about the data is simultaneously transported on an independent signal information line.

When using the MediaLB Port in 3-pin mode, the OS81050 output clock is configurable as 256×Fs (12.288 MHz at 48 kHz), 512×Fs (24.576 MHz at 48 kHz), or 1024×Fs (49.152 MHz at 48 kHz). When configured for 256×Fs, the OS81050 can transmit and receive up to 28 bytes (7 quadlets) per frame of data. When configured for 1024×Fs, the OS81050 can transmit and receive up to 124 bytes (31 quadlets) per frame of data. When using the MediaLB Port in 5-pin mode, the maximum bus clock frequency is 512×Fs (1024×Fs not supported).

MediaLB is a token-passing bus, where the Controller manages the token and generates the data clock. The Controller passes the token by sending out a *ChannelAddress* on the signal information line. The *ChannelAddresses* are pre-assigned by the board integrator and are associated with a specific transmitting Device and a receiving Device. Once the transmitting Device associated with the *ChannelAddress* is granted bus access, it can send one quadlet of data on the MediaLB data line after a defined time delay. The Device provides information about the data being transmitted by simultaneously sending out a command on the signal information line. After receiving the *ChannelAddress*, the receiving Device must decide whether to receive the associated message or reject it. A status byte from the receiving Device is placed on the signal information line, one byte after the command from the sending Device is received. Once per network frame, the Controller generates a unique pattern (*FRAMESYNC*) on the signal information line. This defines the MediaLB frame edge, as well as the byte boundary of signal and data lines for all Devices.

Since MediaLB is a high-speed bus, connections to external devices should be implemented in hardware and not *bit-banged* on generic ports. A MediaLB Device interface, in the form of VHDL code, is available from SMSC. For more information about the Media Local Bus, refer to the *MediaLB Specification* [3].

EHC devices without an integrated MediaLB interface can use one of SMSC's *I/O Companion* devices (IOCs) as a communication bridge between the OS81050 MediaLB Port and a parallel, SRAM-like EHC interface. For example, the IOC *Host Bus Interface* (HBI) is a 16-bit parallel communication interface for exchange of all network data types. This HBI Port is easily accommodated by EHCs with a generic SRAM port. In addition to serving as a communication bridge between its MediaLB and HBI Ports, the IOC provides hardware port expansion to the application via the MediaLB Port, as described in Section 1.3.5.

1.3.3 Streaming Port

The Streaming Port provides a gateway for synchronous data exchange between the OS81050 and external legacy devices. Depending on the configuration of the OS81050, the Streaming Port supports up to four serial data pins, which share a common synchronization signal and bit clock. The data format of the Streaming Port is compatible with industry-standard serial interfaces found on many ADCs, DACs and other devices.

1.3.4 Control Port

The Control Port operates as an I²C bus slave and supports asynchronous packet and control message exchange. An interrupt pin ($\overline{\text{INT}}$) is used to notify the EHC when Control Port servicing is required (e.g. control message or asynchronous packet received). While the EHC can communicate with the OS81050 over the Control Port alone, the I²C data rate limits throughput on the Application Layer. Application performance can be maximized by using the MediaLB Port to transport all network data types. Configuring and enabling of the MediaLB Port is supported via the I²C Control Port.

1.3.5 Port Expansion

Devices belonging to SMSC's *I/O Companion* (IOC) family can be used as companions to INIC, providing expanded hardware ports to the application. The OS85650 IOC, for example, is a high-performance routing engine that supports IO expansion and a *Digital Transmission Content Protection* (DTCP) coprocessor. The primary function of the OS85650 is routing of data streams between various hardware ports, while the integrated DTCP coprocessor provides optional encryption/decryption of data streams routed through the IOC device. Various services are available to support full *Authentication and Key Exchange* (AKE) implementation. For applications that do not require DTCP, the OS85652 IOC provides the same functionality as the OS85650, albeit without the DTCP coprocessor.

When the OS85650 IOC is used as a companion to the OS81050 INIC, the MediaLB bus is typically the medium used to exchange data between the two devices. Data exchanged over MediaLB is routed to another IOC hardware interface on which external devices may reside (e.g. ADC, DSP, EHC). The EHC for the application may reside on the IOC's *Host Bus Interface* (HBI), which provides an SRAM-like parallel port for easy communication with various microcontrollers. In this way, the IOC allows an EHC to exploit the efficiency and bandwidth of the MediaLB bus, even when the EHC does not support a MediaLB interface.

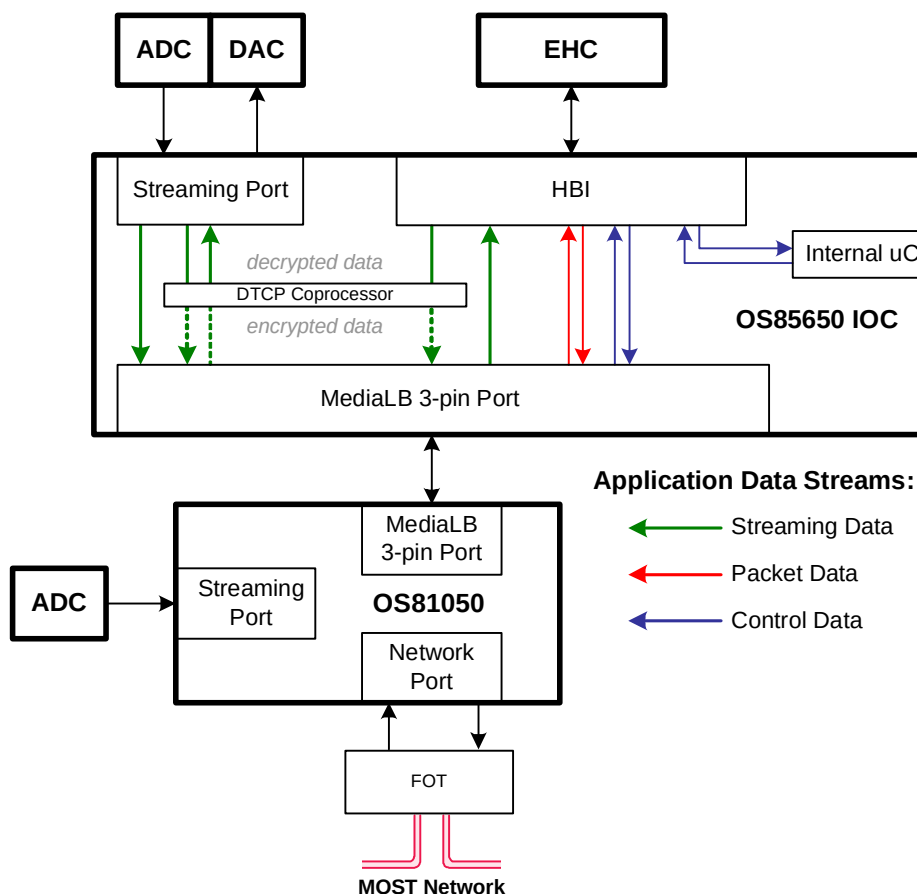


Figure 1-2: INIC-IOC Application Example

Contact SMSC for more information about the IOC family of devices, including the OS85650.

1.4 MOST NetServices

The division of services between the OS81050 INIC and the EHC in MOST applications is illustrated in Figure 1-3. To accelerate development of MOST applications that use INIC, SMSC offers the MOST NetServices API, which provides a seamless software interface between INIC and the EHC. A software library is available to provide all services that are relevant for:

- exchanging application data on the MOST Network,
- managing hardware port data connections (e.g. sockets),
- managing high-level system tasks, and
- using control messages to access INIC for specific operations.

The MOST NetServices code incorporated into the EHC is divided into two distinct components: *Basic Services* (Layer I) and *Applications Socket* (Layer II).

The *Basic Services* component provides the facilities that enable direct communication with INIC, including: a low-level driver for message exchange, message management services, and data wrappers. When transmitting data from the EHC to INIC, the wrapper modules pass data to the message management services. Formatted messages are then sent by the low-level driver across the interface between the EHC and INIC. When receiving data from INIC, the wrappers reformat the data for compatibility with the MOST NetServices code that resides on the EHC.

The *Applications Socket* component resides on top of *Basic Services* and provides the FBlock *NBEHC* services required to implement the full FBlock *NetBlock* on the node. This component also contains a command interpreter, which provides a simple API for developing application FBlocks (e.g. FBlock *AudioAmp*).

MOST NetServices code is modular, allowing it to be customized for a particular application. Implemented in ANSI C, the MOST NetServices API can be adapted for individual requirements through configuration files.

With respect to the ISO communications model (shown in Figure 1-1):

- External components (e.g. FOX and FOR) connect the OS81050 and the MOST Network, supporting the *Physical Layer*.
- OS81050 supports the *Data Link Layer* up to a portion of the *Session Layer*.
- The EHC must provide the remaining portion of the *Session Layer*, the *Presentation Layer*, and the *Application Layer*. When the MOST NetServices code is integrated:
 - the *Basic Services* component provides the remaining portion of the *Session Layer*, and
 - the *Application Socket* component provides the *Presentation Layer* and a portion of the *Application Layer*.

The *OS8105x/6x INIC API User's Manual* [2] defines the FBlocks (and all associated functions) that are supported by the OS81050.

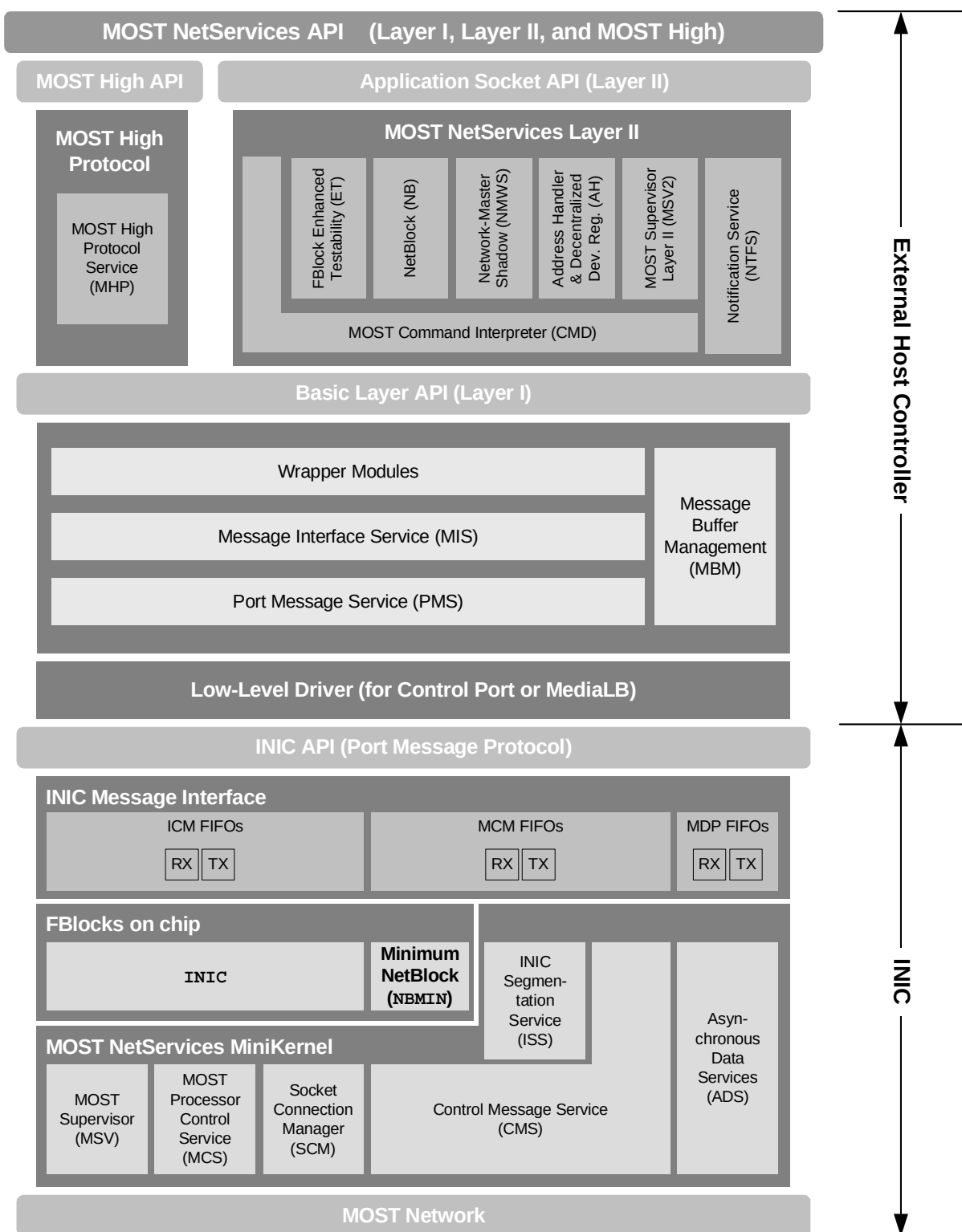


Figure 1-3: Division of Services between INIC and the EHC

2 Pinout

Input pins must not be left floating; therefore, they must be driven, have pull-ups or pull-downs, or be tied directly to one of the power or ground pins.

Digital pins that can be configured as outputs (e.g. $D_{I/O}$, D_{OUT} , $D_{I/OD}$) are high impedance during power-up/reset. Immediately following power-up/reset:

- pins with more than one configuration option are as follows:
 - Pin 29 - **DSCL** functionality ($D_{I/OD}$) - assumes **TMS** high
 - Pin 30 - **DSDA** functionality ($D_{I/OD}$) - assumes **TMS** high
 - Pin 32 - **MLBSIG** functionality ($D_{I/O}$) - default can be changed via the *INIC Configuration String*
 - Pin 33 - **MLBDAT** functionality ($D_{I/O}$) - default can be changed via the *INIC Configuration String*
 - Pin 34 - High impedance until configured
 - Pin 35 - High impedance until configured
 - Pin 41 - **ERR** functionality (D_{OUT}) - unless held low at the rising edge of reset
 - Pin 42 - **DINT** functionality (D_{OUTD})
 - Pin 43 - **INT** functionality (D_{IN}) - assumes **ERR/BOOT** high
- output pins with a disable option remain in the high impedance state, unless specifically enabled by software (e.g. **RMCK**, **SYNC**, **MLBCLK**), and
- all other pin types are as shown in Table 2-1.

2.1 Pinout List

Pin	Name	Type	Pin Description
1	SDA ¹	$D_{I/OD}$	Control Port I ² C data
2	$\overline{\text{RST}}$	D_{IN}	Hardware reset input. (Pull-up resistor to VDDP should be used when not driven high by an external device. A series resistor should be used in lieu of the pull-up when always driven by an external device.)
3	RMCK	D_{OUT}	Recovered master clock output for synchronizing external devices
4	VDDC1		2.5 V core power supply (digital)
5	GNDC1		Ground (digital)
6	VDDP1		3.3 V periphery power supply (digital)
7	TX	D_{OUT}	Network transmitter output
8	TXGAIN ^{1,4}	D_{OUTD}	FOT optical power attenuation control signal
9	GNDP1		Ground (digital)
10	VDDA1		2.5 V analog power supply
11	GNDA1		Ground (analog)
12	NC		Do not connect
13	RX	D_{IN}	Network receiver input
14	VDDA2		2.5 V analog power supply
15	GNDA2		Ground (analog)
16	PS0	D_{IN}	External Power Management status bit 0. (Pull-down resistor to ground required when external power management is not used - 47 k Ω recommended.)
17	FLT	A_{IO}	PLL loop filter output

1. Pull-up resistor required.
2. Pull-down resistor required.
3. A pull-up or pull-down resistor required, based on default mode.
4. When used, this pin should not be wire-or'd with other open-drain outputs in the application.

Table 2-1: Pinout List

Pin	Name	Type	Pin Description
18	PS1	D _{IN}	External Power Management status bit 1. (Pull-down resistor to ground required when external power management is not used - 47 kΩ recommended.)
19	PWROFF	D _{OUT}	External Power Management power-off output. This pin is driven high when the INIC Processor is ready to be shut down. (This pin is Hi-Z after power-up, until initialized by INIC. A pull-up or pull-down resistor is recommended to set the initial state.)
20	VDDP2		3.3 V peripheral power supply (digital)
21	GNPD2		Ground (digital)
22	BIAS	A _{IO}	Regulates 2.5 V supply from 3.3 V switched supply, using an external pass transistor. (See Figure 13-2) If not used, this pin must be connected directly to ground.
23	TMS ¹	D _{IN}	JTAG Test Mode Select
24	XTO	A _{IO}	Crystal oscillator output
25	XTI	A _{IO}	Crystal oscillator input - or external CMOS clock input
26	GNDC2		Ground (digital)
27	VDDC2		2.5 V core power supply (digital)
28	SYNC	D _{OUT}	Clock output for synchronization of switching power supplies. (Disabled by Factory default).
29	TCK ¹	D _{IN}	JTAG Test Clock
	DSCL ¹	D _{I/OD}	Customer Configuration Interface clock
30	TDI ¹	D _{IN}	JTAG Test Data Input
	DSDA ¹	D _{I/OD}	Customer Configuration Interface data
31	MLBCLK ²	D _{OUT}	MediaLB Port clock output
32	MLBSIG ²	D _{I/O}	3-pin MediaLB Port signal information line
	MLBSO ²	D _{OUT}	5-pin MediaLB Port signal information output line
	SR1	D _{IN}	Streaming Port serial data input 1
33	MLBDAT ²	D _{I/O}	3-pin MediaLB Port data line
	MLBDO ²	D _{OUT}	5-pin MediaLB Port data output line
	SX1	D _{OUT}	Streaming Port serial data output 1
34	SR0	D _{IN}	Streaming Port serial data input 0
	MLBSI	D _{IN}	5-pin MediaLB Port signal information input line
	SX1	D _{OUT}	Streaming Port serial data output 1
35	SX0	D _{OUT}	Streaming Port serial data output 0
	MLBDI	D _{IN}	5-pin MediaLB Port data input line
	SR1	D _{IN}	Streaming Port serial data input 1
36	FSY	D _{I/O}	Streaming Port frame sync
37	SCK	D _{I/O}	Streaming Port bit clock
38	NC		No connect
39	NC		No connect
40	RSOUT ^{1,4}	D _{OUTD}	Optional reset for External Host Controller. (Pull-up resistor is required when used. If not used, this pin may be left unconnected.)

1. Pull-up resistor required.
2. Pull-down resistor required.
3. A pull-up or pull-down resistor required, based on default mode.
4. When used, this pin should not be wire-or'd with other open-drain outputs in the application.

Table 2-1: Pinout List (Continued)

Pin	Name	Type	Pin Description
41	ERR	D _{OUT}	Error indicator output. (See <i>OS8105x/6x INIC API User's Manual</i> [2] for information on configuring the behavior of this pin.)
	$\overline{\text{BOOT}}$ ¹	D _{IN}	Configuration pin. This pin is connected to the configuration/debug header and is used by SMSC's <i>INIC Explorer</i> [4] to load initial configuration data into INIC. This pin may also be connected to the EHC to allow INIC flashing or configuration via the Control Port.
42	TDO ¹	D _{OUTZ}	JTAG Test Data Output
	$\overline{\text{DINT}}$ ¹	D _{OUTD}	Customer Configuration Interface interrupt (active low)
43	$\overline{\text{INT}}$ ³	D _{OUTD}	Control Port interrupt (active low). See Note 4.
		D _{IN}	Configuration pin - When <code>PortConfiguration.DefPort()</code> is set to <code>INT_Select</code> : a pull-up resistor enables the Control Port at power-up or reset, and a pull-down resistor enables the MediaLB Port by default at power-up or reset.
		D _{OUT}	During <i>Boot Mode</i> , this pin is driven to VDDCn.
44	SCL ¹	D _{I/OD}	Control Port I ² C clock

1. Pull-up resistor required.
2. Pull-down resistor required.
3. A pull-up or pull-down resistor required, based on default mode.
4. When used, this pin should not be wire-or'd with other open-drain outputs in the application.

Table 2-1: Pinout List (Continued)

2.2 Pinout

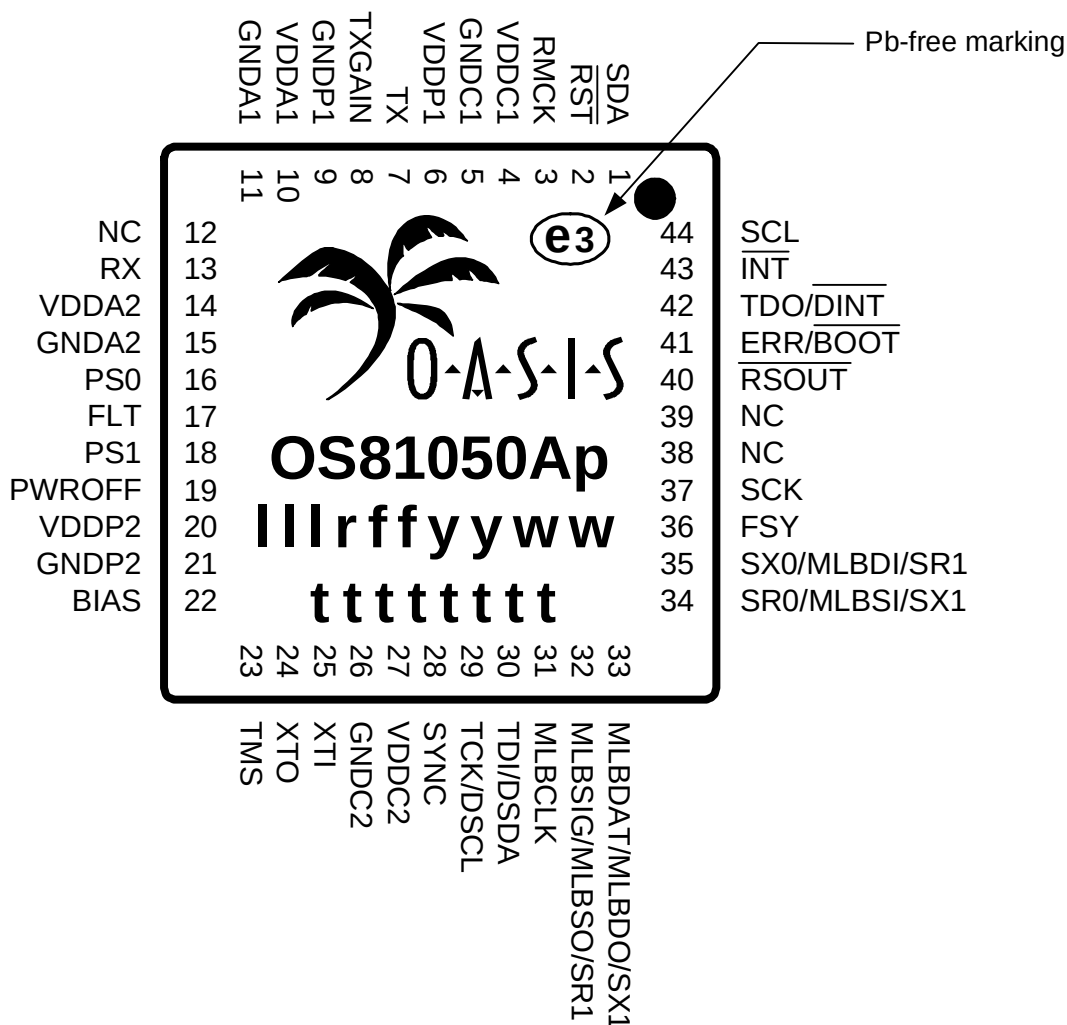


Figure 2-1: OS81050 Pinout

The package designators are:

- p - Package (Q: QFP, H: ETQFP)
- III - Lot Sequence Code
- r - Chip Revision Letter
- ff - ROM Revision Code (Optional)
- yy - last two digits of Assembly Year
- ww - Assembly Work Week
- tttttt - Tracking Number (up to 8 characters)

2.3 Equivalent Schematics for Pins

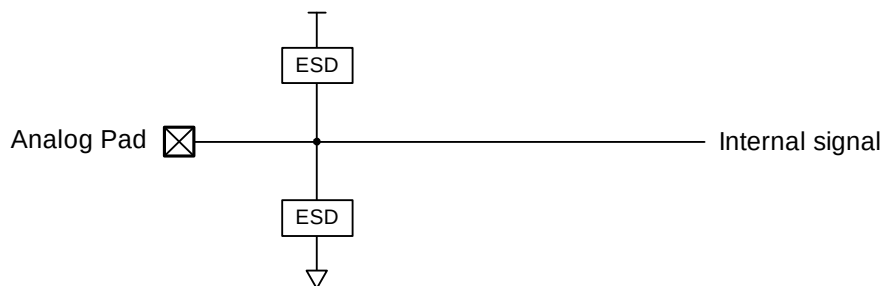


Figure 2-2: Pin-equivalent for Analog I/O pin - A_{IO}

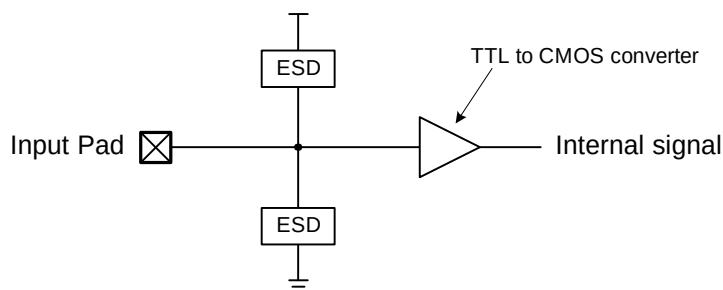


Figure 2-3: Pin-equivalent for Digital Input pin - D_{IN}

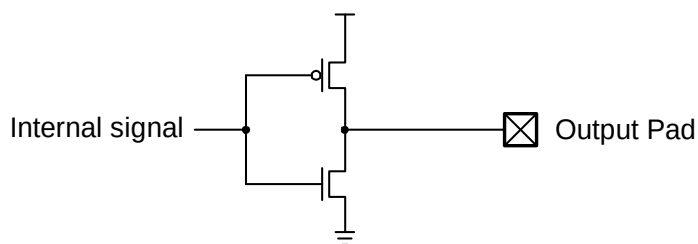


Figure 2-4: Pin-equivalent for Digital Output pin - D_{OUT}

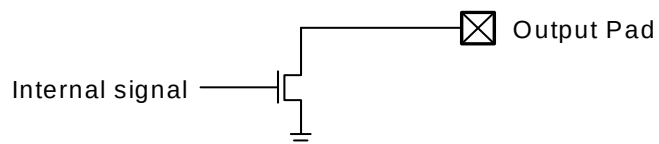


Figure 2-5: Pin-equivalent for Open-Drain Digital Output pin - D_{OUTD}

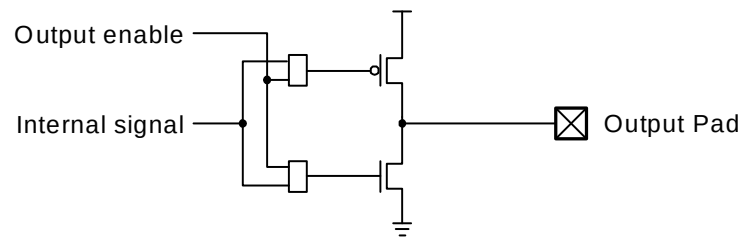


Figure 2-6: Pin-equivalent for Digital Output pin with high-Z control - D_{OUTZ}

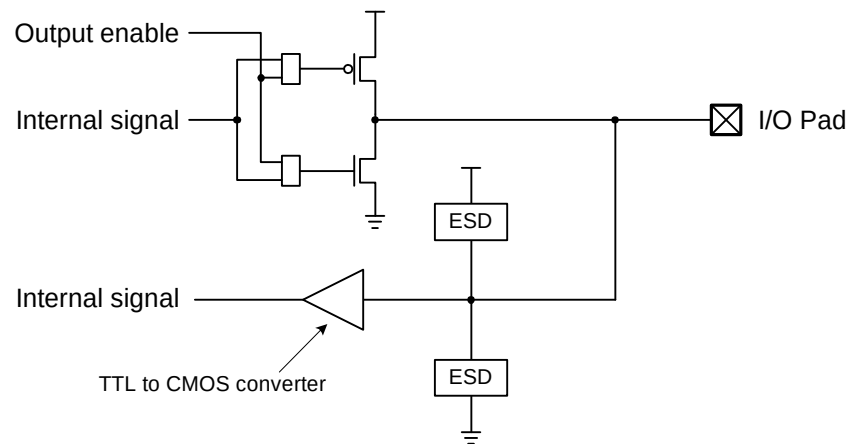


Figure 2-7: Pin-equivalent for Digital I/O pin - $D_{I/O}$

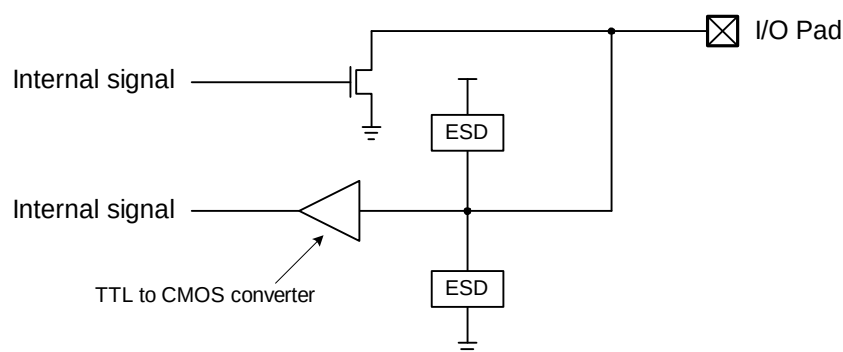


Figure 2-8: Pin-equivalent for Digital Input/Open-Drain Output pin - $D_{I/OD}$

3 INIC Processor

The OS81050 is a valid network node even without an EHC, due to the on-chip INIC Processor. The *INIC Software Stack*, running on the INIC Processor, provides a message-based interface that is easily implemented in a high-level language. This message-based interface not only simplifies the interface between the application and the network, but allows the EHC software to be completely driven by message events. Using the INIC API, as described in the *OS8105x/6x INIC API User's Manual* [2], the EHC has access to the same functionality that would be available on a register-based interface, but the programmer is spared the effort of making software align with the chip architecture. Much of the *INIC Software Stack* is common across INIC chips, minimizing the effort of upgrading the software for different application platforms. The power-up default settings of the INIC Processor, such as timing-master/timing-slave functionality and other configuration options, can be changed through the *Customer Configuration Interface* (see Section 13.5).

Partitioning the node architecture between INIC and the EHC results in an efficient implementation, allowing INIC to respond to many network events without intervention by the EHC. Network initialization is optimized, not requiring the network transceiver at each node to wait for the Application Layer to respond. Application software runs efficiently since the number of interrupts to the EHC is minimized. Interaction between the EHC and INIC can be further reduced if the EHC implements notification for INIC properties. Figure 3-1 illustrates the OS81050 *INIC Software Stack*.

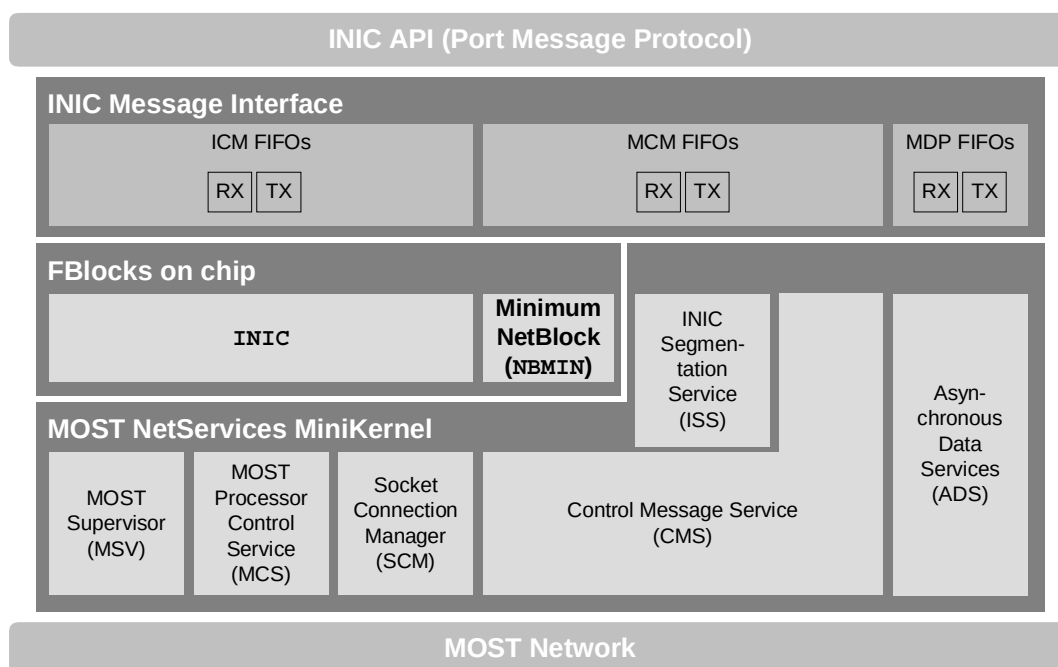


Figure 3-1: INIC Software Stack

3.1 INIC Message Interface

Internal OS81050 FIFOs are used for *message-based* data exchange between INIC and the EHC. Message-based data includes: control messages (e.g. MCMs, ICMs) and asynchronous packets (e.g. MDPs). The *Port Message Protocol* (described in Chapter 4) defines the format for all messages passing through the INIC Message Interface.

Six independent FIFOs support the INIC Message Interface, including:

- *MCM Rx FIFO* - used to receive *MOST Control Messages* (MCMs) from the EHC that are intended for either the local FBlock `NBMIN` or the MOST Network.
- *MCM Tx FIFO* - used to transmit MCMs to the EHC from the local FBlock `NBMIN` or the MOST Network.
- *ICM Rx FIFO* - used to receive *INIC Control Messages* (ICMs) from the EHC intended only for the local FBlock `INIC`.
- *ICM Tx FIFO* - used to transmit ICMs to the EHC from the local FBlock `INIC`.
- *MDP Rx FIFO* - used to receive *MOST Data Packet* messages (MDPs) from the EHC that are intended for transmission onto the MOST Network.
- *MDP Tx FIFO* - used to transmit MDPs to the EHC from the MOST Network.

Each type of message-based data targets a specific INIC FIFO based on direction of transport. These FIFOs are used for message-based data exchange, regardless of the actual hardware port used for communication.

The OS81050 INIC supports four external interfaces (INIC Ports) for exchanging messages. Message FIFOs are used during transmission and reception of data at each port. Most ports can be opened and configured using the `OpenPort()` function within the MOST NetServices MiniKernel's *Socket Connection Manager* (SCM).

- **MediaLB Port (PortID 0)** - The MediaLB Port can be opened by default at power-up/reset based on the *INIC Configuration String*. If not automatically opened by default, this port can be opened and configured through the Control Port, using the `OpenPort()` function described in the API manual.

When the MediaLB Port is automatically opened out of reset, one socket pair is created for Control message transport between INIC and the EHC. A sink socket is assigned *ChannelAddress* 0004h for EHC to INIC exchange. A source socket is assigned *ChannelAddress* 0002h for INIC to EHC exchange. Additional socket pairs for asynchronous and synchronous data transfer between the MediaLB Port and the Network Port can be opened by the user with the `CreateSocket()` INIC function.

- **Control Port (PortID 1)** - The Control Port is not configurable via the API. This port can only be opened at power-up/reset based on the *INIC Configuration String*.
- **Network Port (PortID 2)** - The Network Port is configurable via the API. This port is always opened default at power-up/reset.
- **Streaming Port (PortID 3)** - The Streaming Port is never opened by default at power-up/reset. This port can be opened and configured through the Control Port or MediaLB Port, using the `OpenPort()` function described in the API manual.

4 Port Message Protocol

All message-based data (includes control messages and asynchronous packets) transferred up or down the *INIC Software Stack* adheres to the same low-level message format (defined as a *Port Message*) regardless of the physical hardware port used.

Control messages (e.g. MCMs, ICMs) are used to access the FBlock of a network node. MCMs are typically routed *through* INIC (between the network control channel and the INIC Message Interface), using either the *MCM Rx FIFO* or the *MCM Tx FIFO*, depending on the direction of transport. ICMs target the FBlock *INIC* and are exchanged exclusively between INIC and the EHC, using either the *ICM Rx FIFO* or the *ICM Tx FIFO*, depending on the direction of transport.

Asynchronous packets (e.g. MDPs) are routed between the network packet channel and the INIC Message Interface, using either the *MDP Rx FIFO* or the *MDP Tx FIFO*, depending on the direction of transport.

4.1 Generic Port Messages

A generic *Port Message* consists of three main fields:

- *Port Message Length* (PML) - This 16-bit field indicates the total number of bytes that follow in the Port Message, including the message overhead and payload.
- *Port Message Header* (PMH) - This field contains the Port Message overhead and defines the format of the payload data that follows.
- *Port Message Body* (PMB) - This field contains the message payload whose format is dependant on the preceding header.

Figure 4-1 illustrates the generic format of the *Port Message*.

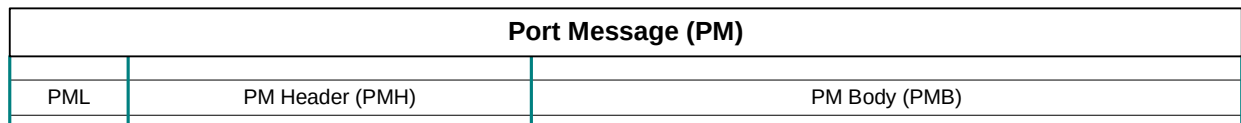


Figure 4-1: Generic Port Message Format

Five types of specific *Port Messages* are defined, each associated with a specific operation to be performed on the targeted FIFO:

- *FIFO Command* - Writes commands to the INIC Message Interface
- *FIFO Status Read* - Reads FIFO status information from the INIC Message Interface
- *FIFO Status Write* - Sends an acknowledge to the INIC Message Interface when a data message is read
- *FIFO Data Write* - Writes a message into a particular FIFO
- *FIFO Data Read* - Reads a message out of a particular FIFO

The following pages outline the basic structure of the *Port Messages*. For more information on the software-controlled, sub-fields of the *Port Messages*, see the *OS8105x/6x INIC API User's Manual* [2].

4.2 FIFO Status and Command Messages

FIFO Status and Command messages are comprised of the following fields:

- *Port Message Length* (PML) - This 16-bit field specifies the total byte length of the message, not including this field.
- *Port Message Header Length* (PMHL) - This single byte field specifies the total byte length of the two fields that follow (*FIFO Protocol Header* and *FIFO Message Field*).
- *FIFO Protocol Header* (FPH) - Typically one byte in length, this field defines the message type and the FIFO to be targeted by the message. Support for an additional header byte is available.
- *FIFO Data Header* (FDH) - Optional field (based on the FPH setting) that supports a one-byte Handle.
- *FIFO Message Field* - This field contains the message payload for FIFO Command and Status messages. In FIFO Command messages, this field specifies INIC behavior for failed messages, along with a FIFO synchronization command. In FIFO Status Read messages, this field contains status information about the receive and transmit FIFOs, as well as results of the last message transmitted.

Figure 4-2 illustrates the main fields of the FIFO Status and FIFO Command messages.

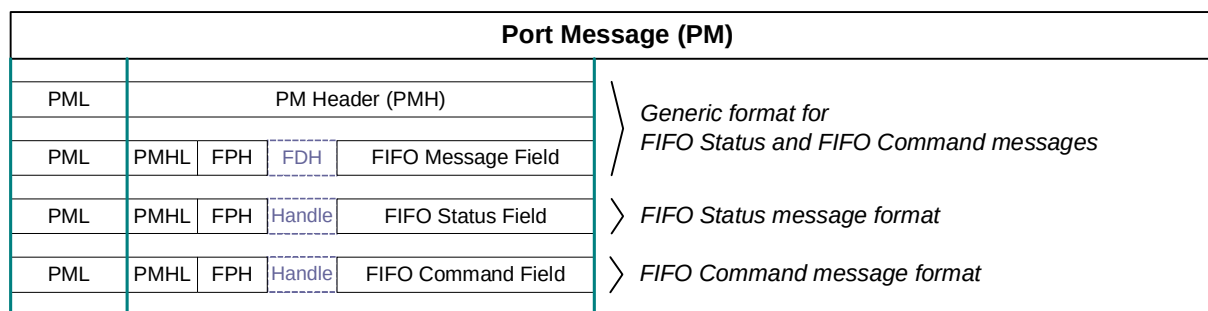


Figure 4-2: FIFO Status and Command Message Format

4.3 FIFO Data Messages

FIFO Data messages, which indicate either a read or write of a control message or asynchronous packet, are comprised of the following fields:

- *Port Message Length* (PML) - This 16-bit field specifies the total byte length of the message, not including this field.
- *Port Message Header Length* (PMHL) - This single byte field specifies the total byte length of the two fields that follow (*FIFO Protocol Header* and *FIFO Data Header*).
- *FIFO Protocol Header* (FPH) - At least one byte in length, this field defines the message type and the FIFO to be targeted by the message. The length is extensible up to three bytes on a byte-basis.
- *FIFO Data Header* (FDH) - This field contains message information, such as Target Address, Source Address, Functional Address (FBlockID.InstID part of the control message), message priority, and retry parameters. This is a variable-length field, based upon FPH setting.
- *Port Message Body* (PMB) - This field (also known as the *FIFO Data Body*), contains the actual message, and has a total length defined by (PML - PMHL - 1). In addition to payload, this field defines the FktID.OpType(Parameters) part of the control message, required by the *MOST Specification* for typical control messages.

For MOST Data Packets (MDPs), this field contains a 16-bit length sub-field and a maximum MOST Packet message payload of 1014 bytes when the message is sent via the MediaLB interface. When sent via the Control Port, the maximum MOST Packet payload is 50 bytes.

Figure 4-3 illustrates the main fields of the FIFO Data (Read/Write) messages.

Port Message (PM)				
PML	PM Header (PMH)			PM Body (PMB)
PML	PMHL	FPH	FIFO Data Header (FDH)	FIFO Data Body

Figure 4-3: FIFO Read and Write Message Format

4.4 FIFO Message Exchange

All FIFO Data messages require a status message response from the receiving device. For example, an ICM FIFO Data message sent to INIC results in the following status message being sent to the EHC:

```
ICM:Status(XmitSuccess)
```

This status message indicates the OS81050 received the message successfully. Likewise, when the OS81050 sends a FIFO Data message to the EHC (from the network or as a response to a previous request), the EHC is required to respond with a status message indicating message reception was successful.

INIC provides an application watchdog to monitor the *Port Message* transfer during a predefined time frame. The watchdog timer times out under either of the following two cases:

- a status message response to a data message is not sent by the EHC within the timeout period, or
- INIC does not receive an MCM or ICM data message from the EHC within the timeout period.

The watchdog timer is configured through the `WatchdogMode()` INIC function. For more information on configuring the watchdog timer, refer to the *OS8105x/6x INIC API User's Manual* [2].

4.4.1 MOST Control Messages (MCM)

MOST Control Messages (MCMs) are transferred between INIC and the EHC on either the MediaLB Port or the Control Port. MCMs can target the EHC FBlocks, INIC's FBlock NBMIN, or FBlocks that exist on the MOST Network. Segmented MCM messages are supported.

The default structure of an MCM sent from INIC to the EHC:

```
PML.06.04.TgtDevType.SrcDevID.FBlockID.InstID.FktID.OpType.TelID.Length.Data
```

The default structure of an MCM sent from the EHC to INIC:

```
PML.05.04.TgtDevID.FBlockID.InstID.FktID.OpType.TelID.Length.Data
```

4.4.2 INIC Control Messages (ICM)

INIC Control Messages (ICMs) are very similar to MCM messages, with the exception that ICM messages are always to/from the local FBlock INIC directly. Unlike MCM messages, ICM messages do not support segmentation.

ICM and MCM messages use separate FIFOs; therefore, ICM messages are never blocked by MCM messages that may need to be retried, or fail to get sent altogether. The *ICM Rx FIFO* and *ICM Tx FIFO* directly interface with the local FBlock INIC (which is always available) and provides faster response time for system-important functions, such as the watchdog timer.

The default structure of an ICM sent from INIC to the EHC:

```
PML.01.14.FktID.OpType.0.Length.Data
```

The default structure of an ICM sent from the EHC to INIC:

```
PML.01.14.FktID.OpType.0.Length.Data
```

4.4.3 MOST Data Packets (MDP)

For *MOST Data Packets* (MDPs), INIC acts as a liaison between the EHC and the MOST Network (no INIC internal FBlocks handle MDPs). All MDP messages from the EHC must target devices that reside on the MOST Network. Similarly, INIC passes all MDPs from the MOST Network to the EHC.

The default structure of an MDP sent from INIC to the EHC:

```
PML.05.0C.TgtDevType.SrcDevID.(FillerBytes).Length.Data
```

The default structure of an MDP sent from the EHC to INIC:

```
PML.05.0C.Prio.TgtDevID.(FillerBytes).Length.Data
```

FIFO Status messages always indicate a successful transmission and are only returned for the MDP FIFO when using the Control Port to exchange asynchronous packets.

MOST Data Packets are sent across the MOST Network with the length value in quadlets. If the MOST Network message is not divisible by four, INIC extends the length and converts the length from bytes to quadlets before sending the message to the MOST Network.

MDP messages from the EHC to INIC require the entire message header (includes: PML, PMHL, FPH, and FDH) be a four-byte multiple. If the message header byte length is not divisible by four, filler bytes (containing zeros) must be added after the FDH, and PML must be updated accordingly.

5 Control Port

The Control Port supports message-based communication exclusively between the OS81050 and the EHC. These messages are used to configure and control INIC and target the local INIC FBlocks (*INIC* or *NBMIN*). The Control Port also supports the routing of asynchronous packets and control messages between the EHC and the network.

The Control Port supports the I²C protocol, where the data is received MSB-first, and is processed in bytes by the internal firmware of the OS81050. The processing time varies based on the overall load of tasks. Clock stretching provides an appropriate handshake mechanism to adapt the data transfer rate dynamically, thereby maximizing the transfer rate at any given time.

The Control Port is enabled by default after power-up/reset based on the *INIC Configuration String*. Specifically, the Control Port is enabled under one of the following two conditions:

- `PortConfiguration.DefPort()` is set to `I2C`, or
- `PortConfiguration.DefPort()` is set to `INT_Select` and $\overline{\text{INT}}$ is pulled high.

If one of these conditions is not met when the OS81050 is powered-up (or reset), the Control Port is disabled and all communication between the EHC and the OS81050 must occur through the MediaLB Port.

The OS81050 Control Port operates as an I²C slave device that must be managed externally. The *SCL* pin clocks data in and out; *SDA* is a bi-directional data pin. $\overline{\text{INT}}$ is an open-drain, active low output that can be used as an interrupt to the EHC. The INIC $\overline{\text{INT}}$ line should be isolated from other signals that may interrupt the EHC (e.g. do not wire-or with other open-drain lines).

Figure 5-1 illustrates the OS81050 in an I²C environment.

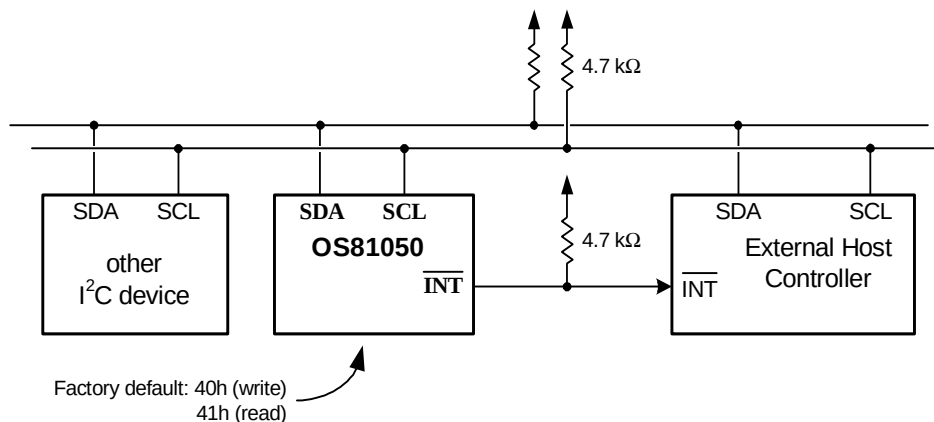


Figure 5-1: Control Port Pin Connections

When the Control Port is enabled, INIC drives the $\overline{\text{INT}}$ interrupt pin low to inform the EHC when service through the port is required (e.g. message available for reading). Following reset, the EHC should wait for the first falling edge of $\overline{\text{INT}}$, which occurs when initialization is complete. The first available message through the Control Port (shown below) indicates INIC is ready to receive messages and the EHC should synchronize all FIFOs.

```
AllFIFOs:Status(SyncS, StartUp)
```

The first byte of an I²C message is the bus address plus the read/write (R/W) bit, which determines whether the EHC is reading or writing *Port Messages*. The OS81050 Factory default I²C address is 41h for read, 40h for write; however, using the *Customer Configuration Interface* (described in Section 13.7), the default I²C address can be changed in the *INIC Configuration String*.

Clock stretching is a handshaking mechanism supported on the Control Port. If the OS81050 cannot keep up with a message, it will stretch the clock at the byte boundary; therefore, the I²C bus master must monitor *SCL* when communicating with INIC.

If the OS81050 is busy supporting higher priority tasks, it will NACK the I²C address, indicating that the master should retry the message at a later time. I²C NACK responses occur at the address byte boundary (e.g. following the R/W bit).

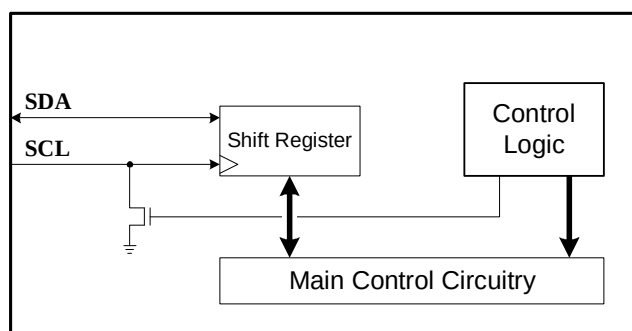


Figure 5-2: Control Port Block Diagram

For information on the I²C protocol, refer to the *I2C-Bus Specification* [5].

5.1 Writing to the Control Port

The beginning of transmission is marked by a start condition. The first byte specifies the chip address, as well as whether the operation is a read or a write. The OS81050 address (Factory default of 0100000) occupies the upper seven bits of the first byte; the R/W bit is the LSB.

When the Control Port receives an address byte of 40h (Factory default of OS81050 address and $\overline{R/W}$ bit clear), it acknowledges reception of the byte through an acknowledge bit and the rest of the data is written into the Control Port. The write data is defined as the *Port Message* (PM) and can be one of the following:

- FIFO Command message
- FIFO Data (Write) message
- FIFO Status (Write) message

Figure 5-3 illustrates the I²C transmission write sequence. The characters “S” and “P” represent the start and stop conditions for messages.

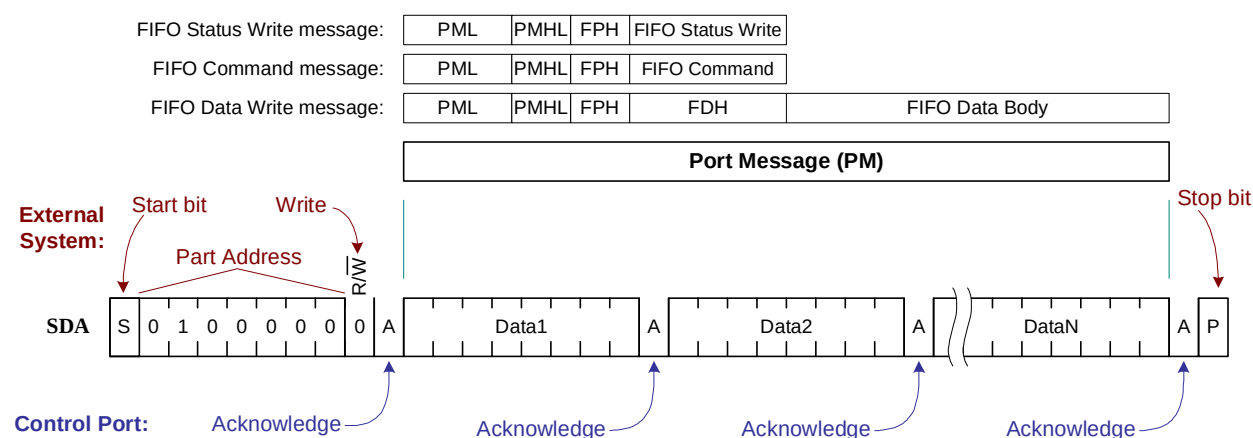


Figure 5-3: Control Port Write Sequence

5.2 Reading from the Control Port

The beginning of transmission is marked by a start condition. The first byte specifies the chip address, as well as whether the operation is a read or a write. The OS81050 address (Factory default of 0100000) occupies the upper seven bits of the first byte; the R/W bit is the LSB.

When the Control Port receives the address byte of 41h (Factory default OS81050 address and R/W bit set), it acknowledges reception of the byte through an acknowledge bit and the rest of the data is read from the Control Port. The read data is defined as the *Port Message* (PM) and can be one of the following:

- FIFO Data (Read) message
- FIFO Status (Read) message

Figure 5-4 illustrates the I²C transmission read sequence. The characters “S” and “P” represent the start and stop conditions for messages.

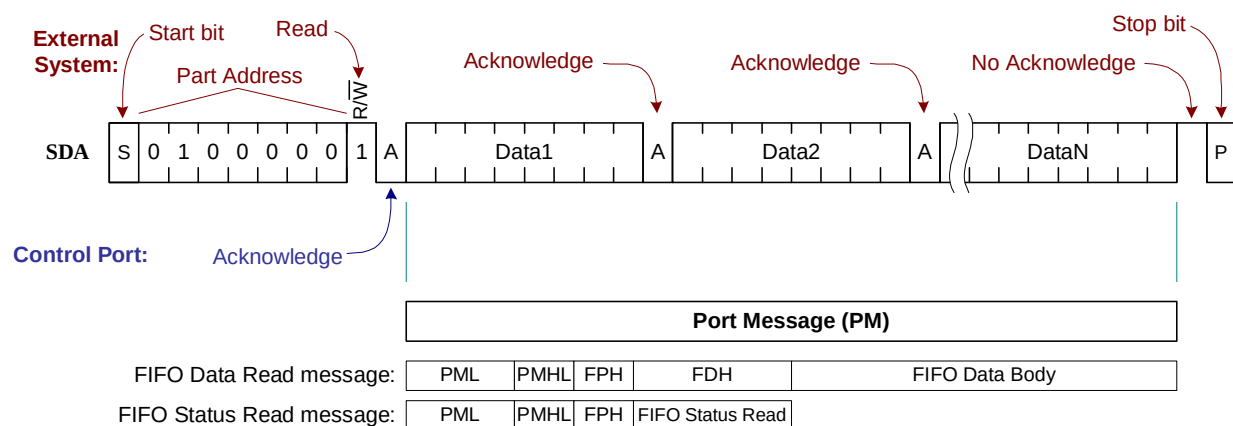


Figure 5-4: Control Port Read Sequence

The $\overline{\text{INT}}$ pin going low indicates that a *Port Message* is available in at least one of the message FIFOs. Once a message starts being read, the $\overline{\text{INT}}$ pin is de-asserted. If other messages are pending (waiting to be read), the $\overline{\text{INT}}$ pin will be asserted again. This pending interrupt can occur as soon as the last byte of the current *Port Message* is read.

If a *Port Message* is not fully read (stop or repeated start bit received before the end of the message), $\overline{\text{INT}}$ will be reasserted and the next read command will start reading from the beginning of the disrupted message, not the position where the read was previously terminated.

Ideally, the EHC reads the entire length of the *Port Message* in one Control Port read cycle, where the total number of bytes read is extracted from the Port Message Length (PML) during the read cycle. For I²C drivers that are not able to decipher the message length during a single read cycle, other methods of reading the *Port Message* are feasible, including the following:

- Example 1: The EHC may issue two consecutive Control Port reads. The first cycle reads two bytes of the *Port Message*, where PML is extracted. Based on the message length obtained from PML in this first read cycle, a second cycle reads the entire *Port Message* (including the first two bytes again).
- Example 2: The EHC may issue read cycles that always read the *maximum* length of a *Port Message*, regardless of the actual PML. Once the *Port Message* is read, application software can later determine the relevant portion of the read data using the actual PML. The OS81050 returns 00h data when a read cycle extends beyond the actual length of the *Port Message*.

6 MediaLB Port

The *Media Local Bus* (MediaLB) Port gives external devices (e.g. the External Host Controller) real-time access to all supported types of data on the MOST Network. The MediaLB Port and the Streaming Port share some pins; therefore, some Streaming and MediaLB modes are mutually exclusive.

The MediaLB Port can be configured for one of the following operational modes:

- MediaLB 3-pin Mode - When configured in this mode, the MediaLB Port provides a 3-pin interface that supports a maximum data rate of $1024 \times F_s$ (49.152 MHz at $F_s=48$ kHz). In this mode, the Streaming Port may be configured with up to two serial interfaces for accessing synchronous network data.
- MediaLB 5-pin Mode - When configured in this mode, the MediaLB Port provides a 5-pin interface that supports a maximum data rate of $512 \times F_s$ (24.576 MHz at $F_s=48$ kHz). In this mode, the Streaming Port is not available.

MediaLB bandwidth is managed in quadlets (4 bytes) and consists of a combination of streaming, packet, and control data. The MediaLB Port rate of $256 \times F_s$ supports up to seven quadlets of data, divided among streaming, packet, and control data types. The MediaLB Port rate of $512 \times F_s$ supports up to 15 quadlets of data. The MediaLB Port rate of $1024 \times F_s$ supports up to 31 quadlets of data. A maximum of 64 bytes can be transferred per frame, in either direction, for the Streaming Port and MediaLB Port combined.

When transmitting data onto the network, the MSB is transmitted first. To comply with the *MOST Specification* [1], when transmitting multi-byte words, the most significant byte should be transmitted first. In addition, when transmitting stereo data, left should be sent before right.

The MediaLB Port can be enabled by default after power-up/reset based on the *INIC Configuration String*. If the MediaLB Port is not enabled by default at power-up/reset, it can be manually enabled and configured via ICMs through the Control Port, if desired. The MediaLB Port is enabled automatically at power-up/reset under one of the following two conditions:

- `PortConfiguration.DefPort()` is set to MediaLB, or
- `PortConfiguration.DefPort()` is set to `INT_Select` and $\overline{\text{INT}}$ is pulled low.

When the MediaLB Port is enabled at power-up/reset, the Factory default mode and speed is 3-pin mode at $512 \times F_s$. This default mode can be changed through the *Customer Configuration Interface*, as described in Section 13.5.

Although $\overline{\text{INT}}$ is not used for MediaLB communications, if pulled high, it can be used to wake up the EHC after a device shut-down. See the `ShutDown()` function in the *OS8105x/6x INIC API User's Manual* [2] for more information.

After reset, and following MediaLB Port initialization, the first INIC message sent to the EHC indicates the OS81050 is initialized and ready to communicate with the EHC.

```
AllFIFOs:Status(SyncS, StartUp)
```

In common applications, this message is used to instruct the EHC to synchronize its FIFOs. Additionally, the EHC may start synchronization of the OS81050 by sending the following message:

```
AllFIFOs:Command(SyncC)
```

The OS81050 responds to the above message once the synchronization is complete. This procedure ensures that OS81050 and the EHC can synchronize communication under various circumstances. The status message sent by the OS81050 is:

```
AllFIFOs:Status(SyncS, SyncRcvd)
```

6.1 MediaLB 3-pin Mode

The MediaLB Port provides a conduit for accessing the content of the MOST Network via a single hardware interface. All data types exchanged across network channels are supported across MediaLB.

The MediaLB Port also allows access to the internal functions of the OS81050, eliminating the need for Control Port access. With MediaLB, only one interface is needed for data and control. When combined with the MediaLB software API library, the system designer is insulated from much of the complexity of managing the interface between the application software and the network.

The MediaLB 3-pin interface consists of:

- **MLBCLK** - MediaLB Controller output signal that clocks data in and out of the MediaLB Port
- **MLBDAT** - bi-directional signal that transports data to and from the MediaLB Port
- **MLBSIG** - bi-directional signal that transports signal information to and from the MediaLB Port

When the MediaLB Port is configured in MediaLB 3-pin mode, the Streaming Port can be simultaneously configured with two serial interfaces.

The MediaLB Controller (OS81050) uses **MLBSIG** to grant bus access to a MediaLB Device (e.g. EHC). That Device can then send data to other Devices (including the Controller). The OS81050 **MLBDAT** pin receives data for configuration or transmission onto the network, and transmits data to other Devices from the network receiver. Data and signal information is shifted into Devices by **MLBCLK**. A complete description of the Media Local Bus can be found in the *MediaLB Specification* [3].

In the MediaLB 3-pin connection diagram shown in Figure 6-1, each MediaLB line has a weak pull-down to keep the signals in a known state when no Device is driving the line. MediaLB Devices that transmit on the line also have a series resistor near the Device for series termination and rise/fall time control. The clock line may optionally have AC-parallel termination at the farthest point from the Controller (clock source) to minimize reflections and ensure a clean clock.

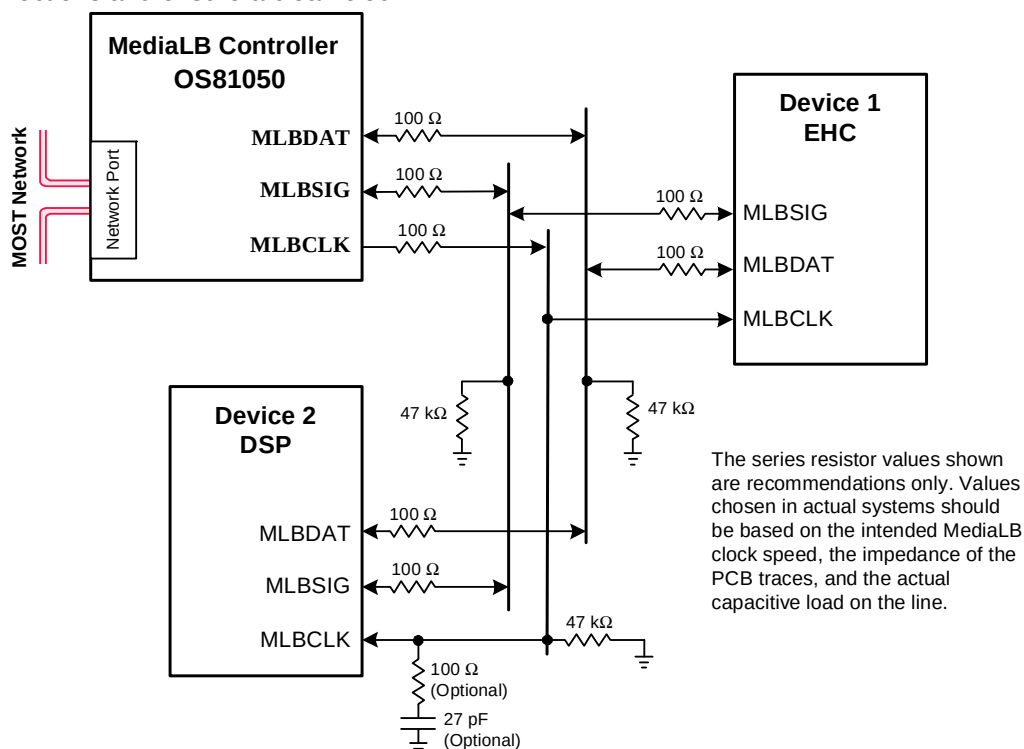


Figure 6-1: MediaLB 3-pin Mode Connection Diagram - Example

The MediaLB protocol supports a set of physical channels for sending data over the Media Local Bus. The physical channels (PCn) are four bytes in length (one quadlet) and can be grouped into logical channels. Each logical channel represents a unidirectional data path (one or more quadlets in length) and is defined by a unique *ChannelAddress*.

On each logical channel, the source (transmitter) is a unique location on a Device, with a destination (receiver) of one or more Devices. In a system, the *ChannelAddresses* are generally static and defined at the development stage; however, the MediaLB Controller can dynamically define additional *ChannelAddresses* during operation to communicate with other Devices.

The MediaLB Controller initiates communication over the Media Local Bus by sending out a *ChannelAddress* on **MLBSIG**. This indicates to the addressed Device that it can begin transmitting on the logical channel associated with that *ChannelAddress*. The structure of the *ChannelAddress* is described in the *MediaLB Specification* [3], *Link Layer Section*.

The number of *ChannelAddresses* supported is based on the MediaLB Port speed and the size of the logical channels associated with the *ChannelAddresses*. The total number of physical channels available at a given MediaLB Port speed is indicated in Table 6-1. The mapping of specific physical channels into a logical channel is arbitrary and managed by the MediaLB Controller. Generally, logical channels are created by the EHC using the `CreateSocket()` INIC function.

MediaLB Speed	Quadlets per MediaLB Frame	Physical Channels	Available Physical Channels*	Bytes Available per Frame*
256×Fs	8	PC0 - PC7	7	28
512×Fs	16	PC0 - PC15	15	60
1024×Fs	32	PC0 - PC31	31	124

* PC0 (first physical channel of the MediaLB frame) is always used as the *SystemChannel*; therefore, the bytes of this physical channel are not available for application data.

Table 6-1: Available MediaLB Bandwidth

Once per network frame, the MediaLB Controller generates a unique 16-bit *FRAMESYNC* pattern on the signal information line (**MLBSIG**). The end of the *FRAMESYNC* pattern defines byte and physical channel boundaries, and indicates that the MediaLB frame boundary occurs one quadlet later. Physical Channel 0 (PC0), defined as the first physical channel of the MediaLB frame, is always used as the *SystemChannel*. The *FRAMESYNC* pattern represents a special *ChannelAddress*, which grants the MediaLB Controller (OS81050) PC0 for system administration.

As illustrated in Figure 6-2, one quadlet after the *ChannelAddress* is sent by the Controller, the Device associated with that *ChannelAddress* sends out a MediaLB *Command* on **MLBSIG**, while simultaneously sending the corresponding data over **MLBDAT**. All MediaLB Devices, including the OS81050 Controller, compare the *ChannelAddress* to their internal Channel Address table to determine whether they are the intended receiver.

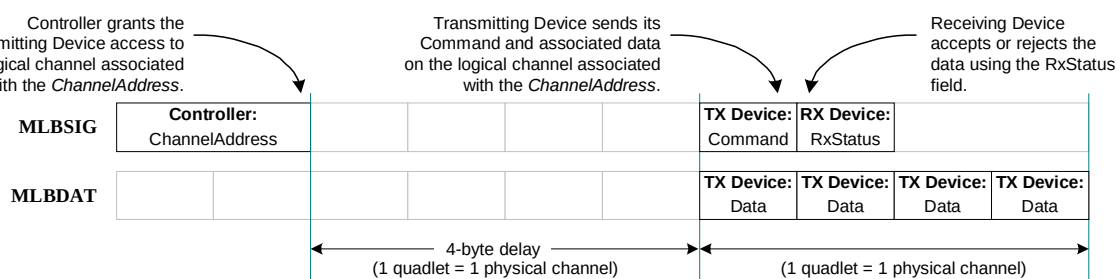


Figure 6-2: MediaLB Data Structure

When the OS81050 Controller is a receiving Device, it places the RxStatus response on **MLBSIG** one byte after the MediaLB *Command* is sent by the transmitting Device. To accept the data, an RxStatus response of *ReceiverReady* is sent. To reject the data, an RxStatus response of *ReceiverBusy* is sent.

In accordance with the *MediaLB Specification* [3], the OS81050 does not acknowledge synchronous data transmissions (RxStatus default response of *ReceiverReady*).

When the OS81050 is the receiving Device for a logical channel and recognizes a reception error, it responds with an RxStatus of *ReceiverProtocolError* in the next physical channel of that logical channel. The following conditions cause INIC to generate *ReceiverProtocolError*, when receiving control messages and asynchronous packets:

- Inappropriate MediaLB Command for a particular channel
(e.g. a logical channel is setup for control messages when a Synchronous Command is received).
- *ControlStart*, *AsyncStart*, *ControlEnd*, or *AsyncEnd* is received while in the middle of a packet reception.
- *ControlEnd* or *AsyncEnd* is not received when the end of the buffer is expected, based on *Port Message Length* (PML).
- *Port Message Header Length* (PMHL) + 3 bytes is not divisible by 4 (applies to packet messages only)

Following an end *Command* on an asynchronous or control RX channel (*AsyncEnd* or *ControlEnd*), the OS81050 will respond with an RxStatus of *ReceiverProtocolError* if it receives a start *Command* (*AsyncStart* or *ControlStart*) in one of the next two physical channels belonging to the same *ChannelAddress*. (Applicable to OS81050 Rev D only).

Figure 6-4 illustrates the MediaLB interface, as various Devices drive the bus (**MLBSIG/MLBDAT**) during a given network frame. This figure depicts the 256×Fs MediaLB speed; however, the logic also applies to the 512×Fs and 1024×Fs speeds (with more PCn channels).

The Controller drives the *ChannelAddresses* (*ChannelAddress* B through H in this example), granting the corresponding Devices bus access. The transmitting Device for the *ChannelAddress* drives the *Command* and *Data*, and the receiving Device responds with reception status (RxStatus). In the second to last physical channel of the frame (PC6 at 256×Fs), the Controller drives the *FRAMESYNC* pattern onto the signal information line (**MLBSIG**) in place of a *ChannelAddress*, for synchronization to the network frame. The *Command* and *Data* associated with the *FRAMESYNC* (also known as the *SystemChannel*) is sent by the Controller one quadlet following the *FRAMESYNC* pattern, in the first physical channel of the frame (PC0).

Depending on the number of physical channels grouped into logical channels, fewer unique *ChannelAddresses* may be seen in the frame. In Figure 6-4, each logical channel is one quadlet (one physical channel), mapping to seven *ChannelAddresses* (B through H). If one logical channel consisted of two quadlets and another consisted of three quadlets, then only four unique *ChannelAddresses* would be seen on the bus (B through E).

The OS81050 opens default communication channels (0004h and 0002h) when the MediaLB Port is enabled at power-up. The OS81050 listens to *ChannelAddress* 0004h for configuration information from the EHC, and responds to the EHC on *ChannelAddress* 0002h. This *ChannelAddress* pair is used for communicating ICM and MCM messages between the EHC and INIC, and can be used to open other channels on MediaLB. A separate channel pair must be opened for MDP communications.

In applications where the EHC does not support the MediaLB interface, an IOC device (e.g. OS85650 or OS85652) can be utilized to translate between INIC's MediaLB Port and a parallel interface, as depicted in Figure 6-3. In this implementation, the IOC allows the EHC to exploit the efficiency and bandwidth of the MediaLB bus, even when the EHC does not support a MediaLB interface. See Section 1.3.5 for more information regarding the IOC family of devices.

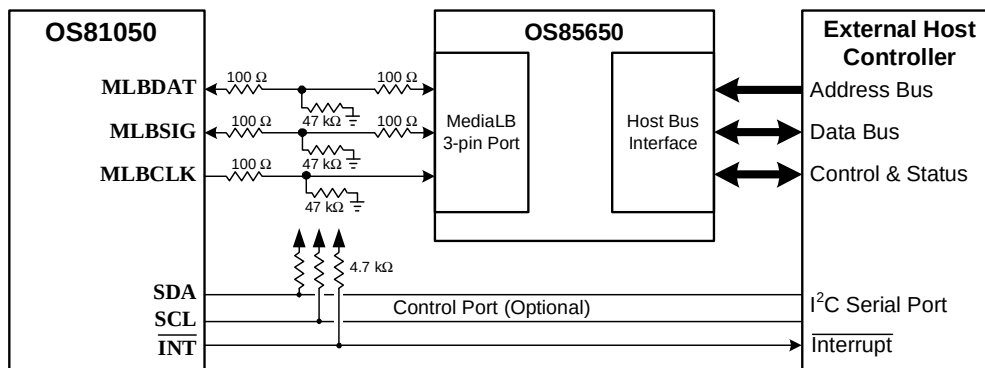


Figure 6-3: MediaLB 3-pin Mode to Parallel Interface

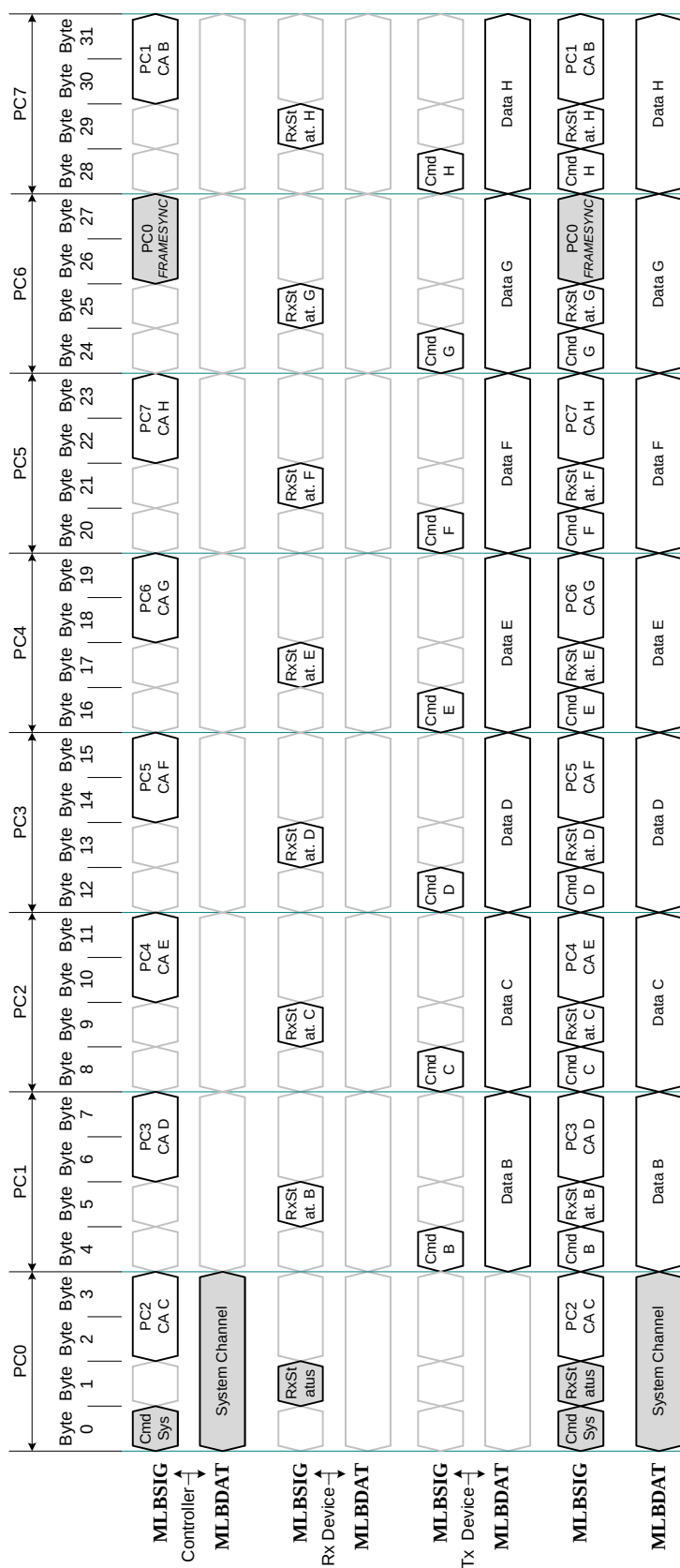


Figure 6-4: MediaLB Interface (at 256xFs)

6.2 MediaLB 5-pin Mode

The MediaLB 5-pin mode is functionally equivalent to the MediaLB 3-pin mode described in Section 6.1; however, two unidirectional data lines and two unidirectional signal information lines exist. Additionally, when configured for MediaLB 5-pin mode, the OS81050 supports a data rate of 256×Fs or 512×Fs (1024×Fs not supported).

The MediaLB 5-pin interface is defined as:

- **MLBDO** - unidirectional signal that transports data out of the MediaLB Port
- **MLBDI** - unidirectional signal that transports data into the MediaLB Port
- **MLBSO** - unidirectional signal that transports signal information out of the MediaLB Port
- **MLBSI** - unidirectional signal that transports signal information into the MediaLB Port
- **MLBCLK** - MediaLB Controller output signal that clocks data in and out of the MediaLB Port

The Streaming Port is not available when the MediaLB 5-pin interface is used.

The MediaLB Controller (OS81050) uses **MLBSO** to grant bus access to a MediaLB Device (e.g. EHC) and receives commands/responses sent out by other MediaLB Devices on **MLBSI**. The Controller receives data for configuration or transmission onto the network on **MLBDI**; data is transmitted to other MediaLB Devices from the network receiver on **MLBDO**.

If output signals on external MediaLB Devices can float (even during reset), then pull-down resistors are required to keep undriven signal and data lines from affecting the other Devices driving the lines.

Figure 6-5 shows the OS81050 as an interface between the MOST Network and the MediaLB Port, when configured for MediaLB 5-pin mode. Since the OS81050 is the interface to the MOST Network, it must be the Controller on the Media Local Bus. Using this interface, MediaLB Devices can access all MOST Network data types.

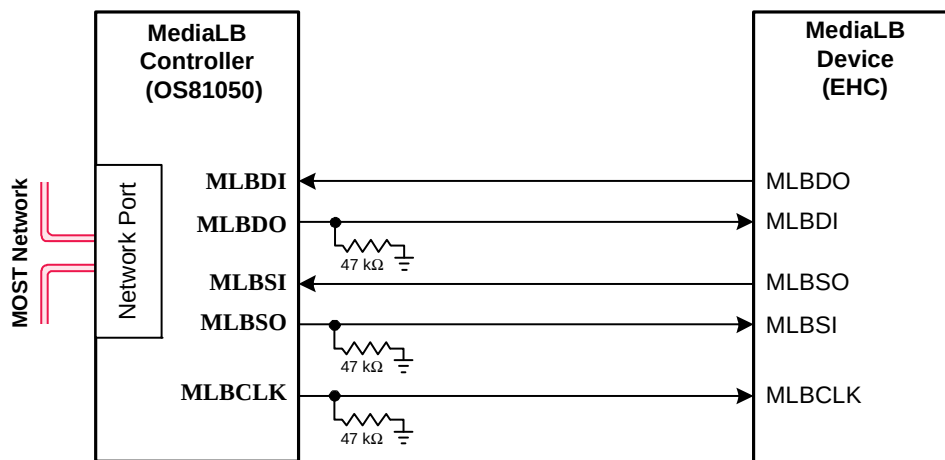


Figure 6-5: MediaLB 5-pin Mode Two-Node Connection Diagram

The MediaLB Port output lines (**MLBSO** and **MLBDO**) from each Device must be OR'ed with the corresponding output lines of the other Devices on the bus. As a result, each MediaLB Device must drive their output lines low when not transmitting data. When connecting more than one MediaLB Device on the 5-pin bus, the external logic needed to sum all data and signal outputs to each Device is shown in Figure 6-6.

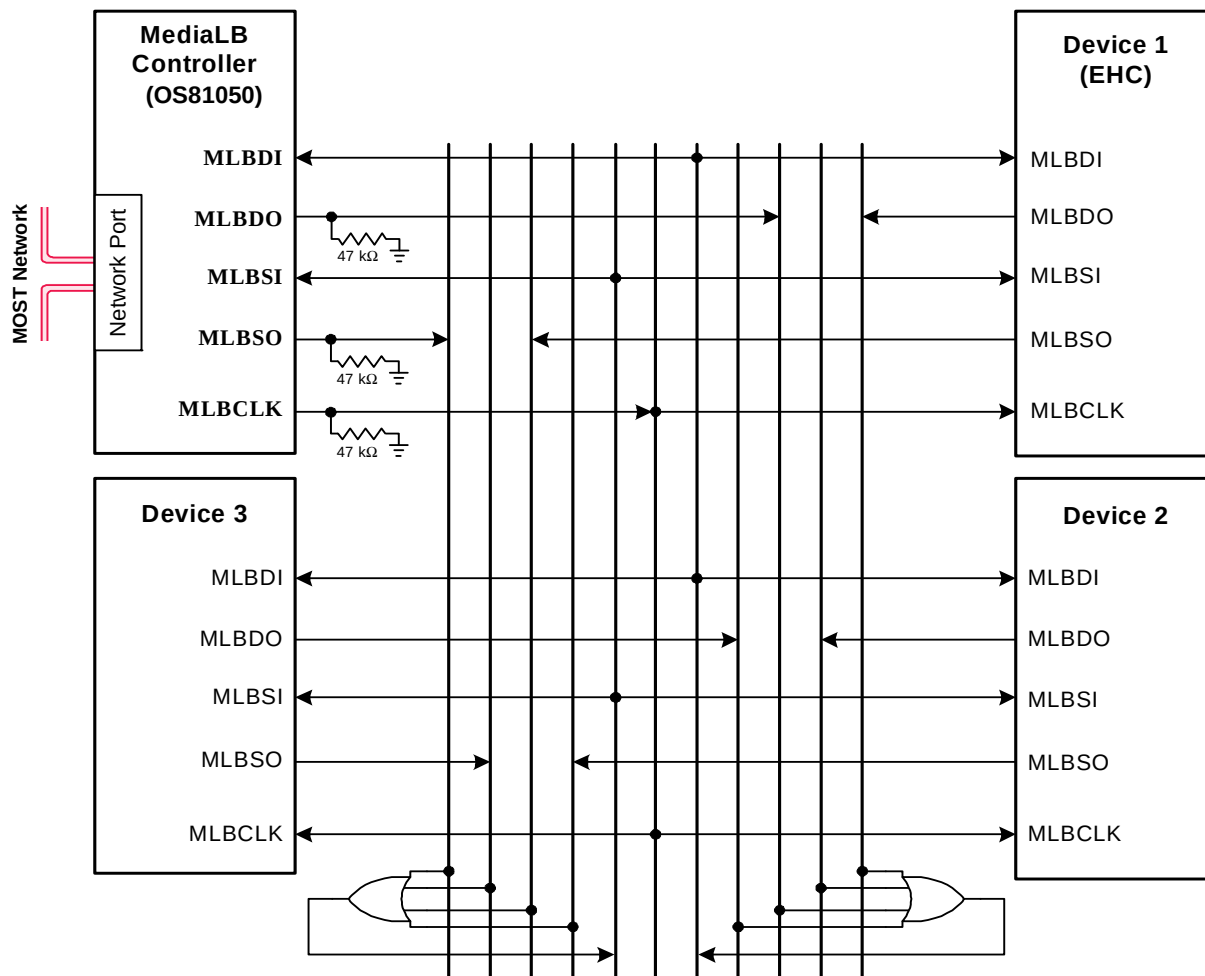


Figure 6-6: MediaLB 5-pin Mode Four-Node Connection Diagram

6.3 MediaLB Commands and Responses Not Supported

The OS81050 INIC does not support the following MediaLB Commands and Responses:

- Normal Commands:
 - All Isochronous Commands
- System Commands:
 - E0h - *MOSTLock*
 - E2h - *MOSTUnlock*
 - E4h - *MlbScan*
 - E6h - *MlbSubCmd*
 - FEh - *MlbReset*
- System Responses:
 - 80h - *DevicePresent*
 - 82h - *DeviceServiceRequest*

6.4 MediaLB Debug Header

A standard *MediaLB Debug Header* is defined to allow various MediaLB analysis and debug tools to connect directly into the application. This header is used for monitoring MediaLB in either the 3-pin or 5-pin mode, and should be placed as close as possible to the MediaLB interface to prevent unnecessary extension of MediaLB signal traces.

The pinout of the MediaLB Debug Header, as shown in Figure 6-7, is compatible with *MediaLB Active Pods* for MediaLB 3-pin and 5-pin modes. Active pods are used with various MediaLB analysis and debug tools [6, 7]. This header should be a 10-pin, 2 mm male connector, such as Molex 87332-1020 (shrouded) or Samtec TMM-105-06-T-D-SM (not shrouded).

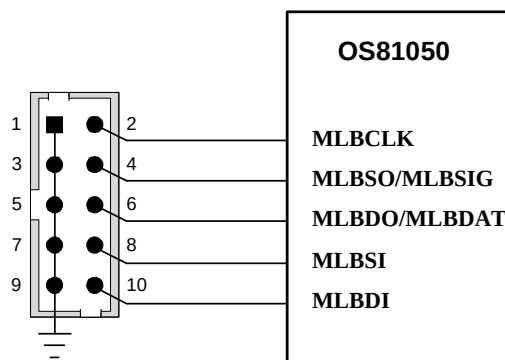


Figure 6-7: MediaLB Debug Header

7 Streaming Port

The Streaming Port is used by external devices (e.g. ADCs, DACs) to source/sink streaming data, with low overhead. The formats supported for data into and out of the Streaming Port are compatible with industry-standard serial interfaces, including: left-justified, right-justified, I²S-compatible or sequential formats.

The Streaming Port and the MediaLB Port share some pins; therefore, some modes are mutually exclusive. Table 7-1 shows the relationship between the availability of Streaming Port and MediaLB Port modes.

Streaming Port	MediaLB Port
Full Streaming Mode (all 4 serial interfaces available)	Not Available
Partial Streaming Mode (2 serial interfaces available)	3-pin mode
Not Available	5-pin mode

Table 7-1: Streaming Port vs. MediaLB Port Availability

A total of 60 bytes (maximum) can be transferred per frame, in either direction, for the Streaming Port and MediaLB Port combined.

Depending on the Streaming Port mode enabled, the OS81050 can support up to four serial data pins. All data pins on the Streaming Port share a common bit clock (SCK) and a common frame synchronization signal (FSY), which delineates the word/channel (left vs. right channel) boundaries. The bit clock (SCK) operates at $64 \times F_s$, $128 \times F_s$, or $256 \times F_s$, where F_s is the frame rate of FSY. When synchronous data is exchanged on the serial data pins, the FSY and SCK pins can be configured as inputs or outputs. As inputs, they must be synchronous to the OS81050 (derived off of RMCK). As outputs, FSY and SCK are driven based on the network frame rate.

The Streaming Port can be enabled, and the format selected, through the `OpenPort()` function, while the amount of streaming data transferred is selected through the `CreateSocket()` function. Figure 7-1 illustrates the pin usage when the OS81050 is configured for Full Streaming Mode or MediaLB 5-pin mode.

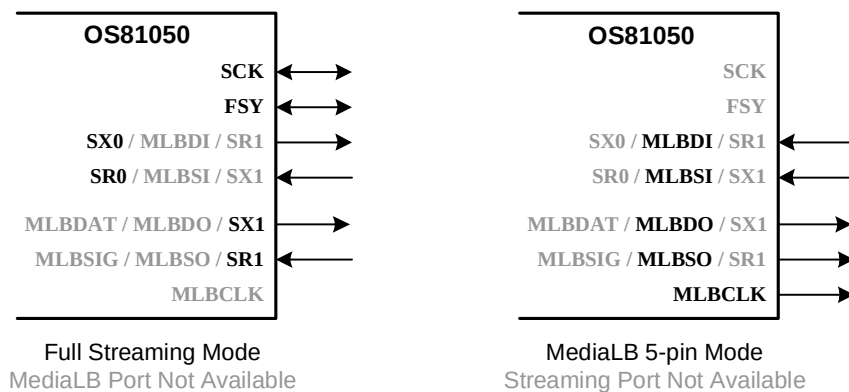


Figure 7-1: Full Streaming Mode vs. MediaLB 5-pin Mode

When the MediaLB Port is configured for MediaLB 3-pin mode, two serial interfaces are available through the Streaming Port. The following input/output configurations are available in Partial Streaming Mode:

- InOut Option - interface supports one serial input (**SR0**) and one serial output (**SX0**)
- DualIn Option - interface supports two serial inputs (**SR0** and **SR1**)
- DualOut Option - interface supports two serial outputs (**SX0** and **SX1**)

Pin usage for the various Partial Streaming Mode options is illustrated in Figure 7-2.

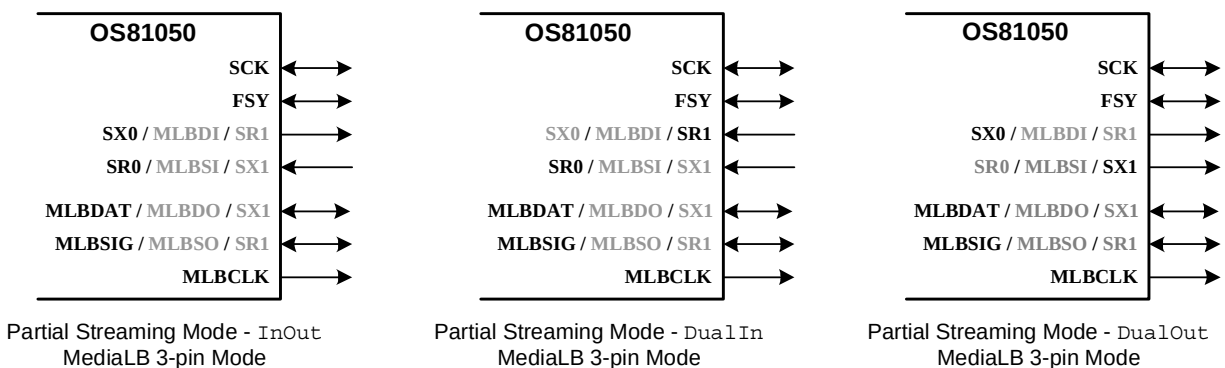


Figure 7-2: Partial Streaming Mode

Table 7-2 shows data format and clock speed options supported by the OS81050 Streaming Port. The data alignment associated with each streaming data format is shown in the following sections, with **SCK** set to $64 \times F_s$. At higher clock rates, the data alignment follows the same patterns; however, with more clocks per frame. With the sequential format (both bit-delayed and non-delayed), higher clock rates make more byte positions available.

Data Format	SCK Rate	Socket Size	Streaming Data Format*
Left-Justified	$64 \times F_s$	2 bytes (1x 16 bit channel)	Left64Fs16Bit
		4 bytes (2x 16 bit channels)	
		3 bytes (1x 24 bit channel)	Left64Fs24Bit
		6 bytes (2x 24 bit channels)	
	$128 \times F_s$	4 bytes (2x 16 bit channels)	Left128Fs16Bit
		6 bytes (2x 24 bit channels)	Left128Fs24Bit
Right-Justified	$64 \times F_s$	2 bytes (1x 16 bit channel)	Right64Fs16Bit
		4 bytes (2x 16 bit channels)	
		3 bytes (1x 24 bit channel)	Right64Fs24Bit
		6 bytes (2x 24 bit channels)	
	$128 \times F_s$	4 bytes (2x 16 bit channels)	Right128Fs16Bit
		6 bytes (2x 24 bit channels)	Right128Fs24Bit
Delayed-Bit (I^2S)	$64 \times F_s$	2 bytes (1x 16 bit channel)	Delay64Fs16Bit
		4 bytes (2x 16 bit channels)	
		3 bytes (1x 24 bit channel)	Delay64Fs24Bit
		6 bytes (2x 24 bit channels)	
		up to 8 bytes	Delay64FsSeq
	$128 \times F_s$	4 bytes (2x 16 bit channels)	Delay128Fs16Bit
		6 bytes (2x 24 bit channels)	Delay128Fs24Bit
		up to 16 bytes	Delay128FsSeq
	$256 \times F_s$	up to 32 bytes	Delay256FsSeq
Sequential	$64 \times F_s$	up to 8 bytes	Seq64Fs
	$128 \times F_s$	up to 16 bytes	Seq128Fs
	$256 \times F_s$	up to 32 bytes	Seq256Fs

* The streaming data format is configured using the `OpenPort()` function, as outlined in the *OS8105x/6x INIC API User's Manual* [2].

Table 7-2: Streaming Data Formats

7.1 Left-Justified Alignment

When using the left-justified alignment format, the first data bit occurs on the first clock after a transition on **FSY**. The data and **FSY** signals change on the falling edges of the clock and are valid on the rising edges. Data is arranged MSB first, high byte first. The start of frame is indicated by the rising edge of **FSY**.

Figure 7-3 illustrates the data alignment and timing when the left-justified format is selected. For higher data bit rates, the channel data remains aligned with the edges of **FSY**, with more unused data bytes between channels.

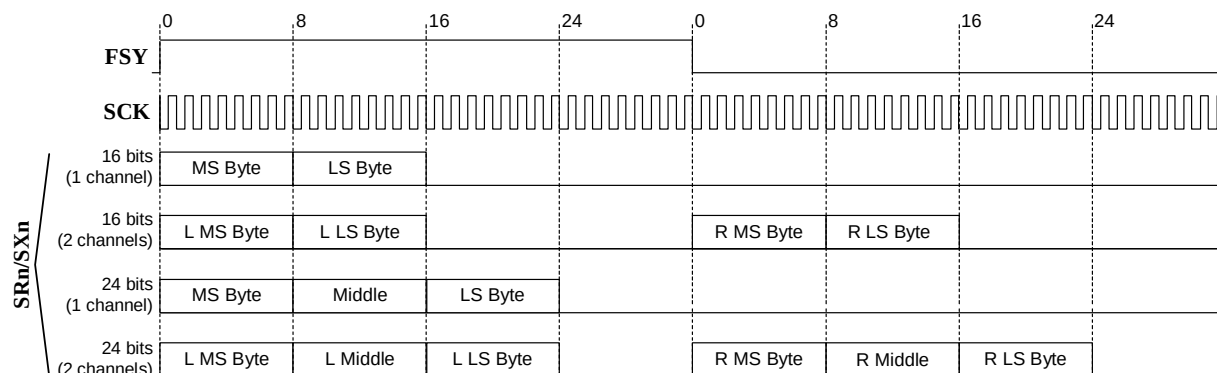


Figure 7-3: Left-Justified Streaming Data Format

The left-justified format does not support a single (mono) channel when **SCK** is $128 \times F_s$. This mode is supported by the sequential format (see Section 7.4).

7.2 Right-Justified Alignment

When using the right-justified alignment format, the last data bit occurs on the last clock before a transition on **FSY**. The data and **FSY** signals change on the falling edges of **SCK** and are valid on the rising edges. Data is arranged MSB first, high byte first. The start of frame is indicated by the rising edge of **FSY**.

Figure 7-4 illustrates the data alignment and timing when the right-justified format is selected. For higher data bit rates, the channel data remains aligned with the edges of **FSY**, with more unused data bytes between channels.

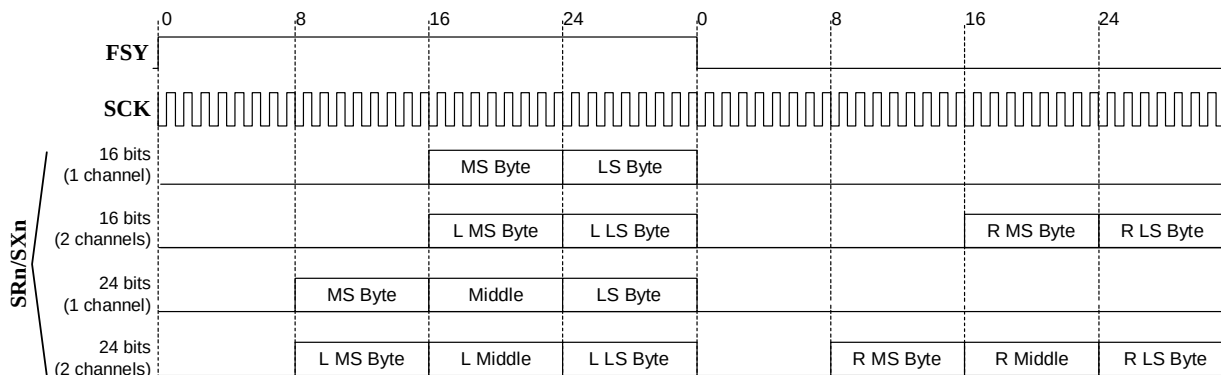


Figure 7-4: Right-Justified Streaming Data Format

The right-justified format does not support a single (mono) channel when **SCK** is $128 \times F_s$. This mode is supported by the sequential format (see Section 7.4).

7.3 Delayed-Bit Alignment

When using the delayed-bit alignment format, which is I²S compatible, there is a single **SCK** clock delay between the start of frame (falling edge of **FSY**) and the start of the frame data on **SRn/SXn**. The data and **FSY** signals change on the falling edges of **SCK** and are valid on the rising edges. Data is arranged MSB first, high byte first.

Figure 7-5 illustrates the data alignment and timing when the delayed-bit format is selected. For higher data bit rates, the channel data remains bit-delayed from the edges of **FSY**, with more unused data bytes between channels.

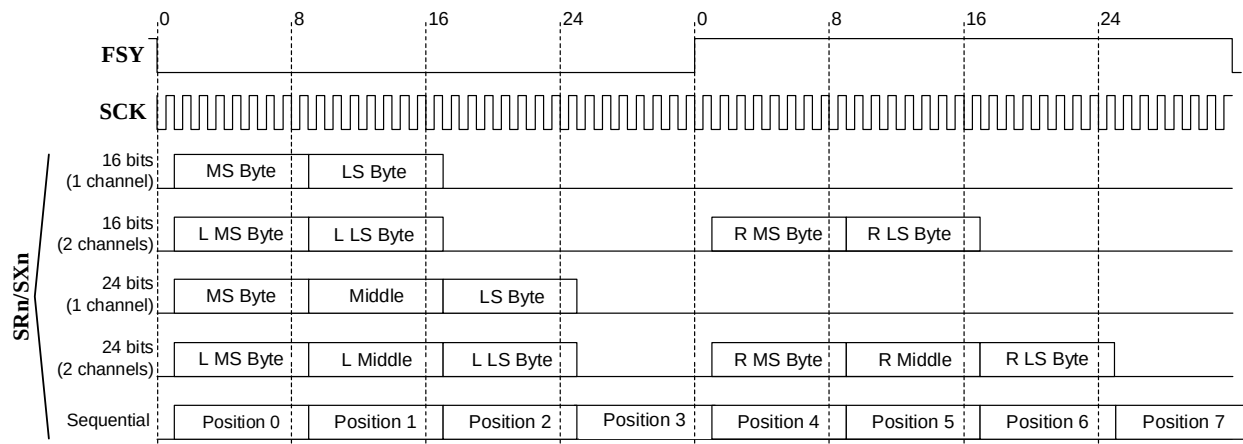


Figure 7-5: Delayed-Bit Streaming Data Format

When **SCK** is $128 \times F_s$ or $256 \times F_s$, a single (mono) channel is supported by the sequential, delayed-bit format (see Section 7.4). When using the sequential, delayed-bit format, byte positions in a frame remain bit-delayed and fixed (as shown in Figure 7-5); however, the actual number of byte positions *used* in the frame is flexible. This allows sockets of various sizes to be connected.

7.4 Sequential Alignment

When using the sequential (non-delayed) alignment format, the data and **FSY** signals change on the falling edges of the clock and are valid on the rising edges. Data is arranged MSB first, high byte first. This format allows data bytes to exist at every byte position in the frame; however, the actual number of byte positions *used* is flexible. This allows a socket of variable size to be connected.

Figure 7-6 illustrates the byte positions and timing for the sequential (non-delayed) alignment format when **SCK** is $64 \times F_s$. At higher **SCK** frequencies, more byte positions are available.

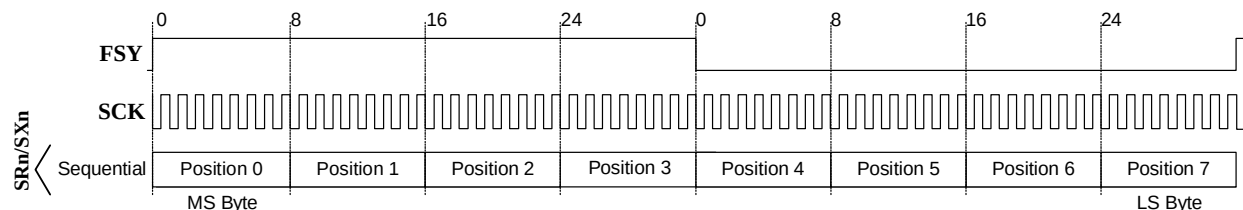


Figure 7-6: Sequential Streaming Data Format

Because the sequential alignment format does not place restrictions on socket size, this format can support a single (mono) channel. For example, with **SCK** at $128 \times F_s$, a single 16 bit data channel could be sent in the following byte positions:

- Positions 0 and 1 (for left-justified alignment)
- Positions 6 and 7 (for right-justified alignment)

8 External Power Management

The OS81050 INIC supports external power management logic by providing two inputs (**PS0** and **PS1**) and one output (**PWROFF**). The **PS0** and **PS1** inputs convey status of the external power management logic. The **PWROFF** output pin is driven low during normal operation and released when the OS81050 INIC is ready to be powered-off by the external power management logic.

The **PS0** and **PS1** inputs should indicate power management status, and communicate when an external power management event has occurred. Table 8-1 describes the power management states that are defined by the *MOST Specification* [1] and should be communicated via **PS0** and **PS1**. More information on system-wide INIC power management architectures can be found in the *MOST INIC Hardware Concepts Technical Bulletin* [8].

PS1	PS0	Status
0	0	U_{Normal}
0	1	Switch-To-Power (STP)
1	0	$U_{Critical}$, U_{Super}
1	1	U_{Low}

Table 8-1: External Power Management States

Hardware recognizes an external power management state when **PS0** and **PS1** remain at a valid logic level for a minimum time, t_{pshold} (see Section 11.5.2); however, the *INIC Software Stack* may use longer hold times for **PS0** and **PS1** states before responding to an external power management event. In addition, the *INIC Software Stack* controls how INIC responds to external power management events (e.g. driving **PWROFF**). If external power management is not used, **PS0** and **PS1** should be pulled to ground.

INIC recognizes an STP event upon power-up (or reset) and on a *transition* to **PS[1:0] = 01**. If these pins continue to indicate an STP event (**PS[1:0] = 01**) after an STP event has already been recognized, then the last known power state is assumed. If an STP event is indicated directly after power-up (or reset), then the U_{Normal} state is assumed. An STP event can cause INIC to enter *Ring-Break Diagnosis* mode, based on the `RBDOptions()` function parameter settings. See the *OS8105x/6x INIC API User's Manual* [2] for more information.

In a typical application, as illustrated in Figure 8-1, the power management circuitry is powered by a continuous supply and manages the switched power to the rest of the node. This circuitry also monitors the activity signal (**STATUS**) coming from the Fiber Optic Receiver (FOR) of the physical layer. When network activity is detected, the circuitry enables the switched supply and drives the **PS[1:0]** pins to the proper state, as depicted in Figure 8-2. When INIC powers up, it could take up to t_{stp} time for INIC to recognize the **PS[1:0]** state. When INIC is ready for shut down (such as when U_{Low} state is recognized), it drives the **PWROFF** pin high.

Additionally, when the U_{Low} state is recognized, **ERR/BOOT** is driven high and the network output (**TX**) is switched off. As long as the U_{Low} state is indicated on **PS[1:0]**, the **TX** output remains disabled, regardless of activity on **RX**.

If the EHC requires time after power-up (or reset) for initialization, it could utilize a **PWRHOLD** line that the circuitry detects. The use of **PWRHOLD** would suspend node power-down until the EHC is ready.

The `PMIConfig()` property specifies whether INIC drives **PWROFF** active when the network is shut down (NetInterface: Off state). Parameters optionally limit INIC from driving **PWROFF** (as a result of NetInterface: Off state) in specific EHCI states (e.g. `EHCIState()`). This property is set in the *INIC Configuration String* (see the *OS8105x/6x INIC API User's Manual* [2] for more information).

The INIC Configuration String may set `PMIConfig()`, such that INIC only drives **PWROFF** active when in *EHCI Protected Mode*. In this case, the EHC is notified of a network shutdown during *EHCI Attached Mode*, prepares itself for power-off, and sets the `EHCISState()` to `EHCI_Protected` when ready. INIC then drives **PWROFF** active and the node can be powered down. This example is applicable when the EHC does not utilize the **PWRHOLD** signal.

INIC supports a low-power mode through the use of *Device ShutDown*, which utilizes the `ShutDown()` function, in the `FBlock NBMIN`, to place the node in a low-power state. When the EHC receives this function it should minimize power consumption, if supported. The network can then re-start the node through the `ShutDown()` function, where INIC drives **INT** low to wake up the EHC.

SMSC's *Automotive Power Management Device* (MPM85000) performs all power management functionality required of MOST Network nodes, including: wake-up detection, power supply monitoring (**PS[1:0]**), reset generation, and *Switch To Power* (STP) pulse detection. The MPM85000 also supplies a 3.3 V continuous output for FOR power. Integrating both the OS81050 and MPM85000 on a MOST Network node minimizes application circuitry and software complexity allowing the application designer to focus on the application, rather than on MOST compliance. Contact SMSC for more information on the MPM85000.

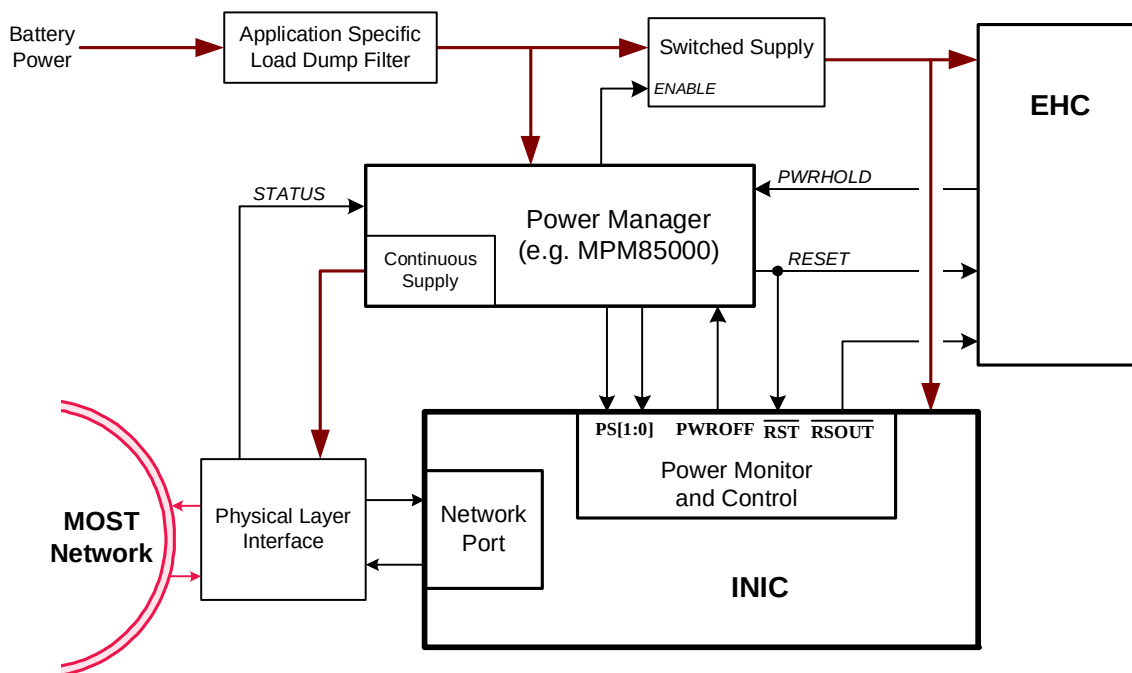


Figure 8-1: Typical Power Management Diagram

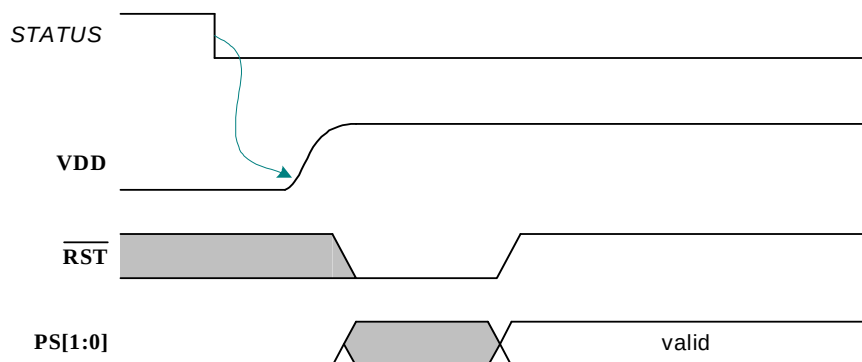


Figure 8-2: Power Management Startup

9 Clock Manager

The Clock Manager generates all internal clocks and two external clocks, **RMCK** and **SYNC**. The **RMCK** output clock supports synchronization of external devices. The **SYNC** output clock supports synchronization of switching power supplies. As shown in Figure 9-1, the Clock Manager consists of an input clock multiplexer, an analog PLL, and output clock dividers. The analog PLL generates the main clock, which is then divided down to generate all other internal clocks.

When configured as the network timing-master, the input clock multiplexer selects an external crystal oscillator (**XTI/XTO**) as the PLL locking source. While reset is asserted, the crystal oscillator is disabled and the PLL is pulled below the normal operating frequency. When reset is de-asserted, the crystal oscillator is automatically enabled and the PLL begins to lock.

For network timing-slave devices, the PLL is clocked from the network interface (**RX**). When not locked to the network, the external crystal oscillator is enabled in order to keep the PLL locked to a nominal frequency. Using the external crystal input allows INIC to lock faster when the network interface recovers and keeps all local clocks (**RMCK**, **MLBCLK**, **SCK**, etc.) close to the locked network frequency.

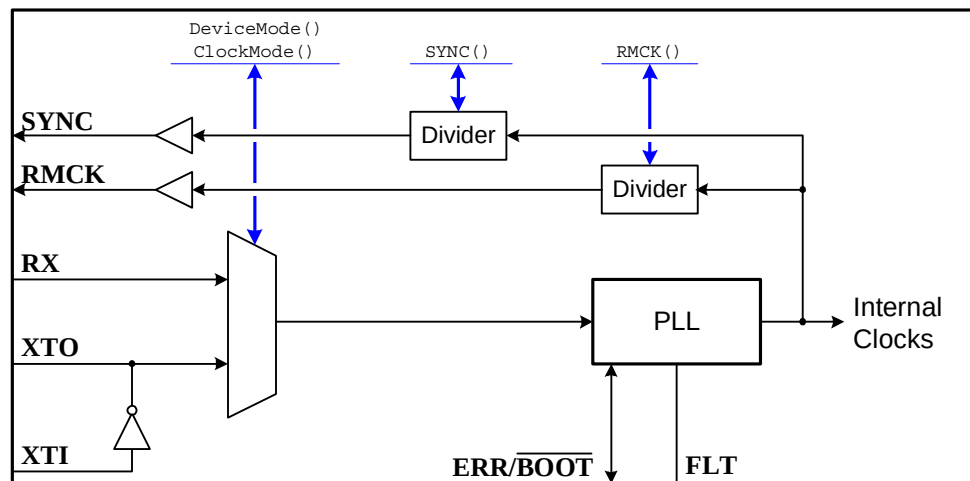


Figure 9-1: Clock Manager

9.1 PLL Lock Status

When the OS81050 initially obtains lock, it takes three frames to synchronize streaming data. Therefore, streaming data will not be transferred properly between the network and any open source or sink ports for three frames when going from the unlock to lock state. While the PLL is unlocked, only control messages targeting the local FBlock **INIC** or **NBMIN** can be transferred (via the Control Port or MediaLB Port).

Network unlock/lock events can affect the behavior of the **ERR/BOOT** pin, based on the **MuteMode()** setting. (For more information on the use of **ERR/BOOT** for lock/unlock detection, refer to the *OS8105x/6x INIC API User's Manual* [2].)

A valid and continuous external reference clock (**XTI/XTO**) must be present for normal operation. If the reference clock is disabled during operation, INIC may become inoperable and/or the PLL lock status may not be accurately reflected by the **ERR/BOOT** pin. A reset and valid reference clock is required to recover from this condition.

9.2 FLT Pin

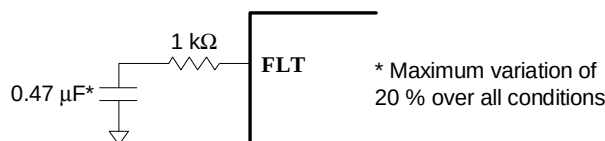


Figure 9-2: FLT Pin Circuit

The circuitry illustrated in Figure 9-2 is the optimum setup for the **FLT** pin of the OS81050. The **FLT** analog component should be placed as close as possible to the **GNDAn** pins to minimize loop currents. The **FLT** pin is a high-impedance node; therefore, leakage should be kept below 1 μ A or average pulse-width distortion tolerance could be adversely affected. Conformal coating is recommended for systems where condensation can occur.

In order to minimize vibration and shock effects on PLL locking, the capacitor used on **FLT** should be tolerant of vibrations typical in the application. Capacitors with X7R dielectrics are more sensitive to shock than metal-film, tantalum, and other ceramic capacitors, such as C0G and X8R.

The tolerance of the **FLT** capacitor must not vary by more than 20 % over all conditions. This includes the tolerance attributed to process, as well as degradation over life and temperature.

Since the layout and component selection of the **FLT** circuitry impact the sensitivity to vibrations, each design should be tested over the full environmental stress conditions of the intended application.

9.3 Crystal Pins (XTI/XTO)

The crystal oscillator is enabled when the OS81050 is configured as the timing-master and reset is de-asserted. Timing-slave nodes require a crystal oscillator to allow the node to run diagnostics when the network is down, and to keep the PLL locked to a nominal frequency when not locked to the network. During normal operation, the crystal oscillator of a timing-slave node may be enabled or disabled, depending on the setting of `MSVOptions()` in the *INIC Configuration String*. It is recommended that `MSVOptions().XTALConfiguration` be set to 1, allowing the crystal oscillator to remain enabled when the PLL is locked to **RX**. This provides improved frequency regulation and faster relocking after a network unlock.

The crystal oscillator should be in a fundamental mode, parallel resonant. Figure 9-3 depicts the external circuitry connected to the OS81050 oscillator circuit. Since the internal inverter/amplifier is operated in its linear region, external series resistors should not be used, as they will lower the gain and could cause start-up problems.

If an external clock is used in lieu of a crystal oscillator, it must support CMOS drive levels to the **VDDCn** supply and be connected to **XTI**, with **XTO** floating and having minimal capacitance (see Figure 9-3).

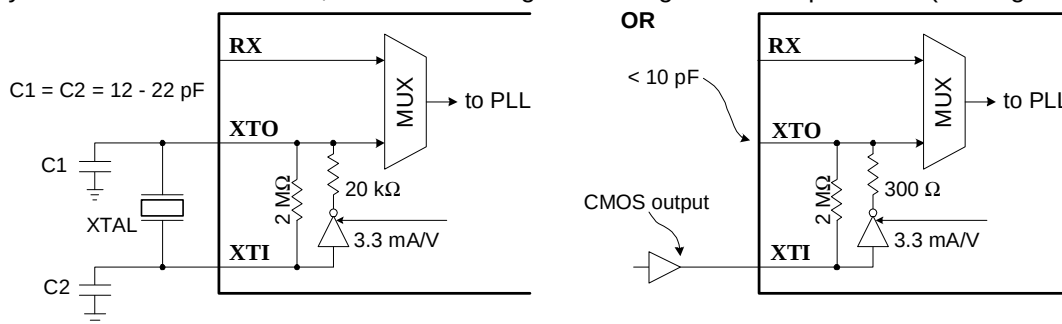


Figure 9-3: Crystal Oscillator Input

The crystal frequency must be 256 times the desired operational frequency (F_s) of the network. Therefore, if the network F_s is 44.1 kHz, the crystal needed is 11.2896 MHz. When the network F_s is 48 kHz, the crystal needed is 12.288 MHz. Several factors must be considered when selecting a crystal, including load capacitance, oscillator margin, cut, and operating temperature. For more information, refer to Section 13.6.

10 JTAG Port

The OS81050 implements the standard 4-pin Test Access Port (TAP) interface (**TDI**, **TDO**, **TCK**, and **TMS**), as defined by the IEEE 1149.1 Test Access Port standard. The optional test-reset input signal (**TRST**) is not supported by INIC. Therefore, INIC should be reset through **RST** when a reset at the TAP interface is needed. This port can also be used for configuration and debug, as defined in Section 13.5.

The test-data input (**TDI**) and test-data output (**TDO**) pins provide the means of shifting data into and out of the OS81050 boundary-scan register (BSR). The TAP interface is controlled by the test clock (**TCK**) and the test-mode select (**TMS**) pins. When **TMS** is low, the OS81050 TAP interface is enabled and the rising edge of **TCK** clocks in data from **TMS** and **TDI**. After reset, the **TDO** pin is high-impedance. **TDO** is driven by the TAP controller only when in the *Shift-IR* or *Shift-DR* state.

For proper operation, all OS81050 TAP interface pins should be pulled high through resistors. A 4.7 k Ω pull-up resistor is recommended for **TCK**, **TDI**, and **TDO** per the IEEE 1149.1 standard. A 47 k Ω resistor is recommended for **TMS**.

For more information on the IEEE 1149.1 Test Access Port standard, see *IEEE Std. 1149.1 "IEEE Standard Test Access Port and Boundary-Scan Architecture"* [9].

The OS81050 TAP interface supports all the mandatory boundary-scan instructions, as specified in the IEEE 1149.1 standard. Supported instructions include:

- **BYPASS** - Selects the bypass register (BYR) to be connected for serial access between **TDI** and **TDO**.
- **EXTEST** - Selects the boundary-scan register (BSR) to be connected for serial access between **TDI** and **TDO**.
- **SAMPLE/PRELOAD** - Captures a snapshot of the data at the digital pins during normal operation, and loads a data pattern into the boundary-scan register (BSR) prior to a new boundary-scan operation.
- **IDCODE** - (optional IEEE 1149.1 instruction) Used to read the Manufacturer ID from the identification register (IDR).

The OS81050 must be placed in *Boot Mode* before accessing the JTAG Port. In *Boot Mode*, the *INIC Software Stack* is disabled and the OS81050 waits for manual access through the JTAG Port. In this mode, standard communication channels (e.g. Control Port, MediaLB Port) are unavailable. The following sequence should be followed when using the OS81050 JTAG Port:

- Place the OS81050 in *Boot Mode*:
 - Hold the OS81050 in reset (drive **RST** pin low),
 - Hold the **ERR/BOOT** pin low,
 - Release the OS81050 from reset (**RST** pin high),
- Perform JTAG operations,
- Release the **ERR/BOOT** pin, and
- Reset the OS81050 to resume a normal mode of operation.

During *Boot Mode*, the **INT** pin is configured as an output and driven to **VDDCn**.

10.1 Instruction Register

The JTAG Instruction Register (IR) is accessed when the TAP controller is in the *Select-IR-scan*, *Capture-IR*, *Shift-IR*, *Exit1-IR*, *Pause-IR*, *Exit2-IR*, or *Update-IR* state.

IR JTAG Instruction Register

Bit	Label	Description	Default
3...0	IR[3:0]	Instruction Code	0001

Table 10-1: JTAG IR Register

IR[3:0] The 4-bit JTAG Instruction Register (IR) contains the instruction code for the next operation. Supported instructions include:

JTAG Instruction	IR[3:0]	Description
EXTEST	0000	Enables EXTEST operation
SAMPLE / PRELOAD	0001	Enables SAMPLE or PRELOAD operation
IDCODE	0010	Enables shifting out of the Manufacturer ID
OSSPRI	0110	Register must be set at the beginning of any JTAG sequence.
BYPASS	1111	Enables BYPASS
Reserved	All other codes	Not supported

10.2 Identification Register

The JTAG Identification Register (IDR) is accessed when the TAP controller is in the *Select-DR-scan*, *Capture-DR*, *Shift-DR*, *Exit1-DR*, *Pause-DR*, *Exit2-DR*, or *Update-DR* state.

IDR JTAG Identification Register

Bit	Label	Description	Default
31...28	REV[0:3]	4-bit Hardware Revision code	†
27...24	rsvd	Reserved	0000
23...16	PID[0:7]	8-bit Part Identifier	11110000
15...12	rsvd	Reserved	0000
11...1	MID[0:10]	11-bit Manufacturer ID	01000100010
0		Least significant bit (must be set per IEEE 1149.1 specification)	1

† The default value of REV[0:3] is dependent on the actual hardware revision.

Table 10-2: IDR Register

The JTAG IDR register is read out lsb first, bit 0 to bit 31. The bits below are listed in reverse order from the way they are read out.

REV[3:0] Hardware Revision.
 0001 – Revision B
 0010 – Revision C
 0011 – Revision D
 0100 – Revision E

PID[7:0] Part ID number.
 0Fh - OS81050

MID[10:0] Manufacturer ID.
 222h - SMSC Manufacturer ID

11 Electrical Characteristics

Specifications are subject to change without notice.

11.1 Absolute Maximum Ratings

Parameter (Note 1)	Min	Max	Unit
Storage Temperature	-65	150	°C
Junction Temperature		150	°C
Power Supply Voltage: Core (VDDCn)	-0.2	2.82	V
Analog (VDDAn)	-0.2	2.82	V
Peripheral (VDDPn)	-0.3	3.9	V
Power Dissipation		550	mW
DC Current to any Pin Except Power		±10	mA
Maximum Input Voltage:			
PS0, PS1	-0.5	VDDPn+0.3	V
XTI	-0.5	VDDCn+0.3	V
RX	-0.5	6	V
Other digital pins	-0.5	4.1	V

1. Operation at or above these limits may damage the device.

11.2 Guaranteed Operating Conditions

Parameter	Min	Max	Unit
Junction Temperature	-40	125	°C
Power Supply Voltage: Core (VDDCn)	2.375	2.625	V
Analog (VDDAn)	2.375	2.625	V
Peripheral (VDDPn)	3.135	3.465	V
Voltage applied to input pins:			
PS0, PS1	0	VDDPn	V
XTI	0	VDDCn	V
RX	0	5.5	V
Other digital pins	0	3.6	V
Operating Sample Frequency (Fs) (Note 1)	44.0	48.1	kHz
Digital Input Transition Rise/Fall rate (Notes 2, 3)		10	ns/V
Flash Erase/Program Cycles		1000	cycles

1. The operating network frame rate (Fs) is defined as the targeted network rate. The OS81050 is designed for operation in either a 44.1 kHz or 48 kHz network; however, testing is performed at 48 kHz.

2. Measured between V_{IL} and V_{IH} , unless otherwise noted.

3. Excludes digital input pins with transition times explicitly noted elsewhere in this chapter, as well as the $\overline{RST}$ pin, on which input hysteresis is specified (Section 11.5.2).

11.3 Thermal Characteristics

Parameter	Symbol	Value	Unit
Typical Junction to Package:			
QFP Multi-layer PCB with power and ground planes	Ψ_{JT}	6.8	°C/W
ETQFP Paddle soldered to the ground plane		2.1	°C/W
Typical Junction to Ambient:			
QFP Multi-layer PCB with power and ground planes	θ_{JA}	36	°C/W
ETQFP Paddle soldered to the ground plane		23	°C/W

11.4 DC Characteristics

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $G_{NDCn}, G_{NDAn}, G_{NDPn} = 0.0\text{ V}$;
Load Capacitance = 30 pF ; PLL locked at 48 kHz ; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Low-Level Input Voltage: XTI Other digital input pins	V_{IL}			$0.3 \times V_{DDCn}$ 0.7	V V	
High-Level Input Voltage: XTI Other digital input pins	V_{IH}	$0.7 \times V_{DDCn}$ 1.8			V V	
Input Leakage Current: MLBDAT, MLBSIG Other input pins	I_L			± 1 ± 10	μA μA	$0 < V_{in} < V_{DDCn}$ $0 < V_{in} < V_{DD}$
Low-Level Output Voltage: ERR, SYNC, SDA, SCL, INT, RSOUT TDO, DSDA, DSCL RMCK, SX0, SX1, FSY, SCK PWROFF TX, TXGAIN	V_{OL}			$0.15 \times V_{DDCn}$ $0.15 \times V_{DDCn}$ $0.15 \times V_{DDCn}$ $0.15 \times V_{DDPn}$ $0.15 \times V_{DDPn}$	V V V V V	$I_{OL} = 2\text{ mA}$ $I_{OL} = 2\text{ mA}$ $I_{OL} = 4\text{ mA}$ $I_{OL} = 2\text{ mA}$ $I_{OL} = 4\text{ mA}$
Low-Level Output Voltage: MLBCLK, MLBDAT, MLBSIG	V_{OL}			$0.05 \times V_{DDCn}$ $0.15 \times V_{DDCn}$	V V	$I_{OL} = 0.1\text{ mA}$ $I_{OL} = 8\text{ mA}$
High-Level Output Voltage: ERR, SYNC, TDO RMCK, SX0, SX1, FSY, SCK PWROFF TX	V_{OH}	$0.85 \times V_{DDCn}$ $0.85 \times V_{DDCn}$ $0.85 \times V_{DDPn}$ $0.85 \times V_{DDPn}$			V V V V	$I_{OH} = -2\text{ mA}$ $I_{OH} = -4\text{ mA}$ $I_{OH} = -2\text{ mA}$ $I_{OH} = -4\text{ mA}$
High-Level Output Voltage: MLBCLK, MLBDAT, MLBSIG	V_{OH}	$0.95 \times V_{DDCn}$ $0.85 \times V_{DDCn}$			V V	$I_{OH} = -0.1\text{ mA}$ $I_{OH} = -8\text{ mA}$
Digital Input Pin Capacitance: RX Other digital input pins				5 10	pF pF	
Analog Supply Current	I_A			25	mA	$F_s = 48.1\text{ kHz}$ Outputs unloaded
Digital Supply Current: Core Peripheral	I_{DDC} I_{DDP}			140 20	mA mA	$F_s = 48.1\text{ kHz}$ Outputs unloaded
Total Power Supply Current: $I_{DDC} + I_{DDP} + I_A$	I_{TOT}			185	mA	

11.5 Switching Characteristics

11.5.1 Clocks

$T_J = -40$ to 125 °C; $V_{DDCn}, V_{DDAn} = 2.5$ V ± 5 %; $V_{DDPn} = 3.3$ V ± 5 %; $G_{NDCn}, G_{NDAn}, G_{NDPn} = 0.0$ V;
Load Capacitance = 30 pF; PLL locked at 48 kHz; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
Rise Time: ERR, SYNC, TDO, PWROFF RMCK, SX0, SX1, FSY, SCK	t_{hdr}			12 5	ns ns	V_{OL} to V_{OH} $C_L = 30$ pF $C_L = 30$ pF
Rise Time: MLBCLK, MLBDAT, MLBSIG MLBCLK, MLBDAT, MLBSIG	t_{mlbr}			3 1	ns ns	0.7 to 1.8 V $C_L = 60$ pF $C_L = 40$ pF
Fall Time: ERR, SYNC, SDA, SCL, INT, RSOUT TDO, DSDA, DSCL, PWROFF RMCK, SX0, SX1, FSY, SCK, TXGAIN	t_{hdf}			12 12 5	ns ns ns	V_{OH} to V_{OL} $C_L = 30$ pF $C_L = 30$ pF $C_L = 30$ pF
Fall Time: MLBCLK, MLBDAT, MLBSIG MLBCLK, MLBDAT, MLBSIG	t_{mlbf}			3 1	ns ns	1.8 to 0.7 V $C_L = 60$ pF $C_L = 40$ pF
Clocks:						
Sample Frequency: PLL Locked PLL Unlocked	F_s	44.0 $0.95 \times f_{osc}$	48	48.1 $1.04 \times f_{osc}$	kHz kHz	(Note 1)
RMCK Master Clock Output	f_{rmck}	$64 \times F_s$		$1536 \times F_s$	Hz	when enabled
SYNC Clock Output	f_{sync}	$1 \times F_s$		$64 \times F_s$	Hz	when enabled
Crystal Oscillator (Note 2)	f_{osc}	11.264		12.3136	MHz MHz	$256 \times F_s$ at 44.0 kHz $256 \times F_s$ at 48.1 kHz
Jitter Tolerance (timing-master): XTI	t_{jm}	0.8			ns (pp)	(Note 3)

- Due to the on-chip frequency regulator, the PLL unlock time is very small and only occurs when switching the PLL input source between the network (**RX**) and the crystal (**XTI/XTO**). At power-up, the PLL starts from zero; therefore, the minimum PLL unlock frequency is only applicable *after* the first network loss of lock. In addition, the minimum PLL unlock frequency assumes a crystal start-up time of less than 1.4 ms when the crystal is reenabled after each network unlock (e.g. when `MSVOptions().XTALConfiguration` is set to 0). When this start-up time cannot be met, the crystal may remain enabled when the PLL is locked to **RX** with `MSVOptions().XTALConfiguration` set to 1.
- The *MOST Specification* requires each node have a crystal-based master-clock source to support ring-break diagnostics, where nodes normally configured as timing-slaves can be re-configured as the timing-master. The crystal frequency must be 256 times the desired operational frequency of the network.
- Jitter Tolerance: The OS81050 tolerates *at least* the amount of clock jitter defined by the t_{jm} (minimum) specification without errors.

11.5.2 Reset and Power Management

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $G_{NDCn}, G_{NDAn}, G_{NDPn} = 0.0\text{ V}$;
Load Capacitance = 30 pF ; PLL locked at 48 kHz ; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Comment
PS[1:0] transition time	$t_{pstrans}$			0.75	ms	(Notes 1, 2, 3)
PS[1:0] hold time	t_{pshold}	2.2			ms	
RST rising to PS[1:0] recognition	t_{stp}	5		100	ms	
RST Input Hysteresis	$\Delta V_{hys-rst}$	200			mV	
RST pulse width	t_{rspw}	150			ns	(Notes 4, 5, 6)
Configuration pin setup to RST rising: INT ERR/BOOT	t_{cpsrs}	0 0			ns ns	
Configuration pin hold from RST rising: INT ERR/BOOT	t_{cphrs}	25 2.5			ms ms	
RST rising to TX output active	t_{bypass}	10		30	ms	(Note 7)

1. Maximum time allowed by hardware for **PS[1:0]** to settle. Longer transition times may cause hardware to recognize an erroneous power management state.
2. Minimum time required for an external power management state (**PS[1:0]**) to be recognized by hardware. The INIC Software Stack may define a longer hold time for responding to an external power event.
3. Immediately following power-up (or reset), the **PS[1:0]** state is recognized after t_{stp} . This requires t_{pshold} be at least $t_{stp}(\text{max})$ after power-up (or reset).
4. After both the 3.3 V and 2.5 V power supplies have stabilized, INIC must be reset for normal operation. A proper reset is when both supplies are $\geq 90\%$ VDD; however, electrical specifications are applicable only at $\pm 5\%$ VDD.
5. Power-up configuration pins include: **ERR/BOOT** and **INT**; however, **INT** is only used as a configuration pin when `PortConfiguration.DefPort()` is set to `INT_Select`. When used as a configuration pin, an external component (e.g. pull-up resistor) must hold **INT** in a valid state (high or low) for at least t_{cphrs} or until **INT** is driven low by INIC, whichever occurs first.
6. The **RST** pulse width indicates the minimum time required to reset the part; however, the configuration pins can take longer to settle to their default state, based on the trace capacitance and size of the on-board pull-up or pull-down resistor. Therefore, the **RST** pulse width must be long enough to allow the external configuration pins to achieve their default state.
7. During reset and immediately following, the INIC transceiver is *inactive* and **RX** is routed directly to **TX** (also known as electrical bypass). After t_{bypass} , the INIC transceiver becomes *active* and **TX** is actively driven.

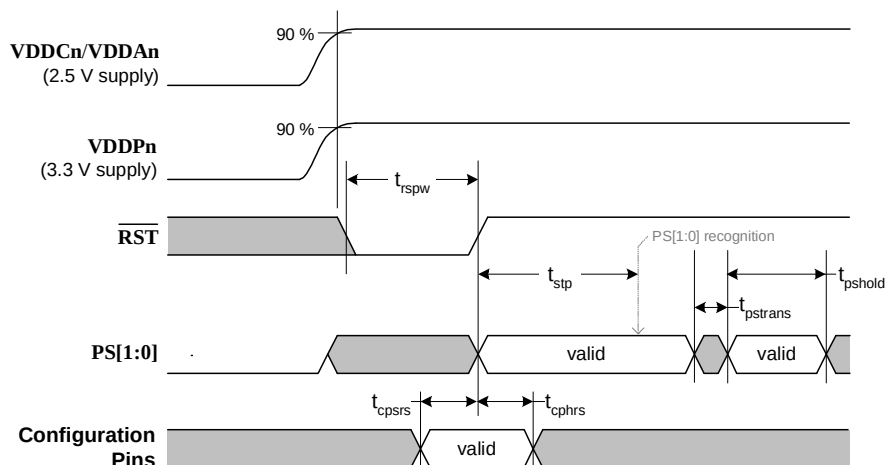


Figure 11-1: Initial Power-Up and Reset Timing

11.6 Network Characteristics

$T_j = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $G_{NDCn}, G_{NDAn}, G_{NDPn} = 0.0\text{ V}$;
Load Capacitance = 30 pF ; PLL locked at 48 kHz ; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Test Conditions
RX Rise/Fall time	t_{rxt}			0.45	UI	Recommended
RX Pulse Width Variation (timing-master) - (Note 1)	t_{pwmn} t_{pwmx}	0.7		1.4	UI UI	Measured at 50 % VDDPn
RX Average Pulse Width Distortion (timing-slave) - (Note 2)	t_{apwd}	-0.18		0.36	UI	
RX Jitter + Drift Tolerance (timing-master) - (Note 3)	t_{jmr}	5			UI	
TX Rise/Fall time	t_{txt}			4.6	ns	10 % to 90 % of VDDP $C_L = 20\text{ pF}$
TX Jitter Generation (Data Dependent + Uncorrelated Jitter)	t_{jittx}			0.0045	UI	
TX Pulse Width Variation	t_{pwmn} t_{pwmx}	0.955		1.045	UI UI	Measured at 50 % VDDPn
TX Average Pulse Width Distortion	t_{apwd}	-0.023		+0.023	UI	

- When the MOST Network frequency is 48 kHz , one UI is 20.345 ns , which is $\frac{t_{rxbp}}{2}$. The pulse width variation is defined as the sum of the Average Pulse Width Distortion plus high-frequency jitter.
- The Average Pulse Width Distortion (APWD) specification, defined as $t_{apwd} = \frac{t_{pwmx} + t_{pwmn} - t_{rxbp}}{2}$, is illustrated in Figure 11-2. The **FLT** pin is a high-impedance node; therefore, leakage current should be kept below $1\text{ }\mu\text{A}$, or APWD tolerance could be adversely affected.
- Jitter Tolerance: The OS81050 tolerates *at least* the amount of jitter defined by the t_{jmr} (minimum) specification without errors.

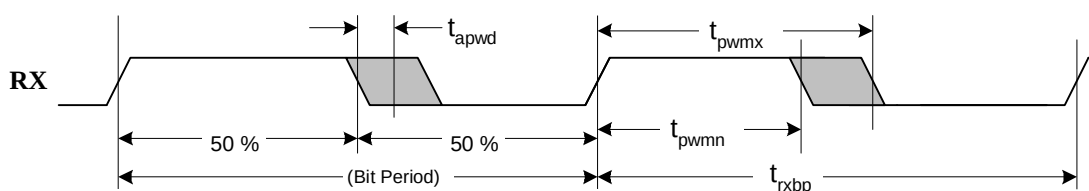


Figure 11-2: RX Pulse Width Distortion Timing

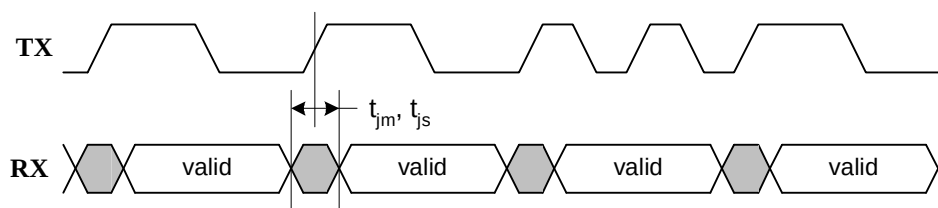


Figure 11-3: RX Jitter Tolerance Timing

11.7 MediaLB Port (3-pin)

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $G_{NDCn}, G_{NDAn}, G_{NDPn} = 0.0\text{ V}$;
Load Capacitance = 60 pF ; MediaLB Speed of $256\times F_s$ or $512\times F_s$; PLL locked at 48 kHz ; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Comment
MLBCLK Operating Frequency (Note 1)	f_{mck}	11.264	12.288 24.576	24.6272 25.600	MHz MHz MHz MHz MHz	$256\times F_s$ at 44.0 kHz $256\times F_s$ at 48.0 kHz $512\times F_s$ at 48.0 kHz $512\times F_s$ at 48.1 kHz $512\times F_s$ PLL unlocked
MLBCLK cycle time	t_{mckc}		81 40		ns ns	$256\times F_s$ $512\times F_s$ (Note 2)
MLBCLK low time	t_{mckl}	31.5 30	37 35.5		ns ns	$256\times F_s$ $256\times F_s$ PLL unlocked
		14.5 14	17 16.5		ns ns	$512\times F_s$ $512\times F_s$ PLL unlocked
MLBCLK high time	t_{mckh}	31.5 30	38 36.5		ns ns	$256\times F_s$ $256\times F_s$ PLL unlocked
		14.5 14	17 16.5		ns ns	$512\times F_s$ $512\times F_s$ PLL unlocked
MLBCLK pulse width variation	t_{mpwv}			2	ns (pp)	(Note 3)
MLBSIG/MLBDAT receiver input valid to MLBCLK falling	t_{dsmcf}	1			ns	
MLBSIG/MLBDAT receiver input hold from MLBCLK low	t_{dhmcf}	0			ns	
MLBSIG/MLBDAT output high impedance from MLBCLK low	t_{mcfdz}	0		t_{mckl}	ns	
Bus hold from MLBCLK low	t_{mdzh}	4			ns	(Note 4)
MLBSIG/MLBDAT output valid from MLBCLK high	t_{mcdrv}			8	ns	

1. The OS81050 can shut off **MLBCLK** to place MediaLB in a low-power state.
2. The Factory default is 3-pin MediaLB mode at $512\times F_s$, but can be reconfigured through the *Customer Configuration Interface*.
3. Pulse width variation is measured at 1.25 V by triggering on one edge of **MLBCLK** and measuring the spread on the other edge, measured in ns peak-to-peak (pp).
4. The board must be designed to ensure that the high-impedance bus does not leave the logic state of the final driven bit for this time period. Therefore, coupling must be minimized while meeting the maximum capacitive load listed.

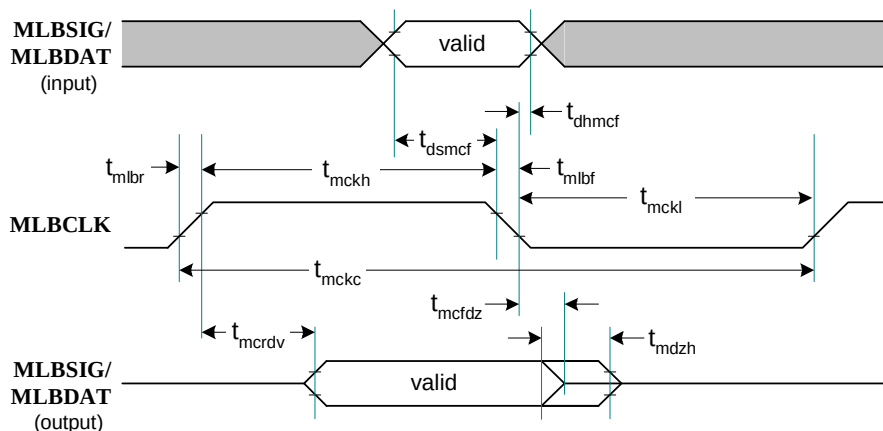


Figure 11-4: MediaLB 3-pin Timing

T_J = -40 to 125 °C; VDDCn, VDDAn = 2.5 V ±5 %; VDDPn = 3.3 V ±5 %; GND Cn, GNDAn, GNDPn = 0.0 V;
Load Capacitance = 40 pF; MediaLB Speed of 1024×Fs; PLL locked at 48 kHz; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Comment
MLBCLK Operating Frequency (Note 1)	f _{mck}	45.056	49.152	49.2544 51.200	MHz MHz MHz MHz	1024×Fs at 44.0 kHz 1024×Fs at 48.0 kHz 1024×Fs at 48.1 kHz 1024×Fs PLL unlocked
MLBCLK cycle time	t _{mckc}		20.3		ns	
MLBCLK low time	t _{mckl}	6.5 6.1	7.7 7.3		ns ns	PLL unlocked
MLBCLK high time	t _{mckh}	9.7 9.3	10.6 10.2		ns ns	PLL unlocked
MLBCLK pulse width variation	t _{mpwv}			0.6	ns (pp)	(Note 2)
MLBSIG/MLBDAT receiver input valid to MLBCLK falling	t _{dsmcf}	1			ns	
MLBSIG/MLBDAT receiver input hold from MLBCLK low	t _{dhmcf}	0			ns	
MLBSIG/MLBDAT output high impedance from MLBCLK low	t _{mcfdz}	0		t _{mckl}	ns	(Note 3)
Bus hold from MLBCLK low	t _{mdzh}	2			ns	(Note 3)
MLBSIG/MLBDAT output valid from MLBCLK high	t _{mcrdv}			7	ns	

1. The OS81050 can shut off **MLBCLK** to place MediaLB in a low-power state.
2. Pulse width variation is measured at 1.25 V by triggering on one edge of **MLBCLK** and measuring the spread on the other edge, measured in ns peak-to-peak (pp).
3. The board must be designed to ensure that the high-impedance bus does not leave the logic state of the final driven bit for this time period. Therefore, coupling must be minimized while meeting the maximum capacitive load listed.

11.8 MediaLB Port (5-pin)

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $GND_{Cn}, GND_{An}, GND_{Pn} = 0.0\text{ V}$;
Load Capacitance = 40 pF ; MediaLB Speed of $256\times F_s$ or $512\times F_s$; PLL locked at 48 kHz ; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Comment
MLBCLK Operating Frequency (Note 1)	f_{mck}	11.264	12.288 24.576	24.6272	MHz MHz MHz MHz	$256\times F_s$ at 44.0 kHz $256\times F_s$ at 48.0 kHz $512\times F_s$ at 48.0 kHz $512\times F_s$ at 48.1 kHz
MLBCLK cycle time	t_{mckc}		81 40		ns ns	$256\times F_s$ $512\times F_s$ (Note 2)
MLBCLK low time	t_{mckl}	31.5	37		ns	$256\times F_s$
		30	35.5		ns	$256\times F_s$ PLL unlocked
		15.5	17		ns	$512\times F_s$
		15	16.5		ns	$512\times F_s$ PLL unlocked
MLBCLK high time	t_{mckh}	31.5	38		ns	$256\times F_s$
		30	36.5		ns	$256\times F_s$ PLL unlocked
		15.5	17		ns	$512\times F_s$
		15	16.5		ns	$512\times F_s$ PLL unlocked
MLBCLK pulse width variation	t_{mpww}			2	ns (pp)	(Note 3)
MLBSI/MLBDI valid to MLBCLK falling	t_{dsmcf}	3			ns	(Note 4)
MLBSI/MLBDI hold from MLBCLK low	t_{dhmcf}	5			ns	
MLBSO/MLBDO low from MLBCLK high	t_{mcrdl}			20 10	ns ns	$256\times F_s$ $512\times F_s$
MLBSO/MLBDO valid from MLBCLK high	t_{mcrdv}			8	ns	

1. The OS81050 can shut off **MLBCLK** to place MediaLB in a low-power state. In addition, the frequency can be slightly higher when network lock is lost (see F_s in Section 11.5).
2. The Factory default is 3-pin MediaLB mode at $512\times F_s$, but can be reconfigured through the *Customer Configuration Interface*.
3. Pulse width variation is measured at 1.25 V by triggering on one edge of **MLBCLK** and measuring the spread on the other edge, measured in ns peak-to-peak (pp).
4. Gate delays due to OR'ing logic on the pins must be taken into consideration.

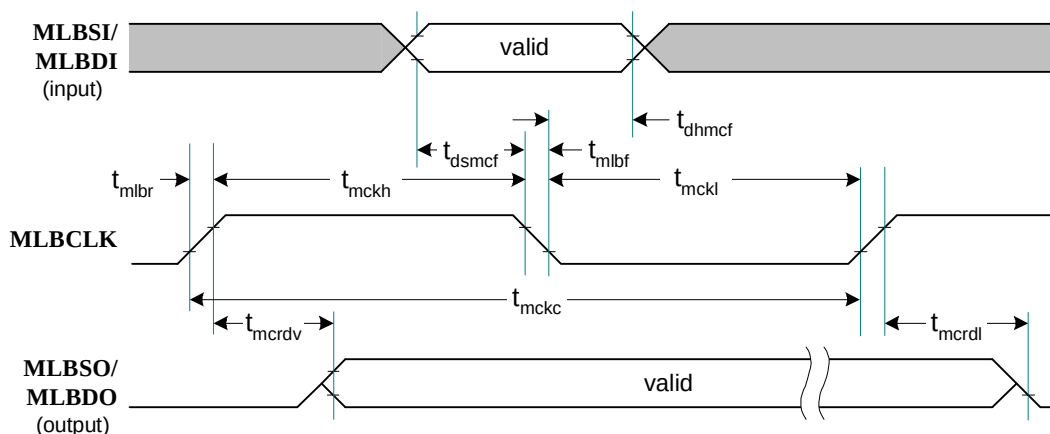


Figure 11-5: 5-Pin MediaLB Timing

11.9 Streaming Port

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $GND_{Cn}, GND_{An}, GND_{Pn} = 0.0\text{ V}$;
Load Capacitance = 30 pF ; PLL locked at 48 kHz ; unless otherwise noted.

Parameter	Symbol	Min	Typ	Max	Unit	Comment
FSY frequency (Note 1)	f_{fsy}	44.0	48	48.1	kHz	
SCK frequency (Note 1)	f_{sck}	2.816		12.3136	MHz MHz	$64 \times F_s$ at 44.0 kHz $256 \times F_s$ at 48.1 kHz
SCK low time	t_{sckl}	25			ns	
SCK high time	t_{sckh}	25			ns	
SR[1:0] valid to SCK rising	t_{srs}	25			ns	(Notes 2, 4)
SR[1:0] hold from SCK rising	t_{srh}	25			ns	(Notes 2, 4)
SX[1:0] valid from SCK falling	t_{sxv}			30	ns	(Notes 2, 4)
SCK and FSY Outputs:						
SCK falling to FSY valid	t_{fsyv}	-25		25	ns	(Notes 2, 3)
SCK and FSY Inputs:						
FSY valid to SCK rising	t_{fsys}	25			ns	(Notes 2, 3)
FSY hold from SCK rising	t_{fsyh}	25			ns	(Notes 2, 3)

1. If **SCK** and **FSY** are inputs, they must be frequency locked to the master clock (**RMCK** output clock). In addition, the frequency can be slightly higher when network lock is lost (see F_s in Section 11.5).
2. Data is stable at the rising edge of **SCK**, as shown in Figure 11-6.
3. **FSY** polarity is determined by the data format selected when the `SCM_OpenPort()` function is called.
4. The MSB of **SR[1:0]** and **SX[1:0]** is the first or the second bit after **FSY** changes, based on the format selected.

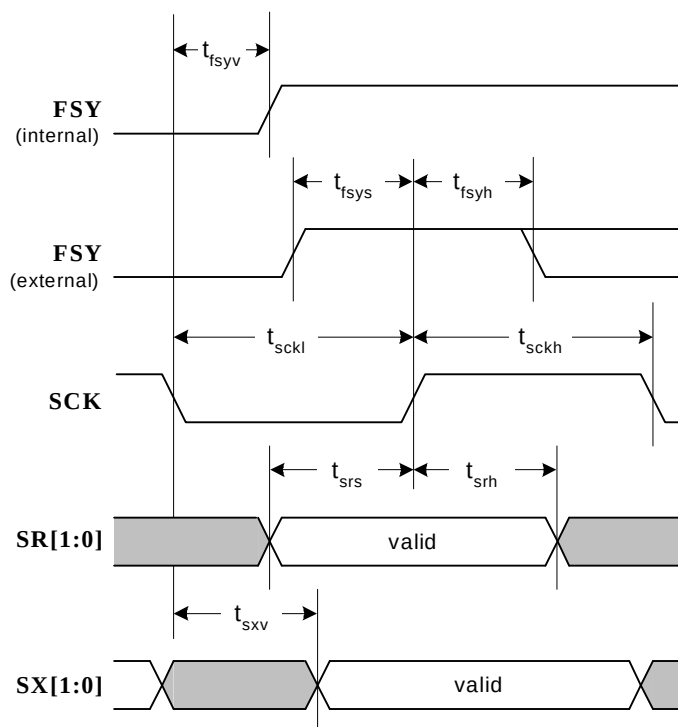


Figure 11-6: Streaming Port Timing

11.10 Control Port

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $GND_{Cn}, GND_{An}, GND_{Pn} = 0.0\text{ V}$;
Load Capacitance = 40 pF ; PLL unlocked; unless otherwise noted.

Parameter	I ² C Slave				
	Symbol	Min	Typ	Max	Unit
SCL frequency	f_{scl}			400	kHz
Bus free between transmissions (SDA high time between start and stop)	t_{buf}	1.3			μs
Start condition hold time (SDA falling to SCL falling)	t_{stah}	0.6			μs
SCL low	t_{scll}	1.3			μs
SCL high	t_{sclh}	0.6			μs
SDA input hold from SCL falling	t_{sdah}	0		900	ns
SDA input valid to SCL rising	t_{sdas}	100			ns
(repeated) start condition setup time	t_{stas}	0.6			μs
SDA and SCL rise time	t_r			300	ns
SDA and SCL fall time	t_f			300	ns
Stop condition setup time (SCL rising to SDA rising)	t_{stps}	0.6			μs

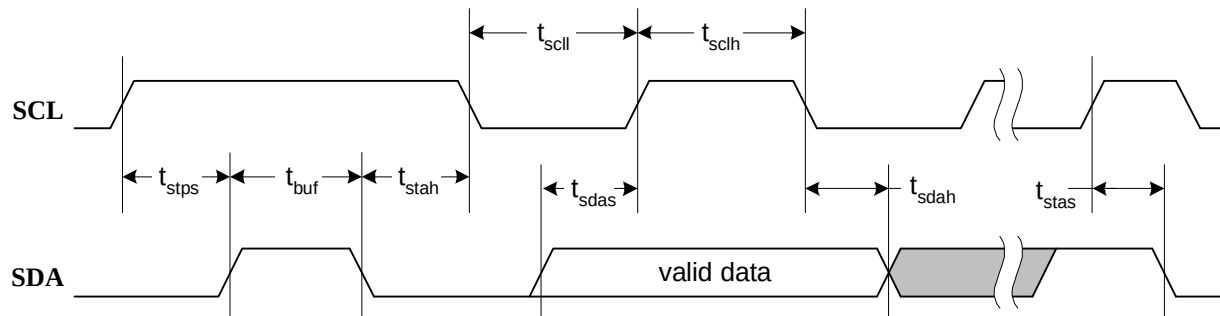


Figure 11-7: Control Port Timing

11.11 JTAG Timing

$T_J = -40$ to $125\text{ }^{\circ}\text{C}$; $V_{DDCn}, V_{DDAn} = 2.5\text{ V} \pm 5\%$; $V_{DDPn} = 3.3\text{ V} \pm 5\%$; $GND_{Cn}, GND_{An}, GND_{Pn} = 0.0\text{ V}$;
Load Capacitance = 30 pF .

Parameter	Symbol	Min	Max	Unit
Maximum TCK Frequency	f_{tck}		10	MHz
TDO output valid high from TCK falling edge	t_{tdovl}		25	ns
TDO output valid low from TCK falling edge	t_{tdovh}		25	ns
TMS and TDI input setup to TCK rising edge	t_{tmss}	5		ns
TMS and TDI input hold from TCK rising edge	t_{tmsh}	5		ns

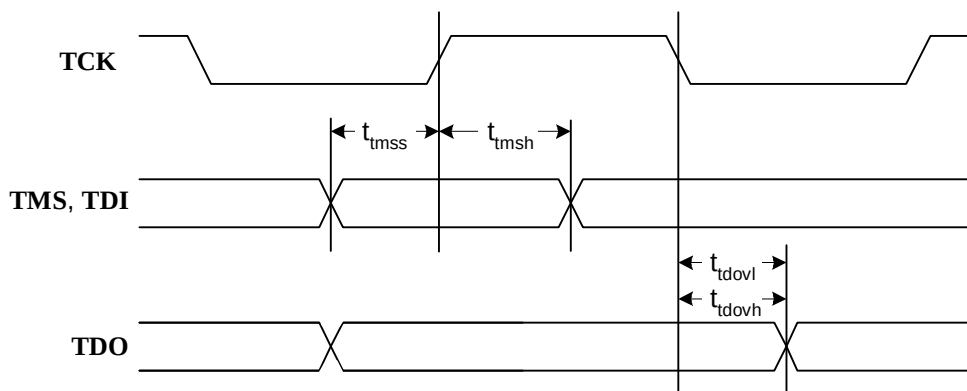
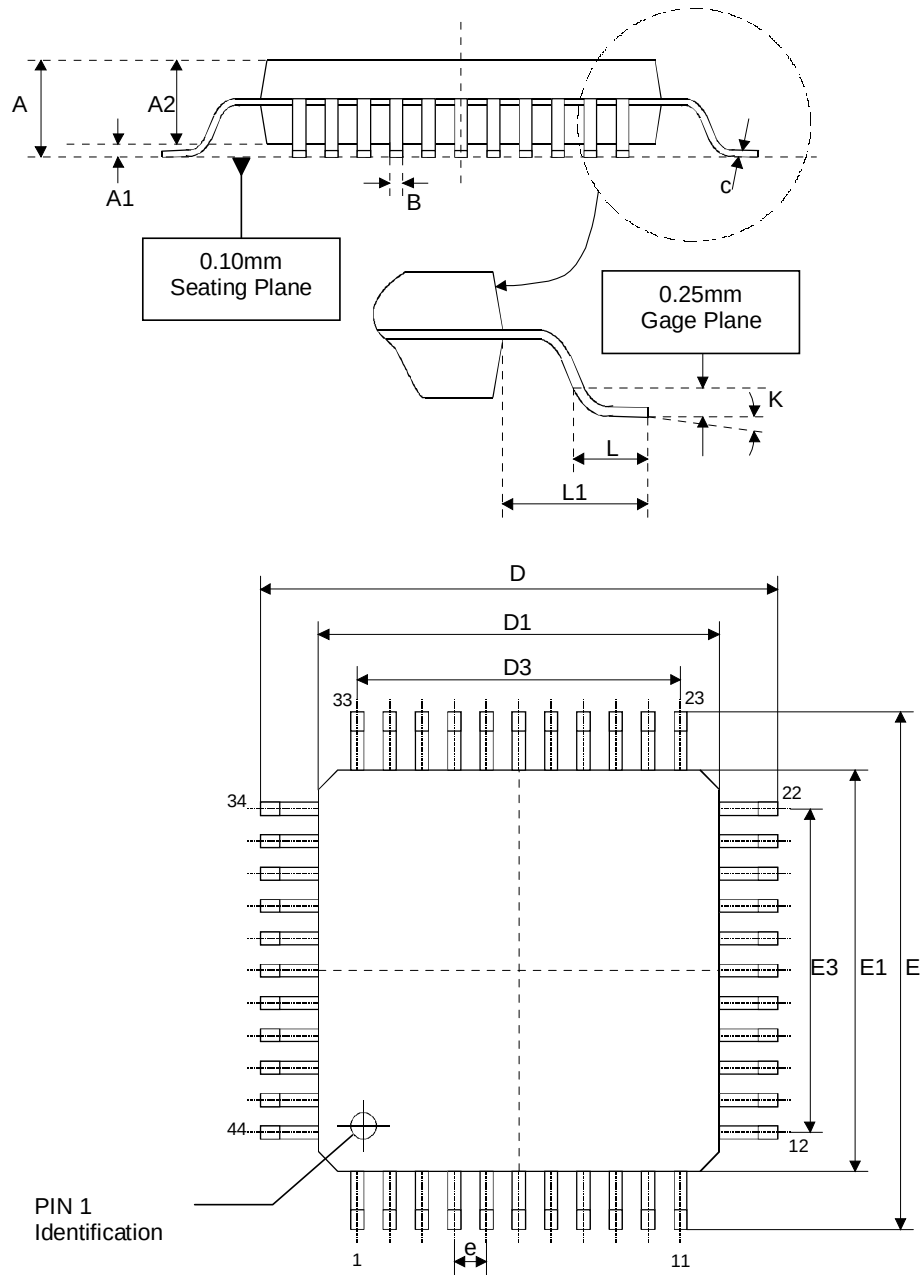


Figure 11-8: JTAG Timing

12 Package Outlines

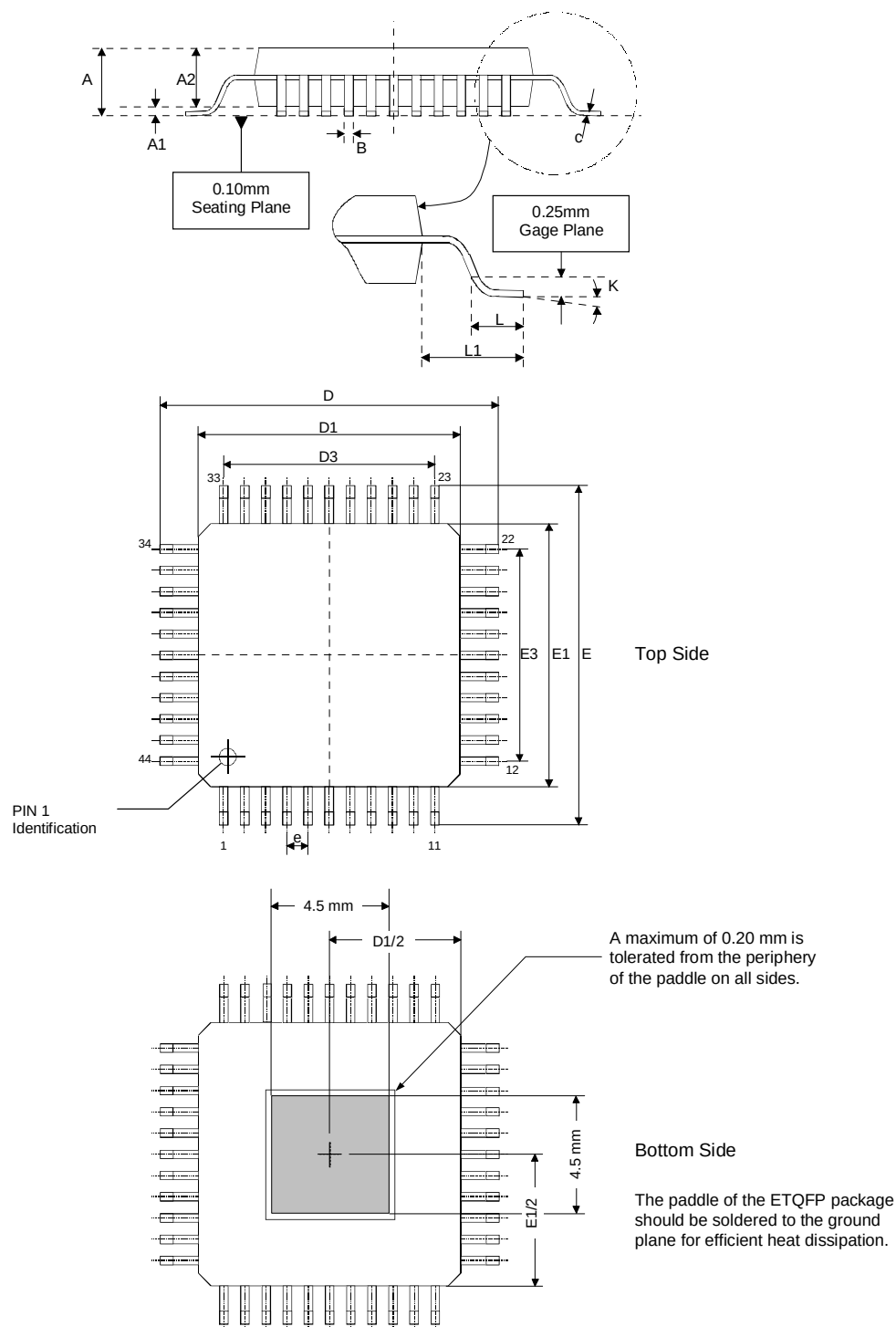
12.1 QFP44



	A	A1	A2	B	c	D	D1	D3	e	E	E1	E3	L	L1	K
Min		0.05	1.35	0.30	0.09	11.80	9.80			11.80	9.80		0.45		0°
Typ			1.40	0.37		12.00	10.00	8.00	0.80	12.00	10.00	8.00	0.60	1.00	3.5°
Max	1.60	0.15	1.45	0.45	0.20	12.20	10.20			12.20	10.20		0.75		7°

Table 12-1: Package Outline Dimensions in mm and degrees

12.2 ETQFP44



	A	A1	A2	B	c	D	D1	D3	e	E	E1	E3	L	L1	K
Min		0.05	0.95	0.30	0.09	11.80	9.80			11.80	9.80		0.45		0°
Typ			1.00	0.37		12.00	10.00	8.00	0.80	12.00	10.00	8.00	0.60	1.00	
Max	1.20	0.15	1.05	0.45	0.20	12.20	10.20			12.20	10.20		0.75		7°

Table 12-2: Package Outline Dimensions in mm and degrees

13.1.2 Internal Regulator

Figure 13-2 illustrates a typical power arrangement for the OS81050, when the 2.5 V supply is linearly regulated by the **BIAS** pin. When the transistor is turned on, it draws from the 3.3 V supply to provide 2.5 V regulated power to the core and analog sections of the device. In this arrangement, the 3.3 V power should be supplied by a switching regulator for maximum efficiency. The switching regulator frequency should be as far from the audio sample frequency as possible. For **VDDP_n** and **VDDA₂** pins, use 0.1 μ F decoupling capacitors; for **VDDC_n** and **VDDA₁** pins, use 0.01 μ F decoupling capacitors.

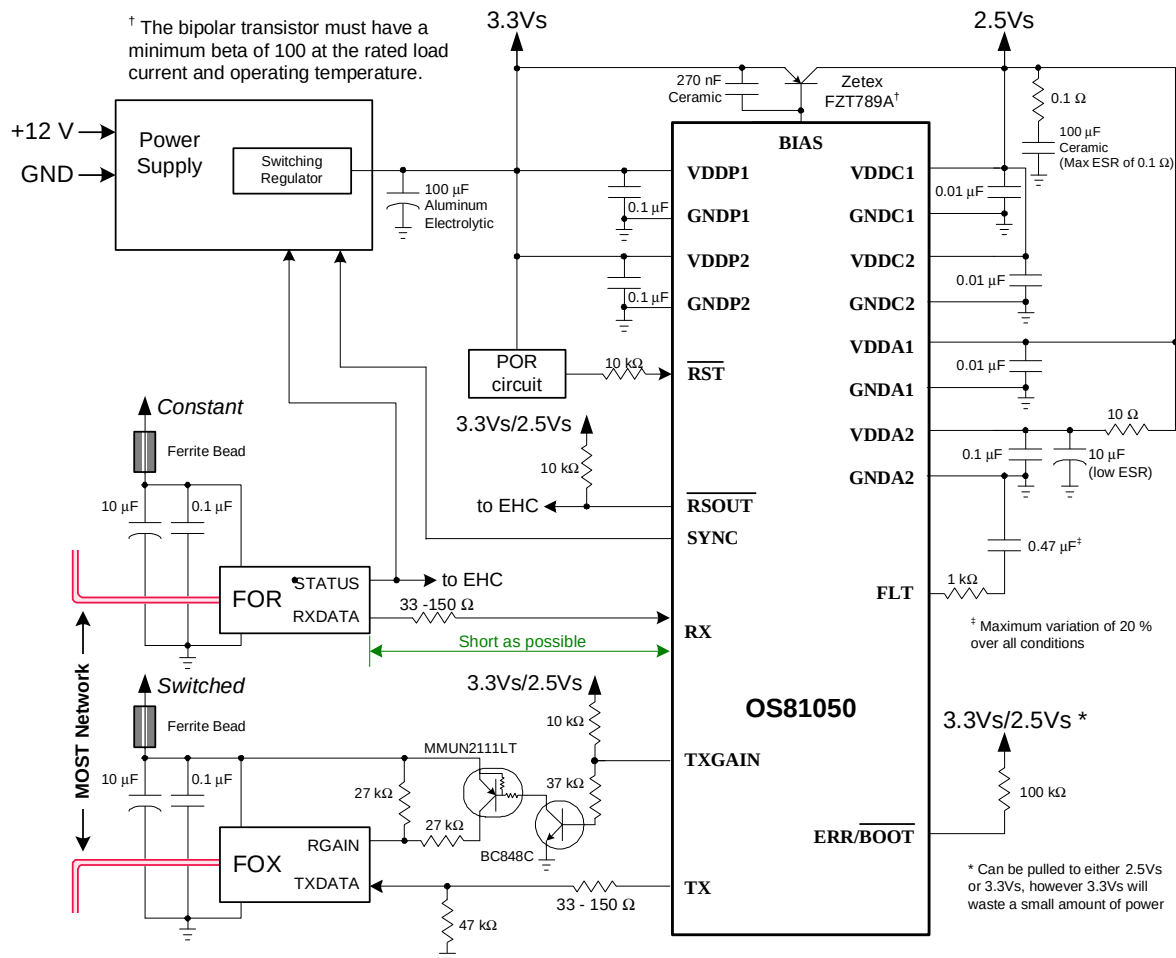


Figure 13-2: Typical Power Supply Connection Diagram (with Internal Regulator)

When this internal regulator power arrangement is used, the **BIAS** pin voltage is between **VDDP_n** and **GND_n**. Connecting other board components to the 2.5 V supply is not advised, with the exception of necessary level-shifters. When used, the current draw on the 2.5 V supply from board level-shifters should be limited to 10 mA. Figure 13-3 illustrates the **BIAS** circuitry arrangement.

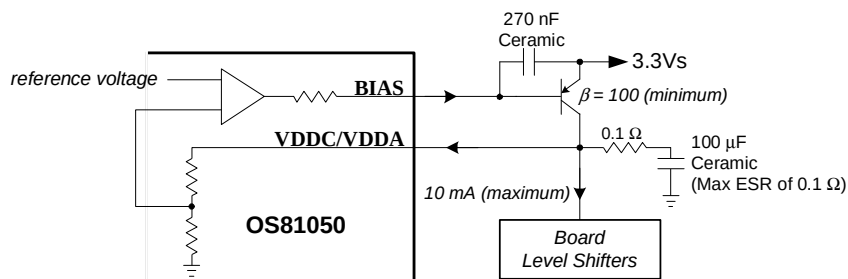


Figure 13-3: BIAS Internals

13.2 Physical Layer

The distance between the Fiber Optic Receiver (FOR) unit and the OS81050 should be as short as possible to minimize capacitance on the receiving data line (**R_X**). Minimizing capacitance shortens transition times out of the FOR, thereby minimizing pulse width distortion and jitter.

Series resistors in the **TX** and **R_X** paths should be used for series-termination and should be as close as possible to the transmitting end. Resistor values of 33-150 Ω are typical; however, the actual series resistor value should be selected based on the board layout and should match the line impedance, which will help minimize reflections and lower EMI. The

Figure 13-2 illustrates a typical optical power control circuit for the Fiber Optic Transmitter (FOX), connected to the **TXGAIN** pin. The INIC **TXGAIN** line should be isolated from other signals that may control FOT power (e.g. do not wire-or with other open-drain lines).

13.3 Reset

After both the 3.3 V and 2.5 V power supplies have stabilized, the OS81050 must be reset (**R_{ST}** low) for normal operation. A proper reset is when both power supplies are $\geq 90\%$ VDD; however, electrical specifications are applicable only within $\pm 5\%$ VDD. Once initialized, INIC is typically reset by the EHC only when a fatal communication error occurs, or when performing an update of the *INIC Configuration String* (see the *OS8105x/6x INIC API User's Manual* [2]).

If the external POR circuit does not drive the **R_{ST}** line high (e.g. open-drain output), an external pull-up resistor to 3.3 V should be used. If the external POR circuit drives the **R_{ST}** line high and low (e.g. push-pull output), a series resistor should be used between the POR circuit and the OS81050 **R_{ST}** pin in lieu of the pull-up. This series resistor allows INIC to be reset by something other than the POR circuit, such as the EHC or SMSC's *INIC Explorer* [4] tool. The pull-up or series resistor should be at least 10 k Ω .

The **R_{ST}** pulse width should be long enough to allow the external configuration pins (**ERR/BOOT** and **INT**) to achieve their default state (see Section 11.5.2). The transition time of **R_{ST}** is not relevant, provided the noise level does not exceed the $\Delta V_{\text{hys-rst}}$ parameter given in Section 11.5.2.

13.4 Other Application Pins

The **FLT** pin is a high-impedance node; therefore, leakage should be kept below 1 μ A or average pulse-width distortion tolerance could be adversely affected. In order to minimize vibration and shock effects on PLL locking, the capacitor used on **FLT** should be tolerant of vibrations typical in the application. Capacitors with X7R dielectrics are more sensitive to shock than metal-film, tantalum, and other ceramic capacitors, such as C0G and X8R. The tolerance of the **FLT** capacitor must not vary by more than 20 % over all conditions. This includes the tolerance attributed to process, as well as degradation over life and temperature. Since component selection (and layout) of the **FLT** circuitry impact the sensitivity to vibrations, each design should be tested over the full environmental stress conditions of the intended application.

For normal operation, the **ERR/BOOT** pin should be pulled to 2.5 V. In order to support the *Customer Configuration Interface*, the on-board pull-up resistor on **ERR/BOOT** must not be less than 10 k Ω ; however, the actual resistor value should be chosen based on the input leakage of the INIC pin and on the other devices connected to the line. Note that **ERR/BOOT** may be pulled up to 3.3 V, however a small amount of power will be wasted since, after initialization, this pin is an output driven at a 2.5 V level.

The OS81050 **RSOUT** pin is an optional output that allows INIC to reset the EHC. When used, the **RSOUT** pin should be pulled high by an on-board resistor (10 k Ω recommended) and connected to the Reset pin of the EHC. The INIC **RSOUT** line should be isolated from other signals that may reset the EHC (e.g. do not wire-or with other open-drain lines). INIC drives **RSOUT** low to reset the EHC when one of two events occurs:

- the `Reset.Start(Target)` INIC function is called and the `Target` parameter specifies the EHC as the device to be reset, or
- the watchdog timer expires and the `WatchdogMode()` INIC function indicates the EHC should be reset.

The length of the reset pulse on **RSOUT** is 10 ms. For more information on the **RSOUT** pin, refer to the *OS8105x/6x INIC API User's Manual* [2].

When an external crystal is used, the **XTI** and **XTO** pins should be connected directly to the crystal (e.g. no series resistance). The crystal used must be 256x Fs and should comply with the specifications outlined in Section 13.6. If an external clock is used in lieu of a crystal oscillator, it must be connected to **XTI** and support drive levels to the 2.5 V supply (as specified in Section 11.4).

The JTAG Port pins **TMS**, **TCK**, **TDI**, and **TDO** require pull-up resistors to 3.3 V, regardless of whether or not the JTAG port is used.

When the 2.5 V OS81050 digital pins interface with external logic, level-translation may be required if the external logic is controlled by a different supply voltage.

13.5 Configuration and Debug

The OS81050 provides internal memory space for storing power-up configuration settings. The initial power-up configurations are collectively referred to as the *INIC Configuration String* (or *ConfigString*). The *INIC Configuration String* includes initial hardware configuration information such as:

- Control Port I²C address
- Default port for INIC-EHC communication
- MediaLB Port default mode and speed
- RMCK configuration, and
- Network device mode (timing-master/timing-slave).

The *INIC Configuration String* is preloaded with the OS81050 Factory default settings and only needs to be programmed when the Factory defaults are not adequate for the application. Refer to the *OS8105x/6x INIC API User's Manual* [2] for the OS81050 Factory defaults, as well as a complete listing of *INIC Configuration String* settings.

The *INIC Configuration String* can be programmed by the EHC via the Control Port. This approach is suitable for production-level applications. See Section 13.5.1 for more information. Alternatively, SMSC's *INIC Explorer* [4] tool can program configuration data via the *Customer Configuration Interface*. This technique is suitable for development-level applications. See Section 13.5.2 for more information.

13.5.1 EHC via Control Port

The Control Port can be used by the EHC to program the *INIC Configuration String* or update the OS81050 Flash memory. To support this feature, the EHC must be connected to the INIC Control Port (*SDA*, *SCL*, and *INT*) and be capable of controlling the *ERR/BOOT* and *RST* pins. The required connections are shown in Figure 13-4.

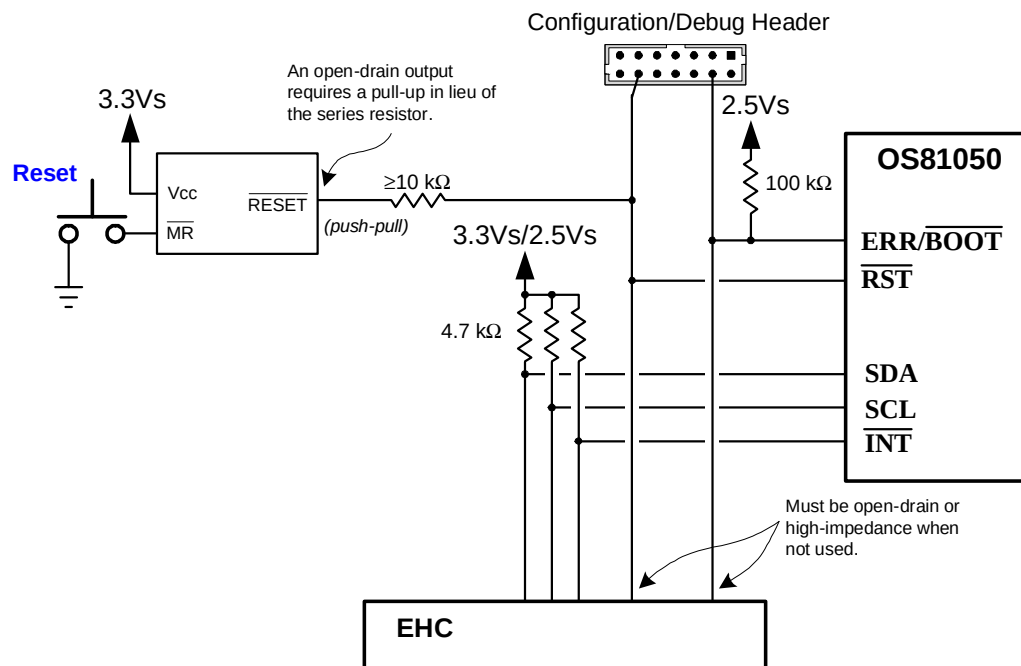


Figure 13-4: Control Port Interface Programming

The sequence required of the EHC, in order to program the OS81050 INIC, is as follows:

- Hold the OS81050 in reset (drive $\overline{\text{RST}}$ low),
- Hold the $\text{ERR}/\overline{\text{BOOT}}$ pin low,
- Release the OS81050 from reset ($\overline{\text{RST}}$ high),
- Erase and program OS81050 INIC, as described in the *INIC Flash Guide Application Note* [10],
- Release the $\text{ERR}/\overline{\text{BOOT}}$ pin, and
- Reset the OS81050 to resume a normal mode of operation.

During *Boot Mode*, the $\overline{\text{INT}}$ pin is configured as an output and driven to VDDCn .

13.5.2 INIC Explorer via JTAG Port

The OS81050 JTAG Port can be converted into a configuration interface for use with SMSC's *INIC Explorer* [4] tool. *INIC Explorer* required that the application PCB provide a standard 14-pin (2x7) 2 mm header (such as Molex 8732-1420 or equivalent). The interface between the *INIC Explorer* tool and the OS81050 is also referred to as the *Customer Configuration Interface*.

In addition to writing the *INIC Configuration String* into Flash memory, *INIC Explorer* provides debugging capabilities by allowing users access to INIC (via PC software) for:

- Exploring internal INIC properties and states while it is operating in the target platform,
- Viewing internal INIC states and routing configurations graphically, and
- Creating an INIC data memory snapshot as a file dump.

Figure 13-5 illustrates the necessary connections between the OS81050 and the configuration/debug header that accommodates *INIC Explorer*. To allow the header to drive the $\text{ERR}/\overline{\text{BOOT}}$ and $\overline{\text{RST}}$ pins during configuration, the pull-up resistors on the lines should be greater than 10 k Ω . If the $\text{ERR}/\overline{\text{BOOT}}$ and $\overline{\text{RST}}$ lines are also connected to the EHC (as described in Section 13.5.1), the EHC must use open-drain logic or set its outputs as high-impedance when the *INIC Explorer* connects to the OS81050 via the configuration/debug header.

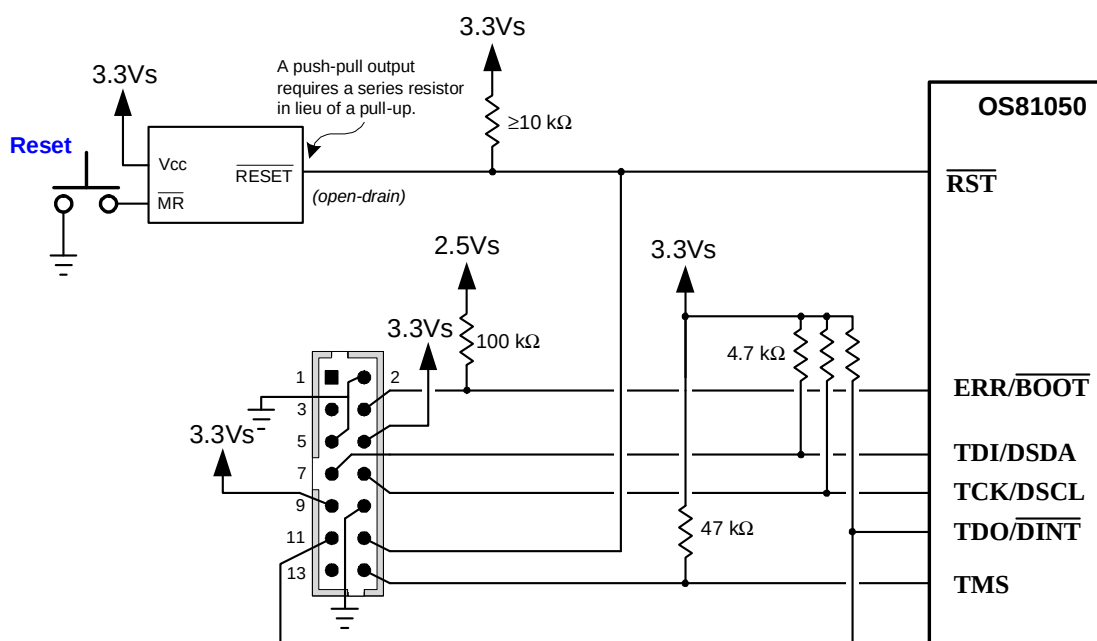


Figure 13-5: Configuration/Debug Header

Contact SMSC for more information on the *INIC Explorer* platform.

13.6 Crystal Oscillator Selection

Several factors must be considered when selecting a crystal. These include load capacitance, oscillator margin, cut, and operating temperature.

Oscillator margin is a measure of the stability of an oscillator circuit, and is defined as the ratio of the oscillator's negative resistance (R_{NEG}) to the crystal's ESR (R_{ESR}), or:

$$\text{Margin} = \frac{|R_{NEG}|}{R_{ESR}} = \frac{|R_{VAR}| + R_{ESR}}{R_{ESR}}$$

The negative resistance can be measured by placing a variable resistor (R_{VAR}) in series with the crystal and finding the largest resistor value where the crystal still starts up properly. This point would be just below where the oscillator does not start-up or where the start-up time is excessively long.

Ideally, oscillator margin should be greater than 10, and should be at least 5. Smaller oscillator margin can affect the ability of the oscillator to start up.

The load capacitance, specified when ordering the crystal, is the series combination of the capacitance on each leg of the crystal. This capacitance includes not only the added capacitors, but also PCB trace (shunt) capacitance and chip pin capacitance. Larger capacitors also have a negative effect on oscillator margin. External capacitors on each leg (C_1 and C_2 in Figure 9-3) are typically in the 12 to 22 pF range.

Name	Value	Description
Frequency	256×Fs	Frequency relative to operating network frame rate (Fs)
Correlation	Parallel Resonant	Mode of oscillation
Osc. Mode	Fundamental	Oscillation mode or operation mode
C_L	12-22 pF	Load capacitance (typical)
C_O	7 pF	Recommended maximum shunt (parallel) capacitance
ESR	100 Ω	Recommended maximum equivalent series resistance
Drive Level	50 μ W	Drive level (typical)
Cut	AT	AT cut produces the best temperature stability
Tolerance	±200 ppm	Frequency tolerance over all conditions (typical)

Table 13-1: Crystal Oscillator Specifications

The crystal cut and tolerance value listed in Table 13-1 are *typical* values and may be changed to suit differing system requirements. Higher ESR values (than those listed in Table 13-1) run the risk of having start-up problems and should be thoroughly tested before being used. Contact the crystal manufacturer for more information.

13.7 Layout Guidelines

INIC board designs should follow the basic guidelines outlined throughout this chapter, with special consideration given to capacitor selection and placement. Figure 13-6 shows an OS81050 pin subset, with components typical for an OS81050 application using the external regulator power arrangement.

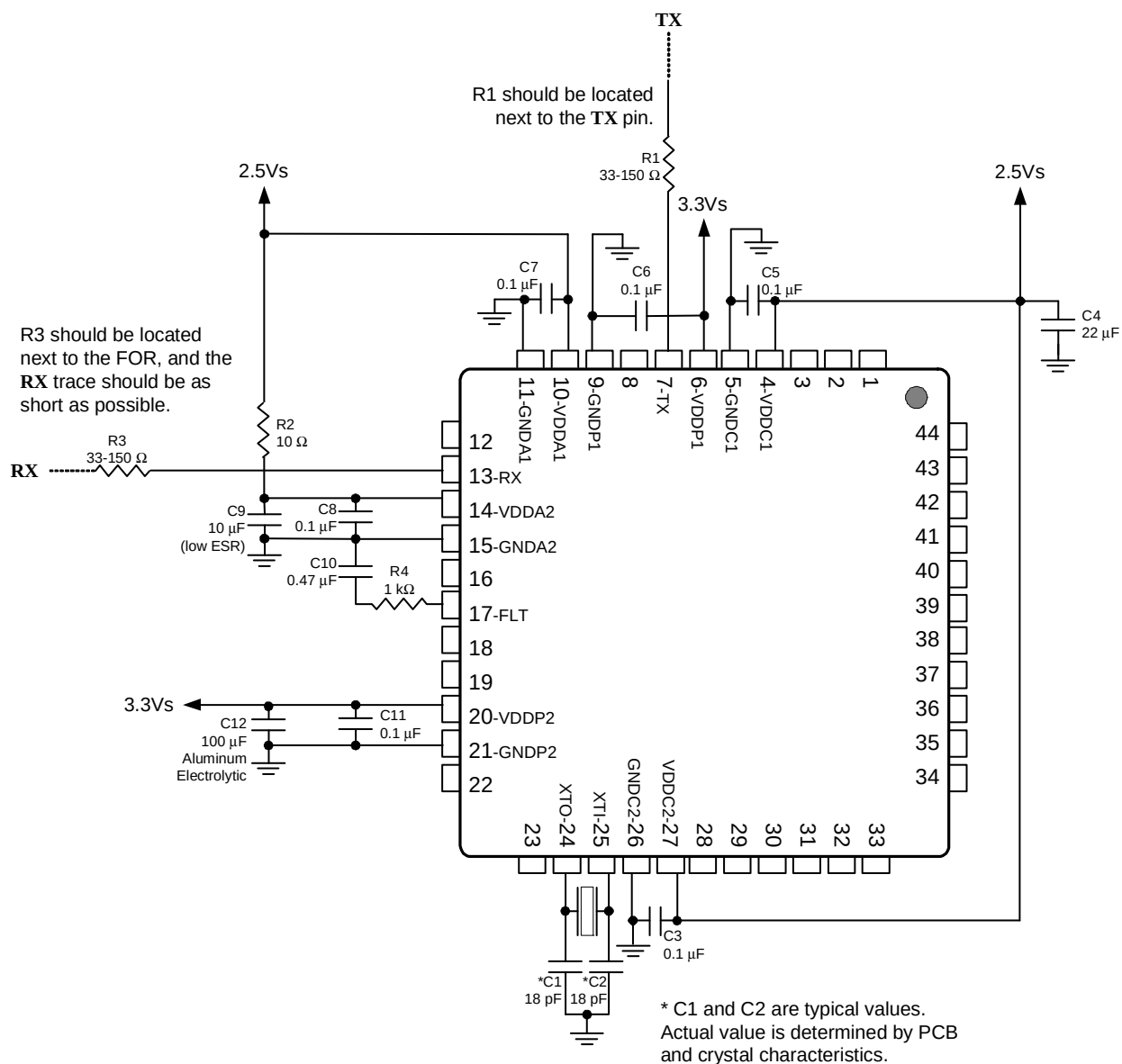


Figure 13-6: Example Layout - Reference Schematic

Reference designators used in Figure 13-6 correspond to components shown in the placement diagram shown in Figure 13-7 on page 72.

Figure 13-7 shows a recommended layout for the OS81050 QFP package. This figure demonstrates appropriate component placement and signal routing for typical INIC applications. Specific recommendations are listed in the subsections that follow.

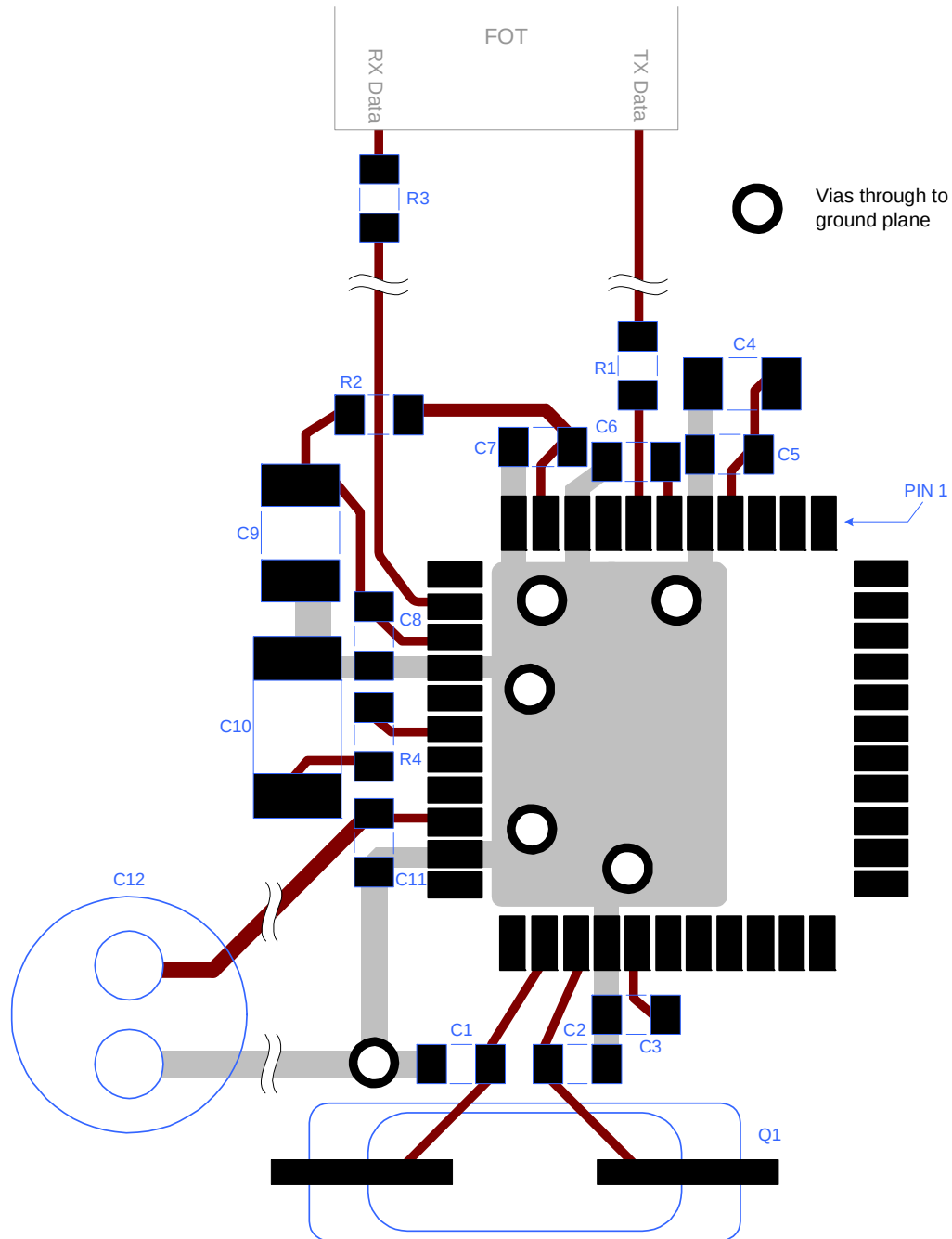


Figure 13-7: Example Layout - Placement Diagram

13.7.1 Power and Decoupling

A decoupling capacitor should be used at each OS81050 power pin for localized decoupling. For EMI-sensitive applications, COG capacitors are strongly recommended for their superior performance at high frequencies. To reduce trace impedance and keep the loop areas small, decoupling capacitors should be positioned as close as possible to the OS81050. Two vias to the ground plane should be used to minimize inductance.

The recommended placement of decoupling capacitors is on the same side of the PCB as the OS81050. However, decoupling capacitors may be mounted on the opposite side of the PCB to reduce the distance between the capacitor and the pin, as inductance from vias is less harmful than long traces that may allow external interference.

In situations where both large (e.g. electrolytic or tantalum) and small (e.g. ceramic) capacitors are used, the smaller capacitor should take priority for placement and layout optimization.

As in all low-noise circuits, two vias should be used from power and ground traces to the respective power and ground planes to reduce parasitic inductance, whenever possible. Additionally, power traces should be as wide as practical and all OS81050 ground pins (digital and analog) should be tied to the same ground plane.

Ground-plane fill is used under the part to minimize the impedance between ground pins, which also helps minimize EMI. If ground-plane fill must be broken by a trace, multiple stitching vias should connect each section of the fill to a reference ground plane on another layer. Vias that connect high frequency decoupling capacitors to the ground plane should be as close as possible to the OS81050 to minimize loop area back to the chip ground pins.

13.7.2 Physical Layer

For optimum performance, careful attention should be given to the routing and placement of physical layer components. The transmit and receive lines between INIC and the FOT are single-ended transmission lines which must be implemented on the PCB as impedance controlled traces (e.g. 50 Ω).

When possible, the transmit and receive lines should be routed on a single layer and the use of vias should be minimized. Additionally, traces should be routed over a continuous reference plane, preferably a ground plane. To minimize reflections, two 45 degree turns should be used rather than a single 90 degree turn. Crosstalk can be avoided by not running other signals in parallel to the transmit and receive lines for long distances.

The distance between the OS81050 and the FOT should be as short as possible. The series termination resistors should be placed as close as possible to the driving source (e.g. the INIC TX pin and the FOT RX Data pin). Ground plane fill should not be too close to the RX data path, as it adds capacitance which can negatively impact PWD performance.

13.7.3 MediaLB Interfaces

The OS81050 *Media Local Bus* (MediaLB) 3-pin and 5-pin interfaces are provided to facilitate high-speed data exchange between INIC and the EHC. MediaLB is intended for use on a single PCB. For effective operation, especially at higher bus speeds, signal routing and component placement is extremely important.

Refer to the *MediaLB Specification* [3] for application guidelines, including layout examples.

13.7.4 Thermal Considerations

When the OS81050 ETQFP package is used, the paddle should be soldered to the thermal land (thermal pad) for efficient heat dissipation. The thermal pad should be at least as large as the exposed portion of the package paddle and allow at least 2 mm of clearance from the pad to the device leads. Additionally, the thermal pad should be connected to the ground plane of the board using heat tubes (vias without thermal reliefs). At least nine heat tubes are recommended, with a drill hole size no larger than 0.33 mm (13 mils).

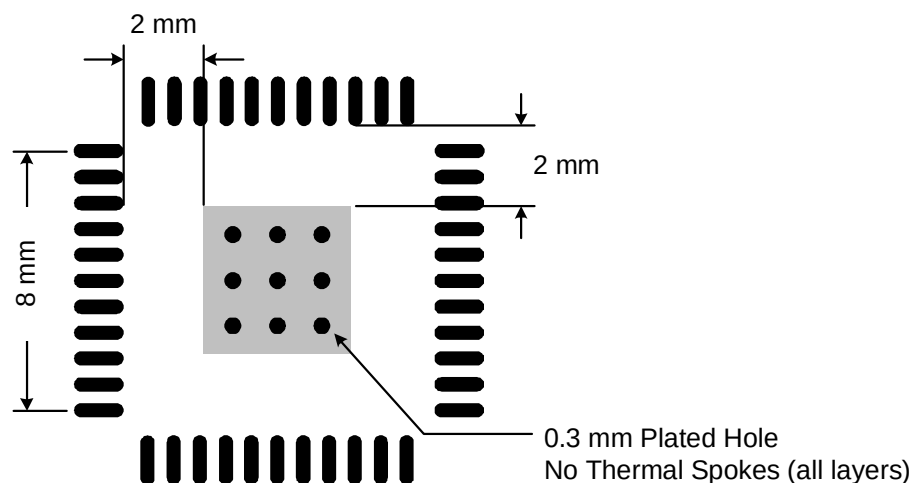


Figure 13-8: Example Layout - Drill Holes (for ETQFP Package)

13.7.5 Miscellaneous

Floating areas of copper fill near INIC should be avoided due to the risk of noise coupling into the system. If needed, a keep-out can be used around the OS81050 and its immediate peripheral circuitry (e.g. decoupling capacitors and external crystal) to ensure islands are not created when copper fill is poured. If a keep-out is not used, multiple stitching vias should tie any floating copper fills to an appropriate reference on other power or ground layers.

The **FLT** analog components should be placed as close as possible to the **GNDAn** pins to minimize loop currents. Conformal coating is recommended for systems where condensation can occur.

The **XTI** and **XTO** traces should be matched in length. Additionally, the distance between INIC and the external crystal should be minimized.

Appendix A: References

- [1] MOST Specification
MOST Cooperation.
Contact contact@mostcooperation.com
- [2] OS8105x/6x INIC API User's Manual
SMSC.
Contact support-ais-de@smc.com
- [3] MediaLB Specification
SMSC. Rev. 4.1 (2009 Sept.).
<http://www.smc-ais.com>
- [4] INIC Explorer Interface Box
SMSC.
Contact support-ais-de@smc.com
- [5] I2C-Bus Specification
NXP (formerly a division of Philips).
<http://www.nxp.com>
- [6] MediaLB Monitor Adaptor User's Manual
SMSC.
Contact support-ais-de@smc.com
- [7] MediaLB Analyzer User's Manual
SMSC.
Contact support-ais-de@smc.com
- [8] MOST INIC Hardware Concepts Technical Bulletin
SMSC.
Contact support-ais-de@smc.com or contact@mostcooperation.com
- [9] IEEE Std. 1149.1 "IEEE Standard Test Access Port and Boundary-Scan Architecture"
Institute of Electrical and Electronics Engineers (IEEE).
<http://www.ieee.org>
- [10] INIC Flash Guide Application Note
SMSC.
Contact support-ais-de@smc.com

Appendix B: Revision History

The most extensive and pertinent application changes are listed in Table B-1, although various other differences may be observed between document revisions.

	Description of Changes
DS81050AP11:	
General	- Clarification of application use cases for D _{OUTD} pins ($\overline{\text{INT}}$, $\overline{\text{RSOUT}}$, TXGAIN).
Electrical Characteristics	- Corrected Note 2 in Section 11.9 to accurately specify the edge of SCK at which Streaming Port data is valid.
Applications Information	-Restructured chapter to provide clarity and/or additional application information, as needed.
DS81050AP10:	
General	- Clarification of $\overline{\text{INT}}$ state during <i>Boot Mode</i> .
DS81050AP9:	
Various format and content changes have been made throughout the document to conform to an updated SMSC data sheet template.	
General	- Changed document confidentiality marker from <i>Restricted Access</i> to <i>Normal</i>. - Restructured document for clarity and to provide additional information, as needed. Includes: - Addition of Feature List, General Description, and Block Diagram to Cover Page - Addition of <i>MOST25 Network</i> section to the <i>Overview</i> chapter - Addition of <i>Hardware Ports</i> section to the <i>Overview</i> chapter - Addition of dedicated <i>Pinout</i> chapter - Addition of <i>Generic Port Messages</i> section to the <i>Port Message Protocol</i> chapter - Relocation of <i>Pinout List</i>, <i>Pinout</i>, and <i>Equivalent Schematics for Pins</i> sections from <i>Packaging and Pinout</i> chapter to dedicated <i>Pinout</i> chapter.
Pinout	- Clarified $\overline{\text{INT}}$ pin behavior by separating descriptions of D_{IN} and D_{OUTD} functionality - Clarified Network Port operation by removing references to unsupported functionality: - Renamed TXP/TX pin to TX - Renamed TXN/TXGAIN pin to TXGAIN - Renamed RXN pin to NC - Renamed RXP/RX pin to RX
MediaLB Port	- Changed recommendation for MLBCLK AC-parallel termination to optional
Streaming Port	- Added Full Streaming Mode vs. MediaLB 5-pin Mode Diagram (Figure 7-1) - Updated Partial Streaming Mode Diagram (Figure 7-2) with complete pin names - Removed Streaming Port Typical Connection Diagram
Ext. Pwr. Management	- Updated Typical Power Management Diagram (Figure 8-1)
Clock Manager	- Added body note in <i>PLL Lock Status</i> section to clarify device reference clock requirements
JTAG Port	- Added Boot Mode as a prerequisite for accessing JTAG Port
Electrical Characteristics	- Added specification for $\Delta V_{\text{hys-rst}}$ ($\overline{\text{RST}}$ Input Hysteresis) to Section 11.5.2 - Corrected minimum f_{sck} specification in Section 11.9
Application Information	- Added pull-down resistor on TX in Typical Application Diagrams (Figure 13-1 and Figure 13-2) - Changed maximum recommended ESR for crystal oscillators in Table 13-1 - Added maximum shunt capacitance recommendation for crystal oscillators in Table 13-1

Table B-1: Data Sheet Revision Summary

Appendix C: List of Figures

Figure 1-1:	INIC Hardware/System Overview	7
Figure 1-2:	INIC-IOC Application Example	12
Figure 1-3:	Division of Services between INIC and the EHC	14
Figure 2-1:	OS81050 Pinout	18
Figure 2-2:	Pin-equivalent for Analog I/O pin - AIO.....	19
Figure 2-3:	Pin-equivalent for Digital Input pin - DIN.....	19
Figure 2-4:	Pin-equivalent for Digital Output pin - DOUT.....	19
Figure 2-5:	Pin-equivalent for Open-Drain Digital Output pin - DOUTD.....	19
Figure 2-6:	Pin-equivalent for Digital Output pin with high-Z control - DOUTZ	20
Figure 2-7:	Pin-equivalent for Digital I/O pin - DI/O.....	20
Figure 2-8:	Pin-equivalent for Digital Input/Open-Drain Output pin - DI/OD	20
Figure 3-1:	INIC Software Stack	21
Figure 4-1:	Generic Port Message Format.....	23
Figure 4-2:	FIFO Status and Command Message Format.....	24
Figure 4-3:	FIFO Read and Write Message Format.....	25
Figure 5-1:	Control Port Pin Connections	28
Figure 5-2:	Control Port Block Diagram	29
Figure 5-3:	Control Port Write Sequence	29
Figure 5-4:	Control Port Read Sequence.....	30
Figure 6-1:	MediaLB 3-pin Mode Connection Diagram - Example.....	32
Figure 6-2:	MediaLB Data Structure	33
Figure 6-3:	MediaLB 3-pin Mode to Parallel Interface.....	35
Figure 6-4:	MediaLB Interface (at 256xFs)	36
Figure 6-5:	MediaLB 5-pin Mode Two-Node Connection Diagram	37
Figure 6-6:	MediaLB 5-pin Mode Four-Node Connection Diagram.....	38
Figure 6-7:	MediaLB Debug Header	39
Figure 7-1:	Full Streaming Mode vs. MediaLB 5-pin Mode	40
Figure 7-2:	Partial Streaming Mode	41
Figure 7-3:	Left-Justified Streaming Data Format	43
Figure 7-4:	Right-Justified Streaming Data Format.....	43
Figure 7-5:	Delayed-Bit Streaming Data Format.....	44
Figure 7-6:	Sequential Streaming Data Format	44
Figure 8-1:	Typical Power Management Diagram.....	46
Figure 8-2:	Power Management Startup	46
Figure 9-1:	Clock Manager.....	47
Figure 9-2:	FLT Pin Circuit	48
Figure 9-3:	Crystal Oscillator Input.....	48
Figure 11-1:	Initial Power-Up and Reset Timing	54
Figure 11-2:	RX Pulse Width Distortion Timing.....	55
Figure 11-3:	RX Jitter Tolerance Timing	55
Figure 11-4:	MediaLB 3-pin Timing	56
Figure 11-5:	5-Pin MediaLB Timing	58
Figure 11-6:	Streaming Port Timing	59
Figure 11-7:	Control Port Timing.....	60
Figure 11-8:	JTAG Timing	61
Figure 13-1:	Typical Power Supply Connection Diagram (with External Regulator).....	64
Figure 13-2:	Typical Power Supply Connection Diagram (with Internal Regulator)	65
Figure 13-3:	BIAS Internals.....	65
Figure 13-4:	Control Port Interface Programming	68
Figure 13-5:	Configuration/Debug Header	69
Figure 13-6:	Example Layout - Reference Schematic	71

Figure 13-7: Example Layout - Placement Diagram	72
Figure 13-8: Example Layout - Drill Holes (for ETQFP Package).....	74

Appendix D: List of Tables

Table 1-1:	Hardware Port Data Transport.....	10
Table 2-1:	Pinout List.....	15
Table 6-1:	Available MediaLB Bandwidth	33
Table 7-1:	Streaming Port vs. MediaLB Port Availability	40
Table 7-2:	Streaming Data Formats.....	42
Table 8-1:	External Power Management States	45
Table 10-1:	JTAG IR Register.....	50
Table 10-2:	IDR Register	50
Table 12-1:	Package Outline Dimensions in mm and degrees.....	62
Table 12-2:	Package Outline Dimensions in mm and degrees.....	63
Table 13-1:	Crystal Oscillator Specifications	70

NOTES: